

C H E B Y S H E V
COLLOCATION METHOD
WITH APPLICATIONS
IN GEOPHYSICS

— PRINCIPLE AND PROGRAMS

JUNYI GUO

WUHAN TECHNICAL UNIVERSITY OF SURVEYING AND MAPPING PRESS

Chebyshev Collocation Method
with Applications in Geophysics
——Principle and Programs

Junyi Guo

Wuhan Technical University of Surveying and Mapping Press

(鄂)新登字 14 号

图书在版编目(CIP)数据

切比雪夫配点法及其在地球物理中的应用:原理和程序:英文/郭俊义著.
武汉:武汉测绘科技大学出版社,2001.1
ISBN 7-81030-798-3

I.切… II.郭… III.切比雪夫逼近-配点法-应用-地球物理-英文
IV.P3

中国版本图书馆 CIP 数据核字(2000)第 85243 号

责任编辑:莫妮 封面设计:曾兵
武汉测绘科技大学出版社出版发行
(武汉市珞喻路 129 号,邮编 430079)
武汉理工大学出版社印刷厂印刷

*

开本:787×1092 1/16 印张:7.75 字数:198 千字
2001 年 1 月第 1 版 2001 年 1 月第 1 次印刷
印数:001—300 册 定价:60.00 元

Preface

Several geophysical problems could be reduced to ordinary differential equations depending on geocentric distance r . These equations are often singular at the Earth center. Typical examples of such equations are Clairaut's equation governing Earth's internal flattening and Earth's free oscillation equations.

In numerical solution of the geophysical ordinary differential equations singular at the Earth center using traditional methods such as the Runge-Kutta method, special treatment must be taken for the Earth center. Normally this is done in two ways. One is to consider a small sphere at the Earth center as homogeneous and solve the equations in this sphere analytically, the other is to represent the unknown functions near the Earth center as exponential functions of r defined by a few coefficients and solve for these coefficients from the equations. The Runge-Kutta method could begin to be used only at a certain non-zero value of r .

In this report, the Chebyshev-collocation method is used to solve a series of geophysical ordinary equations. The principle is to represent the unknown functions as a truncated series of the orthogonal Chebyshev polynomials and to reduce the differential equations to a system of algebraic equations according to the collocation method of approximation. This method could not only give very accurate result, and also handle the singularity of the differential equations very easily without any further development.

The programs given are in C with a few features of C++ such as reference type, inline function, overloading function, function template and some I/O functions.

The programs are designed for two purposes. One is to provide with the reader a background for further researches for the solution of similar ordinary differential equations coming from their own problems, the other is to provide with the reader a toolkit for the problems solved in this report. For this last purpose the physical parameters given in Earth models as well as that computed in this report are output in a 3 kilometers interval form, i.e. their values are given at every 3 kilometers of radius inside the Earth. The values of any physical parameters for any values of r could be simply obtained by interpolation using this 3 kilometers interval model. The linear interpolation is practically sufficient.

Included in the report are the programs for the following problems.

- Computation of some internal physical parameters determined by density, including mean density inside any co-central sphere, gravity and pressure inside the Earth, flattening of any strata etc.
- Computation of Love numbers and loading Love numbers.
- Computation of toroidal and spheroidal free oscillation periods and proper functions.

More features will be added in the future.

The CD-ROM bound with the report contains the programs. The programs are tested with Microsoft Visual C++ 6.0 and Borland C++ builder 5.0 under Windows system, and GNU g++ 2.95 under Linux systems. For users of Visual C++ the Microsoft Developer Studio Workspace Files are provided. For users of Borland C++ Builder the Builder Project Files are provided. And for users of GNU g++, Makefiles are provided. The Makefiles can be modified before running `make` to adapt to other compilers in use. Please read the README files for a guide of compiling and running the programs.

The author would like to express his thanks to Professor M. Deville¹ from whom he learnt the Chebyshev-collocation method. The author would also like to thank Dr. Fei-Peng Zhang² who made the first tentative for solving singular differential equations in geophysics using the Chebyshev-collocation method while he was working on his Master degree with the author.

This report is a part of the results of two scientific research programs supported by the Natural Science Foundation of China under grants 49204050 and 49874003.

¹Professor M. Deville was formally professor at the Institute of Applied Mechanics, Catholic University of Louvain, Louvain-La-Neuve, Belgium. He is now professor at the Laboratory for fluid mechanics, Federal Polytechnical School of Lausanne, Swiss.

²Now working at Shanghai Astronomical Observatory.

Contents

Preface	II
1 Chebyshev collocation method: Principle	1
1.1 General orthogonal polynomials	1
1.2 Gauss-type integral formulas	2
1.3 Collocation approximation of functions	3
1.4 Chebyshev polynomials	4
1.5 Differentiation formula	5
1.6 Integration formula	8
1.7 Linear differential equations without singularity	9
1.8 Linear differential equations with singularity	11
1.9 Combination of differential and algebraic equations	14
2 Chebyshev collocation method: Programming	15
2.1 Introduction	15
2.2 Basic utility programs	16
2.2.1 <code>parameters.h</code>	16
2.2.2 <code>datatype.h</code>	16
2.2.3 <code>nrutil.h</code> and <code>nrutil.cpp</code>	17
2.2.4 <code>nrinterp.cpp</code>	19
2.2.5 <code>nrludcmp.cpp</code>	20
2.2.6 <code>nrgaussj.cpp</code>	21
2.2.7 <code>nrbisec.cpp</code>	21
2.2.8 <code>nr.h</code>	22
2.3 Utility programs for Chebyshev collocation method	22
2.3.1 Computation of collocation coordinates for $t \in [a, b]$ and $x \in [-1, 1]$.	22
2.3.2 Transformation between physical and spectral spaces	23
2.3.3 Computation of x corresponding to known values of t	24
2.3.4 Computation of function from its Chebyshev polynomial series coefficients for arbitrary values of variable	24
2.3.5 Transformation of Chebyshev polynomial series to polynomial of x .	25
2.3.6 Transformation of polynomial of x to polynomial of t	25
2.3.7 Computation of differential	26
2.3.8 Computation of integrals	26

CONTENTS	V
2.4 Solution of differential equations	26
2.4.1 Inversion by block of the matrix of the algebraic equations deduced from collocation method	27
2.4.2 Solution of linear equations	28
2.4.3 Solution of ordinary differential equations using Chebyshev-Gauss- Lobbato integration points	29
2.4.4 Solution of ordinary differential equations using Chebyshev-Gauss- Radau integration points	42
3 Applications in geophysics	55
3.1 Introduction	55
3.2 Input of Earth model parameters	59
3.3 Computation of an Earth model in 3 kilometers interval	69
3.4 Preparation for use in Chebyshev collocation method	73
3.5 Computation of some internal parameters determined by density	75
3.5.1 Mean density inside any co-central sphere	76
3.5.2 Mass inside any co-central sphere	91
3.5.3 Gravity inside the Earth	92
3.5.4 Pressure inside the Earth	93
3.5.5 Mean moment of inertia of the mass inside any internal stratum of the Earth: $I = (A + B + C)/3$	95
3.5.6 Difference of mean density and density	96
3.5.7 Earth's internal flattening	97
3.5.8 Difference of moments of inertia of the mass inside any internal stratum of the Earth: $C - A$	99
3.6 Earth Tides: Computation of Love numbers for the SNREI Earth	101
3.6.1 Principle for computing Love numbers of static response	101
3.6.2 Principle for computing Love numbers of dynamic response	105
3.6.3 Programming	106
3.7 Free Oscillation of the SNREI Earth	109
3.7.1 Toroidal free oscillations	109
3.7.2 Spheroidal free oscillations	112
A Derivation of the formula (1.56)	115
Bibliography	117

Chapter 1

Chebyshev collocation method: Principle

1.1 General orthogonal polynomials

The Chebyshev polynomials pertain to the class of orthogonal polynomials. So we begin with the general case of orthogonal polynomials.

Let $P_k(x)$ be a polynomial of degree k , and $P_1(x), P_2(x), \dots$ be a polynomial system. If

$$\int_{-1}^1 P_k(x)P_j(x)w(x)dx = 0, \quad j \neq k \quad (1.1)$$

then the polynomial system is called orthogonal in $[-1, 1]$ with respect to the weight function $w(x)$.

For any function $u(x)$ defined in $[-1, 1]$, denote

$$\|u\|_w = \left(\int_{-1}^1 |u(x)|^2 w(x) dx \right)^{1/2} \quad (1.2)$$

which is called norm. If the norm of the function $u(x)$, i.e. $\|u(x)\|_w$ is finite, then the function can be expressed formally in series of the polynomial system $P_1(x), P_2(x), \dots$ as

$$Su = \sum_{k=0}^{\infty} \hat{u}_k P_k(x) \quad (1.3)$$

where the coefficients are defined as

$$\hat{u}_k = \frac{1}{\|P_k\|_w} \int_{-1}^1 u(x) P_k(x) w(x) dx \quad (1.4)$$

For any integral N the truncated polynomial series of $u(x)$ is

$$P_N u = \sum_{k=0}^N \hat{u}_k P_k(x) \quad (1.5)$$

According to the classical Weierstrass theorem the polynomial system $P_1(x), P_2(x), \dots$ is complete, i.e.

$$\lim_{N \rightarrow \infty} \|u - P_N u\|_w = 0 \quad (1.6)$$

1.2 Gauss-type integral formulas

Orthogonal polynomials can be used to compute integrals numerically, called Gauss-type integration.

Gauss integration. Let $x_0, \dots, x_N$ be the $N + 1$ roots of the polynomial $P_{N+1}(x)$, and $w_0, \dots, w_N$ the solution of the linear equation system

$$\sum_{j=0}^N (x_j)^k w_j = \int_{-1}^1 x^k w(x) dx, \quad k = 0, \dots, N \quad (1.7)$$

Then the formula

$$\sum_{j=0}^N P(x_j) w_j = \int_{-1}^1 P(x) w(x) dx \quad (1.8)$$

holds for any polynomial $P(x)$ of which the degree is not higher than $2N + 1$.

This is the most popular Gauss-type integration formula. In this formula all the nodes $x_0, \dots, x_N$ are in the interior of $(-1, 1)$. For adding boundary conditions at one or both ends, this formula needs to be modified to include the related ends as well.

Gauss-Radau integration. Define a polynomial

$$q(x) = P_{N+1}(x) + aP_N(x) \quad (1.9)$$

where a is chosen so that $q(-1) = 0^1$, i.e. $a = -P_{N+1}(-1)/P_N(-1)$. Let $-1 = x_0, \dots, x_N$ be the $N + 1$ roots of the polynomial $q(x)$, and $w_0, \dots, w_N$ the solution of the linear equation system

$$\sum_{j=0}^N (x_j)^k w_j = \int_{-1}^1 x^k w(x) dx, \quad k = 0, \dots, N \quad (1.10)$$

Then the formula

$$\sum_{j=0}^N P(x_j) w_j = \int_{-1}^1 P(x) w(x) dx \quad (1.11)$$

holds for any polynomial $P(x)$ of which the degree is not higher than $2N$.

Gauss-Lobatto integration. Define a polynomial

$$q(x) = P_{N+1}(x) + aP_N(x) + bP_{N-1}(x) \quad (1.12)$$

where a, b are chosen so that $q(-1) = q(1) = 0$. Let $-1 = x_0, \dots, x_N = 1$ be the $N + 1$ roots of the polynomial $q(x)$, and $w_0, \dots, w_N$ the solution of the linear equation system

$$\sum_{j=0}^N (x_j)^k w_j = \int_{-1}^1 x^k w(x) dx, \quad k = 0, \dots, N \quad (1.13)$$

¹ $q(-1) = 0$ may be replaced by $q(1) = 0$

Then the formula

$$\sum_{j=0}^N P(x_j)w_j = \int_{-1}^1 P(x)w(x)dx \quad (1.14)$$

holds for any polynomial $P(x)$ of which the degree is not higher than $2N - 1$. If the polynomial system $P_1(x), P_2(x), \dots$ has Jacobi weight, i.e. $w(x) = (1-x)^\alpha(1+x)^\beta$ with $\alpha, \beta > -1$, then the Gauss-Lobatto points are $-1, 1$ and the roots of $q(x) = \frac{d}{dx}P_N(x)$. The most often used polynomials such as Chebyshev polynomials and Legendre polynomials pertain to this class.

The proof of the first theorem could be found in almost any textbook of numerical analysis, and those of the last two in Canuto et al. (1989). They are neglected here.

For Legendre polynomials, $w(x) = 1$, and they are used to compute integrals numerically as what (1.8), (1.11) and (1.14) give in practice are just $\int_{-1}^1 P(x)dx$. If $P(x)$ does not fulfill the requirement of the theorems, then it is approximated by the polynomials that the theorems claim.

1.3 Collocation approximation of functions

The problem here is to approximate the smooth function $u(x)$ in $[-1, 1]$ using its values at the Gauss type integration points $x_0, \dots, x_N$ by polynomial interpolation. This method of approximation is called collocation method. The derivatives of the function is approximated by analytical derivatives of the interpolation polynomial.

Evidently the interpolation polynomial is of degree N , which will be denoted as $I_N u$. It satisfies

$$I_N u(x_j) = u(x_j), \quad j = 0, \dots, N \quad (1.15)$$

As $x_0, \dots, x_N$ are distinguishable $I_N u$ is uniquely determined. Being a polynomial of degree N , $I_N u$ can be expressed as a truncated polynomial series

$$I_N u = \sum_{k=0}^N \hat{u}_k P_k(x) \quad (1.16)$$

By using (1.15) we can directly write out the formula for computing $u(x_j)$ from $\hat{u}_k$:

$$u(x_j) = \sum_{k=0}^N \hat{u}_k P_k(x_j) \quad (1.17)$$

Its inverse formula can be derived out as follows. Introduce a normalization factor

$$\gamma_k = \sum_{j=0}^N P_k^2(x_j)w_j \quad (1.18)$$

We have, according to the orthogonal formula (1.1) and the relevant Gauss-type integration formulas,

$$\sum_{j=0}^N P_l(x_j)P_k(x_j)w_j = \gamma_k \delta_{km} \quad (1.19)$$

This is the discrete orthogonal formula of orthogonal polynomials. The inverse formula of (1.17) is easily obtained from it as:

$$\hat{u}_k = \frac{1}{\gamma_k} \sum_{j=0}^N u(x_j) P_k(x_j) w_j \quad (1.20)$$

The formulas (1.17) and (1.20) are called discrete polynomial transform pair, with $\langle \hat{u}_0, \dots, \hat{u}_N \rangle$ being called spectral space, and $\langle u(x_0), \dots, u(x_N) \rangle$ physical space.

In spectral method using polynomial approximation, (1.16) is adopted as the approximate expression of functions.

1.4 Chebyshev polynomials

The Chebyshev polynomials are defined as

$$T_k(x) = \cos(k \arccos x) \quad (1.21)$$

By using the recurrence relations of trigonometrical functions we can show the following two recurrence relations:

$$T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x) \quad (1.22)$$

$$2T_k(x) = \frac{1}{k+1} \frac{d}{dx} T_{k+1}(x) - \frac{1}{k-1} \frac{d}{dx} T_{k-1}(x), \quad k \geq 2 \quad (1.23)$$

Some lower degree cases can be computed explicitly:

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_2(x) = 2x^2 - 1, \quad T_3(x) = 4x^3 - 3x, \quad \dots \quad (1.24)$$

A general expression is

$$T_k(x) = \frac{k}{2} \sum_{l=0}^{[k/2]} \frac{(-1)^l (k-l-1)!}{l!(k-2l)!} (2x)^{k-2l}, \quad k > 0 \quad (1.25)$$

where $[k/2]$ is the maximum integral number not greater than $k/2$. A useful orthogonal formula is

$$\int_{-1}^1 \frac{T_k(x) T_j(x)}{\sqrt{1-x^2}} dx = \frac{\pi}{2} c_k \delta_{kj} \quad (1.26)$$

where

$$c_k = \begin{cases} 2, & k = 0 \\ 1, & k > 0 \end{cases} \quad (1.27)$$

$$\delta_{kj} = \begin{cases} 1, & k = j \\ 0, & k \neq j \end{cases} \quad (1.28)$$

If the function $u(x)$ is continuous and $\|u(x)\|_w$ is finite, then it can be expressed in series of Chebyshev polynomials as

$$u(x) = \sum_{k=0}^{\infty} \hat{u}_k T_k(x) \quad (1.29)$$

$$\hat{u}_k = \frac{2}{\pi c_k} \int_{-1}^1 \frac{u(x)T_k(x)}{\sqrt{1-x^2}} dx \quad (1.30)$$

Some other properties are

$$|T_k(x)| \leq 1 \quad (1.31)$$

$$T_k(\pm 1) = (\pm 1)^k \quad (1.32)$$

$$\left| \frac{d}{dx} T_k(x) \right| \leq k^2 \quad (1.33)$$

$$\left[\frac{d}{dx} T_k(x) \right]_{x=\pm 1} = (\pm 1)^k k^2 \quad (1.34)$$

By comparing with the general orthogonal polynomials the weight function for the Chebyshev polynomials is

$$w(x) = 1/\sqrt{1-x^2} \quad (1.35)$$

The parameters related to Gauss type integrations are

Chebyshev-Gauss

$$x_j = \cos \frac{(2j+1)\pi}{2N+2}, \quad w_j = \frac{\pi}{N+1}, \quad j = 0, \dots, N \quad (1.36)$$

Chebyshev-Gauss-Radau

$$x_j = \cos \frac{2\pi j}{2N+1}, \quad w_j = \begin{cases} \frac{\pi}{2N+1} & j = 0 \\ \frac{2\pi}{2N+2} & j \geq 1 \end{cases}, \quad j = 0, \dots, N \quad (1.37)$$

Chebyshev-Gauss-Lobatto

$$x_j = \cos \frac{\pi j}{N}, \quad w_j = \frac{\pi}{\bar{c}_j N}, \quad j = 0, \dots, N \quad (1.38)$$

where

$$\bar{c}_k = \begin{cases} 2, & k = 0, N \\ 1, & k = 1, 2, \dots, N-1 \end{cases} \quad (1.39)$$

The normalization factor is

$$\gamma_k = \begin{cases} \frac{\pi}{2} c_k, & \text{Gauss and Gauss-Radau} \\ \frac{\pi}{2} \bar{c}_k, & \text{Gauss-Lobatto} \end{cases} \quad (1.40)$$

1.5 Differentiation formula

In solving differential equations by Chebyshev collocation method, the unknown functions are approximated by truncated Chebyshev polynomial series. In this section the analytical formulas of the differentiation of such series are derived out.

First we discuss the complete series

$$u(x) = \sum_{k=0}^{\infty} \hat{u}_k T_k(x) \quad (1.41)$$

Its first order derivative is

$$\frac{du(x)}{dx} = \sum_{k=0}^{\infty} \hat{u}_k \frac{d}{dx} T_k(x) \quad (1.42)$$

We shall reduce it to the following forms:

$$\frac{du(x)}{dx} = \sum_{k=0}^{\infty} \hat{u}_k^{(1)} T_k(x) \quad (1.43)$$

What remains is to derive out the formulas for computing $\hat{u}_k^{(1)}$ from $\hat{u}_k$.

Substituting (1.23) into (1.43), noticing that when $k = 0$ and 1 , (1.23) should be replaced by

$$2T_k(x) = \frac{c_k}{k+1} \frac{d}{dx} T_{k+1}(x) \quad (1.44)$$

we obtain

$$\begin{aligned} \frac{du(x)}{dx} &= \sum_{k=0}^{\infty} \hat{u}_k^{(1)} \frac{c_k}{2(k+1)} \frac{d}{dx} T_{k+1}(x) - \sum_{k=2}^{\infty} \hat{u}_k^{(1)} \frac{1}{2(k-1)} \frac{d}{dx} T_{k-1}(x) \\ &= \sum_{k=1}^{\infty} \hat{u}_{k-1}^{(1)} \frac{c_{k-1}}{2k} \frac{d}{dx} T_k(x) - \sum_{k=1}^{\infty} \hat{u}_{k+1}^{(1)} \frac{1}{2k} \frac{d}{dx} T_k(x) \\ &= \sum_{k=1}^{\infty} \left[\frac{c_{k-1}}{2k} \hat{u}_{k-1}^{(1)} - \frac{1}{2k} \hat{u}_{k+1}^{(1)} \right] \frac{d}{dx} T_k(x) \end{aligned} \quad (1.45)$$

By comparing it with (1.42) we obtain $\hat{u}_k = \frac{c_{k-1}}{2k} \hat{u}_{k-1}^{(1)} - \frac{1}{2k} \hat{u}_{k+1}^{(1)}$, which is more convenient to be written as

$$2k\hat{u}_k = c_{k-1}\hat{u}_{k-1}^{(1)} - \hat{u}_{k+1}^{(1)} \quad (1.46)$$

The formula of $\hat{u}_k^{(1)}$ can be obtained from it as

$$\hat{u}_k^{(1)} = \frac{2}{c_k} \sum_{\substack{p=k+1 \\ p+k \text{ odd}}}^{\infty} p\hat{u}_p \quad (1.47)$$

For truncated series of Chebyshev polynomials, the useful formulas are

$$I_N u = \sum_{k=0}^N \hat{u}_k T_k(x) \quad (1.48)$$

$$D_N u = \sum_{k=0}^N \hat{u}_k^{(1)} T_k(x) \quad (1.49)$$

$$\hat{u}_k^{(1)} = \begin{cases} \frac{2}{c_k} \sum_{\substack{p=k+1 \\ p+k \text{ odd}}}^N p \hat{u}_p & k = 0, \dots, N-1 \\ 0 & k = N \end{cases} \quad (1.50)$$

The formulas (1.49) and (1.50) are the differentiation formulas in spectral space. In the following we transform them to physical space.

For convenience, let's rewrite (1.50) as

$$\hat{u}_k^{(1)} = \sum_{p=0}^N \hat{D}_{kp} \hat{u}_p \quad (1.51)$$

where

$$\hat{D}_{kp} = \begin{cases} \frac{2}{c_k} p, & p \geq k+1, p+k \text{ odd}, k \neq N \\ 0, & \text{otherwise} \end{cases} \quad (1.52)$$

Substituting (1.51) into (1.49), we obtain

$$D_N u = \sum_{k=0}^N \sum_{p=0}^N \hat{D}_{kp} \hat{u}_p T_k(x) \quad (1.53)$$

Substituting into it the formula (1.20) after replacing $P_k(x_j)$ by $T_k(x_j)$, we obtain the value of $D_N u$ at the Gauss-type integration points x_l :

$$\begin{aligned} (D_N u)_l &= \sum_{k=0}^N \sum_{p=0}^N \hat{D}_{kp} \frac{1}{\gamma_p} \sum_{j=0}^N u(x_j) T_p(x_j) w_j T_k(x_l) \\ &= \sum_{j=0}^N (D_N)_{lj} u(x_j) \end{aligned} \quad (1.54)$$

where the coefficient $(D_N)_{lj}$ is defined as

$$(D_N)_{lj} = \sum_{k=0}^N \sum_{p=0}^N \hat{D}_{kp} \frac{1}{\gamma_p} T_p(x_j) w_j T_k(x_l) \quad (1.55)$$

The above formulas are applicable to any kind Gauss-type integration points. For Gauss-Lobatto points a closed form of $(D_N)_{lj}$ is derived out:

$$(D_N)_{lj} = \begin{cases} \frac{\bar{c}_l}{\bar{c}_j} \frac{(-1)^{l+j}}{x_l - x_j}, & l \neq j, \\ -\frac{x_l}{2(1-x_l^2)}, & l = j = 1, \dots, N-1 \\ \frac{2N^2+1}{6}, & l = j = 0 \\ -\frac{2N^2+1}{6}, & l = j = N \end{cases} \quad (1.56)$$

The derivation of this formula is given in Appendix A.

It is more convenient to write the differentiation formula in physical space in matrix form. Denote

$$\{D_N u\} = \begin{Bmatrix} (D_N u)_0 \\ (D_N u)_1 \\ \dots \\ (D_N u)_N \end{Bmatrix}, \quad \{u\} = \begin{Bmatrix} u(x_0) \\ u(x_1) \\ \dots \\ u(x_N) \end{Bmatrix} \quad (1.57)$$

$$[D_N] = \begin{bmatrix} (D_N)_{00} & (D_N)_{01} & \dots & (D_N)_{0N} \\ (D_N)_{10} & (D_N)_{11} & \dots & (D_N)_{1N} \\ \vdots & \vdots & \ddots & \vdots \\ (D_N)_{N0} & (D_N)_{N1} & \dots & (D_N)_{NN} \end{bmatrix} \quad (1.58)$$

Then (1.54)) can be written as

$$\{D_N u\} = [D_N]\{u\} \quad (1.59)$$

For second order derivative, we have similarly

$$\{D_N^2 u\} = [D_N^2]\{u\} \quad (1.60)$$

where

$$[D_N^2] = [D_N][D_N] \quad (1.61)$$

This is easy to be understood. Higher order derivative can be computed similarly.

1.6 Integration formula

In this section we discuss the computation of integral of truncated Chebyshev polynomial series in the form of (1.48), i.e.

$$\int_{-1}^1 I_N u dx = \int_{-1}^1 \sum_{k=0}^N \hat{u}_k T_k(x) dx \quad (1.62)$$

By using (1.24) and the recurrence relation (1.23) we obtain

$$\begin{aligned}
\int_{-1}^1 I_N u(x) dx &= \sum_{k=0}^N \hat{u}_k \int_{-1}^1 T_k(x) dx \\
&= \hat{u}_0 \int_{-1}^1 1 dx + \hat{u}_1 \int_{-1}^1 x dx + \\
&\quad \sum_{k=2}^N \hat{u}_k \left[\frac{1}{2(k+1)} \frac{d}{dx} T_{k+1}(x) - \frac{1}{2(k-1)} \frac{d}{dx} T_{k-1}(x) \right] dx \\
&= 2\hat{u}_0 + \sum_{k=2}^N \hat{u}_k \left[\frac{T_{k+1}(x)}{2(k+1)} - \frac{T_{k-1}(x)}{2(k-1)} \right]_{-1}^1 \\
&= 2\hat{u}_0 + \sum_{k=2, k \text{ even}}^N \hat{u}_k \frac{-2}{(k+1)(k-1)} \\
&= \sum_{k=0, k \text{ even}}^N \frac{-2\hat{u}_k}{(k+1)(k-1)} \tag{1.63}
\end{aligned}$$

This formula could be used to compute the integral of a function according to its truncated Chebyshev polynomial series expansion coefficients.

1.7 Linear differential equations without singularity

Now we are ready to discuss the solution of differential equations. Let's examine the linear system of ordinary differential equations defined in $[-1, 1]$:

$$\frac{dy_m}{dx} + \sum_{n=1}^M a_{mn}(x) y_n = f_m(x), \quad m = 1, \dots, M \tag{1.64}$$

The boundary conditions are

$$\text{Either } y_m(x)_{x=1} = y_m(1) \text{ or } y_m(x)_{x=-1} = y_m(-1), \quad m = 1, \dots, M \tag{1.65}$$

The equations having no singularity means that the coefficients a_{mn} are all finite in the interval $[-1, 1]$. In this case the number of boundary conditions must be equal to the number of equations.

The Chebyshev-Gauss-Lobatto integration points which include both ends of the interval $[-1, 1]$ are adopted. The ends of the interval $[-1, 1]$ correspond to the collocation points $x_0 = 1$ and $x_N = -1$. It can be written out according to (1.54) that

$$\left(\frac{dy_m}{dx} \right)_{x=x_l} = \sum_{j=0}^N (D_N)_{lj} y_m(x_j), \quad m = 1, \dots, M \tag{1.66}$$

The use of the collocation method to (1.64) means that the equations are satisfied at the collocation points. So with (1.66) we can write out the linear algebraic equations with

unknowns $y_m(x_0), \dots, y_m(x_N)$ to be solved numerically:

$$\left\{ \begin{array}{l} \sum_{j=0}^N (D_N)_{lj} y_m(x_j) + \sum_{n=1}^M a_{mn}(x_l) y_n(x_l) = f_m(x_l) \\ l = 1, \dots, N; \quad m = 1, \dots, M \\ \text{Either } y_m(x_0) = y_m(1) \text{ or } y_m(x_N) = y_m(-1) \\ m = 1, \dots, M \end{array} \right. \quad (1.67)$$

We see that the algebraic equations deduced from the differential equations at the collocation point $x_0 = 1$ are replaced by boundary conditions. In fact the boundary conditions can be used to replace the algebraic equations deduced from the differential equations at the collocation point $x_N = -1$ as well. Furthermore, one can use some of the boundary conditions to replace the algebraic equations deduced from the differential equations at the boundary $x_0 = 1$ and some at the boundary $x_N = -1$.

If a boundary condition is in the form of

$$\left(\frac{dy_m}{dx} \right)_{x=1} = y'_m(1) \quad (1.68)$$

then the algebraic equation for it is (as $x_0 = 1$)

$$\sum_{j=0}^N (D_N)_{0j} y_m(x_j) = y'_m(1) \quad (1.69)$$

The reader may write out other possible cases.

For illustrating the method more clearly, let's discuss an example. Consider the one dimensional Poisson's equations:

$$\frac{d^2 u}{dx^2} + \pi^2 \sin \pi x = 0 \quad (1.70)$$

with boundary conditions

$$u(-1) = 0, \quad u(1) = 0 \quad (1.71)$$

The exact solution of the problem is

$$u = \sin \pi x \quad (1.72)$$

Let

$$y_1 = u, \quad y_2 = \frac{du}{dx} \quad (1.73)$$

The problem (1.70) and (1.71) can be written as a system of differential equations

$$\left\{ \begin{array}{l} \frac{dy_1}{dx} - y_2 = 0 \\ \frac{dy_2}{dx} = -\pi^2 \sin \pi x \end{array} \right. \quad (1.74)$$

with boundary conditions

$$y_1(-1) = 0, \quad y_2(1) = 0 \quad (1.75)$$

Evidently this system of differential equations has no singularity.

This system of ordinary differential equations are solved in various ways:

- The value of $y_1(-1)$ and $y_2(1)$ are used to replace the algebraic equations at $x_0 = 1$. In this case the algebraic equations to be solved are

$$\begin{cases} \sum_{j=0}^N (D_N)_{lj} y_1(x_j) + y_2(x_l) = 0, & l = 1, \dots, N \\ \sum_{j=0}^N (D_N)_{lj} y_2(x_j) = -\pi^2 \sin \pi x_l, & l = 1, \dots, N \\ y_1(x_0) = 0 \\ y_2(x_N) = 0 \end{cases} \quad (1.76)$$

- The value of $y_1(-1)$ and $y_2(1)$ are used to replace the algebraic equations at $x_N = -1$. In this case the algebraic equations to be solved are

$$\begin{cases} \sum_{j=0}^N (D_N)_{lj} y_1(x_j) + y_2(x_l) = 0, & l = 0, \dots, N-1 \\ \sum_{j=0}^N (D_N)_{lj} y_2(x_j) = -\pi^2 \sin \pi x_l, & l = 0, \dots, N-1 \\ y_1(x_0) = 0 \\ y_2(x_N) = 0 \end{cases} \quad (1.77)$$

- The value of $y_1(-1)$ is used to replace the algebraic equation of the first differential equation at $x_0 = 1$, and $y_2(1)$ of the second at $x_N = -1$. In this case the algebraic equations to be solved are

$$\begin{cases} \sum_{j=0}^N (D_N)_{lj} y_1(x_j) + y_2(x_l) = 0, & l = 1, \dots, N \\ \sum_{j=0}^N (D_N)_{lj} y_2(x_j) = -\pi^2 \sin \pi x_l, & l = 0, \dots, N-1 \\ y_1(x_0) = 0 \\ y_2(x_N) = 0 \end{cases} \quad (1.78)$$

1.8 Linear differential equations with singularity

Here we consider the differential equations in the same form as (1.64). But some of the coefficients $a_{mn}(x)$ or the function $f_m(x)$ have infinite values at $x = -1$. In this case special care must be taken to the end point $x = -1$ of the interval $[-1, 1]$. The same is for boundary conditions. We illustrate by examples how such differential equations are solved by Chebyshev collocation method.

EXAMPLE 1: Solve the differential equation

$$\frac{dy}{dx} + \frac{3y}{1+x} = \frac{3f(x)}{1+x}, \quad -1 \leq x \leq 1 \quad (1.79)$$

where $f(x)$ is finite and continuous and $f(-1) \neq 0$, and y and $\frac{dy}{dx}$ must have finite values.

We see that no boundary condition is imposed. In fact the regularity condition that y and $\frac{dy}{dx}$ finite at $x = -1$ requires

$$y(-1) = f(-1) \quad (1.80)$$

at the end $x = -1$. This can be easily obtained by setting $x = -1$ in (1.79). That is to say, the differential equation (1.79) accepts no more boundary condition other than (1.80).

The differential equation can be solved in two ways. The first way is to use Chebyshev-Gauss-Lobatto integration points in replacing the algebraic equation deduced from the differential equation with the boundary condition at the end $x_N = -1$, which can avoid computing the coefficient $\frac{3}{1+x}$ and the function $\frac{3f(x)}{1+x}$ at $x = -1$. The second way is to use the Chebyshev-Gauss-Radau integration points which does not contain the end $x = -1$ as collocation point.

When using the Chebyshev-Gauss-Lobatto integration points the algebraic equation to be solved are

$$\begin{cases} \sum_{j=0}^N (D_N)_{lj} y(x_j) + \frac{3}{1+x_l} y(x_l) = \frac{3f(x_l)}{1+x_l}, & l = 0, \dots, N-1 \\ y(x_N) = 0 \end{cases} \quad (1.81)$$

When using the Chebyshev-Gauss-Radau integration points the algebraic equation to be solved are

$$\sum_{j=0}^N (D_N)_{lj} y(x_j) + \frac{3}{1+x_l} y(x_l) = \frac{3f(x_l)}{1+x_l}, \quad l = 0, \dots, N \quad (1.82)$$

EXAMPLE 2: Solve the differential equations

$$\begin{cases} \frac{dy_1}{dx} - \frac{1}{1+x} y_2 = 0 \\ \frac{dy_2}{dx} - \frac{6[1 - e^{-(1+x)}]}{1+x} y_1 - \frac{1 - 6e^{-(1+x)}}{1+x} y_2 = 0 \end{cases} \quad (1.83)$$

with the boundary condition

$$y_1(1) = f \quad (1.84)$$

The solution must have the properties that y_1 , y_2 and $\frac{dy_1}{dx}$, $\frac{dy_2}{dx}$ are all finite.

This is a system of two differential equations. But only one boundary condition is imposed. In fact the conditions that y_1 , y_2 and $\frac{dy_1}{dx}$, $\frac{dy_2}{dx}$ are all finite imply a condition at $x = -1$:

$$y_2(-1) = 0 \quad (1.85)$$

which can be easily obtained by setting $x = -1$ in (1.83). So the system of differential equations accepts only one boundary condition, although it has two differential equations.

Like the first example these equations can also be solved in two ways using either the Chebyshev-Gauss-Lobatto integration points or the Chebyshev-Gauss-Radau integration points.

When using the Chebyshev-Gauss-Lobatto integration points the algebraic equation to be solved are ($x_0 = 1$ in this case)

$$\left\{ \begin{array}{l} \sum_{j=0}^N (D_N)_{lj} y_1(x_j) + \frac{1}{1+x_l} y_2(x_l) = 0, \quad l = 0, \dots, N-1 \\ \sum_{j=0}^N (D_N)_{lj} y_2(x_j) - \frac{6[1 - e^{-(1+x_l)}]}{1+x_l} y_1(x_l) \\ \quad - \frac{1 - 6e^{-(1+x_l)}}{1+x_l} y_2(x_l) = 0, \quad l = 0, \dots, N-1 \\ y_1(x_0) = f \\ y_2(x_N) = 0 \end{array} \right. \quad (1.86)$$

When using the Chebyshev-Gauss-Radau integration points the algebraic equation to be solved are

$$\left\{ \begin{array}{l} \sum_{j=0}^N (D_N)_{lj} y_1(x_j) + \frac{1}{1+x_l} y_2(x_l) = 0, \quad l = 1, \dots, N \\ \sum_{j=0}^N (D_N)_{lj} y_2(x_j) - \frac{6[1 - e^{-(1+x_l)}]}{1+x_l} y_1(x_l) \\ \quad - \frac{1 - 6e^{-(1+x_l)}}{1+x_l} y_2(x_l) = 0, \quad l = 0, \dots, N \\ y_1(x_0) = f \end{array} \right. \quad (1.87)$$

or

$$\left\{ \begin{array}{l} \sum_{j=0}^N (D_N)_{lj} y_1(x_j) + \frac{1}{1+x_l} y_2(x_l) = 0, \quad l = 0, \dots, N \\ \sum_{j=0}^N (D_N)_{lj} y_2(x_j) - \frac{6[1 - e^{-(1+x_l)}]}{1+x_l} y_1(x_l) \\ \quad - \frac{1 - 6e^{-(1+x_l)}}{1+x_l} y_2(x_l) = 0, \quad l = 1, \dots, N \\ y_1(x_0) = f \end{array} \right. \quad (1.88)$$

Before conclusion a brief discussion is given to the general case

$$\frac{dy_m}{dx} + \sum_{n=1}^M a_{mn}(x) y_n = f_m(x), \quad m = 1, \dots, M \quad (1.89)$$

including singularities at $x = -1$. If the number of regularity conditions at $x = -1$ is known to be s , then $M - s$ boundary conditions must be imposed at $x = 1$. Otherwise, the author's experience is to determine the number of singularity conditions by numerical test. Impose d boundary conditions at $x = 1$, and solve the problem with various values

of N . If the solution is the same for all values of N , then the value of d is correct, i.e. there are $M - d$ regularity conditions at $x = -1$. For a wrong value of d , different results for the solution are normally obtained for different values of N .

1.9 Combination of differential and algebraic equations

Here we consider only combination of linear differential and algebraic equations. In this situation the system of equations can be written in the general form

$$a_{m0}(x) \frac{dy_m}{dx} + \sum_{n=1}^M a_{mn}(x) y_n = f_m(x), \quad m = 1, \dots, M \quad (1.90)$$

If $a_{m0} = 0$, then the m -th equation is an algebraic equation. Otherwise it is a differential equation.

The algebraic equations to be solved numerically deduced from (1.90) are

$$a_{m0}(x_l) \sum_{j=0}^N (D_N)_{lj} y_m(x_j) + \sum_{n=1}^M a_{mn}(x_l) y_n(x_l) = f_m(x_l) \quad l = 0, \dots, N \quad (1.91)$$

for any kinds of Gauss type integration points. The boundary conditions can be imposed in the same ways as in the last two sections. The total number of regularity and imposed boundary conditions is equal to the number of differential equations in the system.

It is seen that the cases studied in section 1.7 and 1.8 are special cases of present one.

Chapter 2

Chebyshev collocation method: Programming

2.1 Introduction

In this introduction we give the file structure of the programs introduced in this chapter. The root directory is named **GeophysicalODEs**. Subdirectories and files under this directory are listed below:

```
GeophysicalODEs\  
    Utilities_general\  
        parameters.h  
        datatype.h  
        nrutil.h  
        nrutil.cpp  
        nrinterp.cpp  
        nrludcmp.cpp  
        nrgaussj.cpp  
        nrbisec.cpp  
        nr.h  
    spectral_method\  
        chebyshev_utils.cpp  
        chebcoll_odes.cpp  
        chebyshev.h  
        Example_Chebcoll_Gauss_Lobatto\  
            Example_Chebcoll_Gauss_Lobatto.cpp  
            Example_Chebcoll_Gauss_Lobatto.dsw  
        Example_Chebcoll_Gauss_Radau\  
            Example_Chebcoll_Gauss_Radau.cpp  
            Example_Chebcoll_Gauss_Radau.dsw
```

The names ended with a back slash are directories. More subdirectories will be added in the next chapter.

2.2 Basic utility programs

2.2.1 parameters.h

In this file some useful parameters such as π , G (the universal constant of gravitation), Ω (Earth's rotation rate) are defined:

```
#ifndef _PARAMETERS_
#define _PARAMETERS_
#define EPS 1.0e-8
#define PI 3.141592653589793238
#define G 6.67E-11
#define _PARAMETERS_
#endif
```

When some of these parameters are needed it should be included with the line

```
#include "parameters.h"
```

in the source program.

2.2.2 datatype.h

Every file of source program should have the following line:

```
#include "datatype.h"
```

In `datatype.h` the data types are defined as listed below:

```
#ifndef _DATATYPE_
#define _DATATYPE_
#define real double //or #define real float
#define int int //or #define int long
//or #define int short
static real _real_;
static int _int_;
static char _char_;
#endif
```

Throughout the programs, `real` will be used for defining a real variable which could be `double` or `float` type, and `int` an integer variable which could be `int`, `long` or `short` type. In this manner, it only needs to modify `datatype.h` for choosing `double` or `float` as real variable and `int`, `long` or `short` as integer variable. `_real_` and `_int_` and `_char_` will be used just as normal variables of type `real`, `int` and `char` respectively. They will be used as arguments for calling overloading functions defined in `nrutil.h`.

2.2.3 nrutil.h and nrutil.cpp

Some utility programs and function prototypes are given in `nrutil.h` which is modified from a program given by Press et al. (1992). It contains some inline functions listed below:

```
/* ===== */
#define SIGN(a,b) ((b)>=0.0? fabs(a):-fabs(a))
/* ===== */
template<class T>
inline T SQR(T a){return a*a;}
/* ===== */
template<class T>
inline T CUB(T a){return a*a*a;}
/* ===== */
template<class T>
inline T MAX(T a,T b){return (a>b)? a:b;}
/* ===== */
template<class T>
inline T MIN(T a,T b){return (a<b)? a:b;}
/* ===== */
template<class T>
inline void SWAP(T &a,T &b) {T temp;temp=a;a=b;b=temp;}
/* ===== */
```

The meaning of `SIGN`, `SQR`, `CUB`, `MAX`, `MIN` and `SWAP` could be easily understood. The remaining of `nrutil.h` are some function prototypes which are defined in `nrutil.cpp`. The file `nrutil.cpp` will not be listed. We only give the usage of the functions defined in it.

The function `nrerror` is used to print an error message and terminate the execution of the program. It is called in the form of

```
nrerror("error message text");
```

The rest is some overloading functions generated from template functions for dynamic array manipulation. In standard C, the first index of an array is always zero, but in our program it could be any value.

For allocating a real vector

```
v[n1], v[n1+1],v[n1+2] ... v[n2]
```

the program is

```
real *v;
int n1,n2;
..... //evaluate n1 and n2
v=vector(_real_,n1,n2);
```


These lines of program could be compiled in all the cases when we use `float` or `double` as real variable and `int`, `short` or `long` as integer variable. The real variable `_real_` is defined in `datatype.h`. When the vector is not needed anymore, it could be removed from memory by

```
free_vector(v,n1,n2);
```

For allocating a real matrix `a`:

```
a[m1][n1]  a[m1][n1+1]  ... a[m1][n2]
a[m1+1][n1] a[m1+1][n1+1] ... a[m1+1][n2]
. . .
a[m2][n1]  a[m2][n1+1]  ... a[m2][n2]
```

the program is

```
real **a;
int m1,m2,n1,n2;
..... //evaluate m1, m2, n1, n2
a=matrix(_real_,m1,m2,n1,n2);
```

And for removing it from the memory the program is

```
free_matrx(a,m1,m2,n1,n2); //remove
```

The case for a three-dimensional array is similar. For allocating the array the program is

```
real ***a;
int m1,m2,n1,n2,l1,l2;
..... //evaluate m1, m2, n1, n2, l1, l2
a=tensor(_real_,m1,m2,n1,n2,l1,l2);
```

And for removing it from the memory the program is

```
free_tensor(a,m1,m2,n1,n2,l1,l2);
```

Here the word "tensor" is used to represent a three dimensional array as is seen in the program.

The same is for a four-dimensional array. For allocating the array the program is

```
real ****a;
int m1,m2,n1,n2,l1,l2,k1,k2;
..... //evaluate m1, m2, n1, n2, l1, l2, k1, k2
a=tensor4(_real_,m1,m2,n1,n2,l1,l2,k1,k2);
```

And for removing it from the memory the program is

```
free_tensor4(a,m1,m2,n1,n2,l1,l2,k1,k2);
```

The function `submatrix` is used to assign a part of an existing matrix to a new matrix, and `free_submatrix` remove the new matrix. Assume a part of the existing matrix `a` is to be assigned to `suba`. If the part of `a` to be assigned to `suba` is

```
a[ml][nl]    a[ml][nl+1]    ... a[ml][nh]
a[ml+1][nl]  a[ml+1][nl+1]  ... a[ml+1][nh]
.
.
.
a[mh][nl]    a[mh][nl+1]    ... a[mh][nh]
```

and the lower limits of rows and columns of `suba` are respectively mln and nln , then the program is

```
real **suba;
int ml,mh,nl,nh,mln,nln;
..... //evaluate ml, mh, nl, nh, mln, nln
suba=submatrix(a,ml,mh,nl,nh,mln,nln);
```

Clearly the matrix `suba` has $mh - ml + 1$ rows and $nh - nl + 1$ columns. It must also be noticed that the elements of `suba` occupy the same positions in memory as its corresponding elements of `a`. Hence any change of `suba` would also cause change of `a` and vice versa. For removing `suba` from memory the program is

```
free_submatrix(suba,mln,mln+mh-ml+1,nln,nln+nh-nl+1);
```

When `suba` is removed, `a` is not.

2.2.4 nrinterp.cpp

In `nrinterp.cpp` four interpolation methods are implemented:

```
void linint(int n,real *xa,real *ya,int m,real *x,real *y)
void polint(int n,real *xa,real *ya,int m,real *x,real *y)
void ratint(int n,real *xa,real *ya,int m,real *x,real *y)
void splint(int n,real *xa,real *ya,int m,real *x,real *y)
```

They are respectively the linear, polynomial, rational and natural Spline interpolation.

The parameters of all these functions are the same. The input data are

- `n`: number of known values,
- `xa`: coordinates of known values in the form of `xa[1]`, `xa[2]`, ..., `xa[n]`,
- `ya`: known values in the form of `ya[1]`, `ya[2]`, ..., `ya[n]`,
- `m`: number of values to be interpolated,
- `x`: coordinates for interpolation in the form of `x[1]`, `x[2]`, ..., `x[m]`,

The output is

- `y`: interpolation value in the form of `y[1]`, `y[2]`, ..., `y[m]`.

They are also modified from Press et al. (1992) where a detailed description of the methods could be found.

In the future these functions will be called via the function `interp`:

```

void interp(char *Int_method,int n,real *xa,real *ya, int m,real *x,
            real *y){
if(strcmp(Int_method,"Liner")==0)linint(n,xa,ya,m,x,y);
else if(strcmp(Int_method,"Polinomial")==0)polint(n,xa,ya,m,x,y);
else if(strcmp(Int_method,"Rational")==0)ratint(n,xa,ya,m,x,y);
else if(strcmp(Int_method,"Spline")==0)splint(n,xa,ya,m,x,y);
else nrerror("Interpolation method error");
}

```

This is the last function in `nrinterp.cpp`. Its usage is easy to be understood.

2.2.5 nrludcmp.cpp

The file `nrludcmp.cpp` contains the functions for solving linear equations and reversing matrix using the LU decomposition.

These programs are also modified from Press et al. (1992) where their detailed description are given.

The functions which will be called directly are

```

int ludcmp(real **a,int n,real *b)
int ludcmp(real **a,int n,real **b,int m)
int ludcmp(real **a,int n,real **b)

```

The first solves a system of linear equations in the form of

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

The second solves a system of linear equations with multiple columns in the right hand side:

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1m} \\ b_{21} & b_{22} & \cdots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nm} \end{bmatrix}$$

And the third function reverses the matrix **a**.

In all these three functions the indices of the arrays **a** and **b** are the same as the subscripts in the last two equations. The results are all stored in **b**. All the functions destroy the matrix **a**. If the computations are successful the functions return 1, otherwise the functions return 0 indicating either the matrix is singular or the LU decomposition is unsuccessful and thus the result is meaningless.

2.2.6 nrgaussj.cpp

The file nrgaussj.cpp contains the functions for solving linear equations and reversing matrix using column pivoting Gauss-Jordan elimination.

Three functions are given:

```
int gaussj(real **a,int n,real *b)
int gaussj(real **a,int n,real **b,int m)
int gaussj(real **a,int n,real **b)
```

Their usage are principally the same as the three functions of LU decomposition.

In all these programs attempt is made to avoid the computations of multiplying with zero for saving computation time.

2.2.7 nrbisec.cpp

This file contains a function for solving non-linear equations using bi-section method:

```
int bisec(int (*func)(real x,real &f),real x1,real x2,
          real step,real &x,real toll)
```

It tries to find (only) one value of x where the function $f(x)$ changes sign in the interval between $x1$ and $x2$.

For the input function, x is the variable, f is the value, and the returned value is a flag indicating if the computation of the value from the variable of the function is meaningful. This flag is designed for handling singularities such as division by zero. If the value of the function computed is a meaningful value the function returns 1, if a singularity occurs for the value of x , the function returns 0, indicating the value of the function given is meaningless. As example the function could be

```
int func(real x,real &f){
if (x==0)return 0;
f=10.0/x;
return 1;
}
```

The property of the returned value is used in the function `bisec` for avoiding meaningless comparisons.

The function `bisec` searches for the solution from $x1$ to $x2$. First it determines if the values of the function $f(x)$ have different signs at the points $x1$ and $x1+step$, if not it continues to do the same thing at the points $x1+step$ and $x1+step+step$ and so on. If no pair of different sign is found up to $x2$ it returns the value "0". Once a pair of different signs is found it uses the bi-section method to determine the value of x where the sign of $f(x)$ changes. The result is stored in the reference variable x . The precision is `toll`. In this latter case if $f(x)$ decrease as x approaching the value found, the it returns "1", otherwise it returns "-1".

It is not required that $x2 > x1$. If $x2 > x1$, then one must have `step > 0`, otherwise one must have `step < 0`.

As result:

- If it returns "0", $f(x)$ does not change sign in the interval between x_1 and x_2 .
- If it returns "-2", the computation of the function $f(x)$ for the value of x is failed, i.e. the function returned 0.
- If it returns "-1", $f(x)$ changes sign at the value x but this value is not likely a solution of the equation $f(x) = 0$ as $f(x)$ does not decrease to zero.
- if it returns "1", $f(x)$ changes sign at the value x and this value is likely a solution of the equation $f(x) = 0$ as $f(x)$ does likely decrease to zero.

2.2.8 nr.h

This file is listed below:

```
#ifndef _NR_H_
#define _NR_H_
void interp(char *Int_method,int n,real *xa,real *ya,int m,real *x,
           real *y);
int ludcmp(real **a,int n,real *b);
int ludcmp(real **a,int n,real **b,int m);
int ludcmp(real **a,int n,real **b);
int gaussj(real **a,int n,real *b);
int gaussj(real **a,int n,real **b,int m);
int gaussj(real **a,int n,real **b);
int bisec(real (*f)(real),real x1,real x2,real step,real &x,real toll);
```

If one or several functions listed in this file is called in a program file, this file should have the line

```
#include "nr.h"
```

2.3 Utility programs for Chebyshev collocation method

The functions are collected in the file `chebyshev_utils.cpp`.

2.3.1 Computation of collocation coordinates for $t \in [a, b]$ and $x \in [-1, 1]$

For generality, let's consider a system of differential equations in $t \in [a, b]$. For using the Chebyshev collocation method developed in the previous chapter, we must transform the definition domain of the differential equations $t \in [a, b]$ to $x \in [-1, 1]$.

We use the linear transformation. The relation between $t \in [a, b]$ and $x \in [-1, 1]$ is

$$t = \frac{b+a}{2} + \frac{b-a}{2}x, \quad x = \frac{2t - (b+a)}{b-a}. \quad (2.1)$$

It is seen that $x = -1$ and $x = 1$ correspond respectively to $t = a$ and $t = b$.

Different from what was described in Chapter 1, here we store the collocation coordinates of x in ascendant order, i.e. $x_0 = -1, \dots, x_N = 1$. This is just the reverse of what was given in Chapter 1.

The collocation coordinates are computed by the function

```
void ChebCollPnt(real a,real b,int Num_Coll,int type,
                real *X_Coll,real *T_Coll)
```

The inputs are

- **a, b:** Lower and higher limits of the variable of the differential equations, i.e. $t \in [a, b]$.
- **Num_Coll:** Number of collocation points. If N is the degree of the truncated Chebyshev polynomial series, then $\text{Num_Coll} = N + 1$.
- **type:** Type of Gauss-type integration points being used for collocation approximation. **type**=0, 1 and 2 respectively correspond to Gauss, Gauss-Radau and Gauss-Lobatto type.

The outputs are

- **X_Coll:** The coordinates of the collocation points of x , i.e. $\text{X_Coll}[i] = x_i$, $i = 0, \dots, N$ where $N = \text{Num_Coll} - 1$.
- **T_Coll:** The coordinates of the collocation points of t , i.e. $\text{T_Coll}[i] = t_i$, $i = 0, \dots, N$ where $N = \text{Num_Coll} - 1$. The relation between t_i and x_i is given in equation (2.1).

2.3.2 Transformation between physical and spectral spaces

The physical space refers to $u(x_0), \dots, u(x_N)$, and the spectral space $\hat{u}_0, \dots, \hat{u}_N$.

The transformations from physical to spectral spaces is realized by the functions

```
void ChebPhysToSpec(int Num_Coll,int type,real *Phys,real *Spec)
```

The inputs are

- **Num_Coll:** The same as before.
- **type:** The same as before.
- **Phys:** The values of the function at collocation points, i.e. $\text{Phys}[i] = u(x_i)$, $i = 0, \dots, N$ where $N = \text{Num_Coll} - 1$ is the degree of the Chebyshev polynomial series.

The output is

- **Spec:** The coefficients of the Chebyshev polynomial series of the function, i.e. $\text{Spec}[i] = \hat{u}_i$, $i = 0, \dots, N$ where $N = \text{Num_Coll} - 1$ is the degree of the Chebyshev polynomial series.

The transformations from spectral to physical spaces is realized by the function

```
void ChebSpecToPhys(int Num_Coll,int type,real *Spec,real *Phys)
```

The inputs are

- Num_Coll: The same as before.
- type: The same as before.
- Spec: The same as before.

The output is

- Phys: The same as before.

When using these functions, it must be remembered that the ordering of x_i is different from Chapter 1.

2.3.3 Computation of x corresponding to known values of t

Some times it is needed to compute the values of x corresponding to the known values of t . This is done by the function

```
void ChebCollCoor(real a,real b,int Num_Coor,real *T_Coor,real *X_Coor)
```

The inputs are

- a and b: As before.
- Num_Coor: The number of known values of t .
- T_Coor: Known values of t in the form of T_Coor[1], ..., T_Coor[Num_Coor].

The output is

- X_Coor: The values of x computed for known values of t . X_Coor[1], ..., X_Coor[Num_Coor] corresponding to T_Coor[1], ..., T_Coor[Num_Coor].

2.3.4 Computation of function from its Chebyshev polynomial series coefficients for arbitrary values of variable

It also happens to compute the values of a function according to its Chebyshev polynomial series coefficients for arbitrary values of variable. This is done by the function

```
void ChebSpecToValu(int Num_Coll,real *Spec,int Num_Coor,
                    real *X_Coor,real *Valu)
```

The inputs are

- Num_Coll: The same as before.
- Spec: The same as before.

- Num_Coor: The same as before.
- X_Coor: The same as before.

The output is

- Valu: Valu[i] is the value of $u(x)$ for $x = X_Coor[i]$, $i=1,...,Num_Coor$.

2.3.5 Transformation of Chebyshev polynomial series to polynomial of x

The function

```
void ChebSpecToPoly(int Num_Coll,real *Spec,real *X_PolyCoef)
```

transforms the Chebyshev polynomial series to the polynomial

$$a_0 + a_1x + \cdots + a_Nx^N \quad (2.2)$$

according to equation (1.25). The inputs are

- Num_Coll: The same as before.
- Spec: The same as before.

The output is

- X_PolyCoef: The coefficients in (2.2), $X_PolyCoef[i]=a_i$, $i = 0, ..., N$ with $N = Num_Coll-1$.

2.3.6 Transformation of polynomial of x to polynomial of t

The function

```
void ChebPolyToPoly(real a,real b,int Num_Coll,
    real *X_PolyCoef,real *T_PolyCoef)
```

transform a polynomial of x to a polynomial of t according to the relation between x and t given by (2.1). The inputs are

- a, b: The same as before.
- Num_Coll: The same as before.
- X_PolyCoef: The same as before.

The output is

- T_PolyCoef: An array of Num_Coll real numbers. $T_PolyCoef[i]$ is the value of the coefficient of t^i , $i = 0, ..., N$ where $N = Num_Coll-1$.

2.3.7 Computation of differential

The function

```
void ChebDiffCoef(int Num_Coll,int type,real **C_Diff)
```

compute the matrix $[D_N]$ defined in (1.58) and (1.55). The inputs are

- **Num_Coll**: The same as before.
- **type**: The same as before.

The output is

- **C_Diff**: The matrix $[D_N]$. $C_Diff[l][j] = (D_N)_{lj}$ where $l, j = 0, \dots, N$ with $N = \text{Num_Coll} - 1$.

The result could be used to compute differentials of functions by simply multiplying the matrix with the values of the function at collocation points.

It should be remarked that this matrix is transformed to the form corresponding to the order of collocation points computed in the function

```
void ChebCollPnt(real a,real b,int Num_Coll,real *X_Coll,real*T_Coll)
```

So the formula (1.54) or (1.59) which computes the differential with respect to x remains unchanged and the differential with respect to t is

$$\left[\frac{du_N(t)}{dt} \right]_{t=t_i} = \frac{2}{b-a} \left[\frac{du_N(x)}{dx} \right]_{x=x_i} = \frac{2}{b-a} (D_N u)_i \quad (2.3)$$

2.3.8 Computation of integrals

The function

```
real Integration_Chebcoll(real a,real b,int Num_Coll,real *Spec);
```

compute the integral

$$\int_a^b f(t) dt = \frac{b-a}{2} \int_{-1}^1 f(x) dx \quad (2.4)$$

where the variable t is transformed to x with the relation (2.1). The inputs **a**, **b**, **Num_Coll** are the same as before, and **Spec** the coefficients of the truncated Chebyshev polynomial series of $f(x)$. The function returns the value of the integration.

2.4 Solution of differential equations

The programs are in the file `chebyshev_odes.cpp`.

2.4.1 Inversion by block of the matrix of the algebraic equations deduced from collocation method

The matrix of the algebraic equations obtained by applying the collocation method to a system of linear ordinary differential equations has a special form. For the case of 4 ordinary differential equations and 6 collocation points, this matrix has the form

$$\begin{bmatrix} \begin{array}{cccccc|cccc} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & & & & \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & & \cdot & & & \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & & & \cdot & & \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & & & & \cdot & \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & & & & & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & & & & & \cdot \end{array} & \begin{array}{cccc|cccc} \cdot & & & & & & & \\ & \cdot & & & & & & \\ & & \cdot & & & & & \\ & & & \cdot & & & & \\ & & & & \cdot & & & \\ & & & & & \cdot & & \\ & & & & & & \cdot & \end{array} & \begin{array}{cccc|cccc} \cdot & & & & & & & \\ & \cdot & & & & & & \\ & & \cdot & & & & & \\ & & & \cdot & & & & \\ & & & & \cdot & & & \\ & & & & & \cdot & & \\ & & & & & & \cdot & \end{array} & \begin{array}{cccc|cccc} \cdot & & & & & & & \\ & \cdot & & & & & & \\ & & \cdot & & & & & \\ & & & \cdot & & & & \\ & & & & \cdot & & & \\ & & & & & \cdot & & \\ & & & & & & \cdot & \end{array} \\ \hline \begin{array}{cccc|cccc} \cdot & & & & & & & \\ & \cdot & & & & & & \\ & & \cdot & & & & & \\ & & & \cdot & & & & \\ & & & & \cdot & & & \\ & & & & & \cdot & & \\ & & & & & & \cdot & \end{array} & \begin{array}{cccc|cccc} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{array} & \begin{array}{cccc|cccc} \cdot & & & & & & & \\ & \cdot & & & & & & \\ & & \cdot & & & & & \\ & & & \cdot & & & & \\ & & & & \cdot & & & \\ & & & & & \cdot & & \\ & & & & & & \cdot & \end{array} & \begin{array}{cccc|cccc} \cdot & & & & & & & \\ & \cdot & & & & & & \\ & & \cdot & & & & & \\ & & & \cdot & & & & \\ & & & & \cdot & & & \\ & & & & & \cdot & & \\ & & & & & & \cdot & \end{array} \\ \hline \begin{array}{cccc|cccc} \cdot & & & & & & & \\ & \cdot & & & & & & \\ & & \cdot & & & & & \\ & & & \cdot & & & & \\ & & & & \cdot & & & \\ & & & & & \cdot & & \\ & & & & & & \cdot & \end{array} & \begin{array}{cccc|cccc} \cdot & & & & & & & \\ & \cdot & & & & & & \\ & & \cdot & & & & & \\ & & & \cdot & & & & \\ & & & & \cdot & & & \\ & & & & & \cdot & & \\ & & & & & & \cdot & \end{array} & \begin{array}{cccc|cccc} \cdot & & & & & & & \\ & \cdot & & & & & & \\ & & \cdot & & & & & \\ & & & \cdot & & & & \\ & & & & \cdot & & & \\ & & & & & \cdot & & \\ & & & & & & \cdot & \end{array} & \begin{array}{cccc|cccc} \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \end{array} \end{bmatrix} \quad (2.5)$$

where the dots represent non-zero elements. This matrix is inverted by blocks according to the formulas

$$A = \begin{bmatrix} P & Q \\ R & S \end{bmatrix} \quad A^{-1} = \begin{bmatrix} \tilde{P} & \tilde{Q} \\ \tilde{R} & \tilde{S} \end{bmatrix} \quad (2.6)$$

$$\begin{cases} \tilde{P} = P^{-1} + (P^{-1} \cdot Q) \cdot (S - R \cdot P^{-1} \cdot Q)^{-1} \cdot (R \cdot P^{-1}) \\ \tilde{Q} = -(P^{-1} \cdot Q) \cdot (S - R \cdot P^{-1} \cdot Q)^{-1} \\ \tilde{R} = -(S - R \cdot P^{-1} \cdot Q)^{-1} \cdot (R \cdot P^{-1}) \\ \tilde{S} = (S - R \cdot P^{-1} \cdot Q)^{-1} \end{cases} \quad (2.7)$$

The function for computing the inverse matrix is

```
void ChebCollInv(int Num_Equ,int Num_Coll,real **Coef,real **Inv_Coef)
```

The inputs are

- Num_Equ: Number of differential equations.

- **Num_Coll**: Number of collocation points.
- **Coef**: The coefficient matrix. Its elements are `Coef[i][j]`, $i, j=1, \dots, \text{Num_Equ} \times \text{Num_Coll}$.

The output is

- **Inv_Coef**: The inverse of the coefficient matrix. Its elements are `Inv_Coef[i][j]`, $i, j=1, \dots, \text{Num_Equ} \times \text{Num_Coll}$.

In this program the multiplications with zero are avoided.

2.4.2 Solution of linear equations

The linear algebraic equation system to be solved is

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{bmatrix} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1m} \\ b_{21} & b_{22} & \cdots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nm} \end{bmatrix}$$

which is simply written as

$$A \cdot x = b \quad (2.8)$$

and B is the inverse matrix of A with error. One iteration is made to improve the accuracy of the solution according to the following formulas:

$$x_0 = B \cdot b, \quad x = x_0 + B \cdot (b - A \cdot x_0) \quad (2.9)$$

The function for solving the linear equation system is

```
void ChebCollSol(int Num_Equ, int Num_Coll, real **Coef,
                 real **Inv_Coef, int Num_Solution, real **Vector)
```

The inputs are

- **Num_Equ**: The same as before.
- **Num_Coll**: The same as before.
- **Coef**: The same as before.
- **Inv_Coef**: The same as before.
- **Num_Solution**: Number of columns of b in (2.8).
- **Vector**: The right hand side of (2.8) stored as `Num_Solution` rows. It is in fact b^T as compared to (2.8), i.e. its components are `Vector[i][j]`, $i=1, \dots, \text{Num_Solution}$, $j=1, \dots, \text{Num_Equ} \times \text{Num_Coll}$.

The output is

- **Vector**: Solution of the equations. It is x^T as compared to (2.8), i.e. `Vector[i][j]` is the value of i -th solution and j -th variable.

2.4.3 Solution of ordinary differential equations using Chebyshev-Gauss-Lobatto integration points

We consider the ordinary differential equation system

$$a_{m0} \frac{dy_m}{dt} = \sum_{n=1}^M a_{mn}(t) y_n + f_m(t), \quad t \in [a, b], \quad m = 1, \dots, M \quad (2.10)$$

with boundary conditions

$$\text{Either } y_m(t)_{t=a} = y_m(a) \text{ or } y_m(t)_{t=b} = y_m(b), \quad m = 1, \dots, M. \quad (2.11)$$

This system of differential equations is solved by the function

```
int ChebColl_ODES(real a, real b, int Num_Equ,
    real (*Equ_Coef)(int, int, int), real (*Equ_Vect)(int, int),
    int Num_Solution, int **BC_Desc, real **BC_Vect,
    int Num_Coll, real ***Solution);
```

It gives the solution of the differential equations for several sets of boundary conditions. The variables in it are defined as follows. The inputs are

- **a** and **b**: Defined as before.
- **Num_Equ**: Number of equations in the system, also the number of unknown functions.
- **Num_Coll**: Number of collocation points.
- **Equ_Coef(int i, int j, int k)**: The function returning the value of the coefficient $a_{ij}(t)$ at k th collocation point t_k , $i, j=1, \dots, \text{Num_Equ}$, $k=0, \dots, \text{Num_Coll}-1$.
- **Equ_Vect(int i, int j)**: The function returning the value of the vector component $f_i(t)$ at j th collocation point t_j , $i=1, \dots, \text{Num_Equ}$, $j=0, \dots, \text{Num_Coll}-1$.
- **Num_Solution**: Number of solutions to be computed, also the number of sets of boundary condition assigned.
- **BC_Desc[i][j]**: Description of the boundary conditions. $i=1, \dots, \text{Num_Equ}$ identifies the function y_i . $j=1$ or 2 . **BC_Desc[i][1]=1** means the boundary condition is given at $t = a$, **BC_Desc[i][1]=2** means the boundary condition is given at $t = b$, **BC_Desc[i][2]=1** means the boundary condition is used to replace the algebraic equations deduced from the ordinary differential equations at $t = a$, **BC_Desc[i][2]=2** means the boundary condition is used to replace the algebraic equations deduced from the ordinary differential equations at $t = b$. These properties are the same for every set of boundary conditions.
- **BC_Vect[i][j]**: Boundary value of the i -th solution of j -th function at the boundary as described by **BC_Desc[i][1]**.

The output is

- `Solution[i][j][k]`: The solution, value of i -th solution of j -th function at k -th collocation point t_k , $i=1,\dots,\text{Num_Solution}$, $j=1,\dots,\text{Num_Equ}$, $k=0,\dots,\text{Num_Coll}-1$.

The function returns 1 if meaningful solution is found. Otherwise it returns 0 if no meaningful solution is found due to such as the fail of matrix inversion.

For inverting the matrix of the linear algebraic equations of the problem the function `ChebColl_ODES` calls the function

```
int ChebCollInv(int Num_Equ,int Num_Coll,real **Coef,
               real **InvCoef)
```

which returns 1 if the matrix is successfully inverted, otherwise it returns 0. This may be done when a_{m0} are all not zero (even so it fails sometimes). When some of the coefficients a_{m0} are zero, (2.10) is a combination of differential and algebraic equations, and the blocks on the diagonal of (2.5) are no more all regular. In this case the above function is not applicable. The function

```
int ChebColl_ODES1(real a,real b,int Num_Equ,
                  real (*Equ_Coef)(int,int,int),real (*Equ_Vect)(int,int),
                  int Num_Solution,int **BC_Desc,real **BC,
                  int Num_Coll,real ***Solution);
```

solve the problem in such situation. In this function the matrix is inverted using Gauss-Jordan elimination:

```
for(i=1;i<=DIM;i++)for(j=1;j<=DIM;j++)Coef1[i][j]=Coef[i][j];
flag=gaussj(Coef1,DIM,Inv_Coef);
```

which destroys the array `Coef1` after. The function `ChebColl_ODES1` returns the value of `flag` which has the same meaning as the function `ChebColl_ODES`. It is applicable to any cases except when the matrix is really singular.

EXAMPLE

Let's consider the system of ordinary differential equations

$$\begin{cases} y_1' = y_2 \\ y_2' = -y_1 \\ y_3' = -y_3 \end{cases} \quad 0 \leq t \leq 1 \quad (2.12)$$

with the boundary condition

$$\begin{cases} y_1(0) = -1.0 \\ y_2(0) = 0.0 \\ y_3(0) = 1.0 \end{cases} \quad (2.13)$$

The exact solution of the problem is

$$\begin{cases} y_1 = -\cos t \\ y_2 = \sin t \\ y_3 = e^{-t} \end{cases} \quad (2.14)$$

The solution at $t = 1$ is

$$\begin{cases} y_1(1) = -\cos(-1) \\ y_2(1) = \sin(1) \\ y_3(1) = e^{-1} \end{cases} \quad (2.15)$$

We shall use any 3 of the 6 values $y_1(0), y_2(0), y_3(0), y_1(1), y_2(1), y_3(1)$ as boundary conditions. This system of ordinary differential equations are solved in various ways:

1. The value of $y_1(0), y_2(0), y_3(0)$ are used to replace the algebraic equations at $t = 0$.
2. The value of $y_1(0), y_2(0), y_3(0)$ are used to replace the algebraic equations at $t = 1$.
3. The value of $y_1(1), y_2(1), y_3(1)$ are used to replace the algebraic equations at $t = 0$.
4. The value of $y_1(1), y_2(1), y_3(1)$ are used to replace the algebraic equations at $t = 1$.
5. The value of $y_1(0)$ and $y_2(1)$ are used to replace the algebraic equations at $t = 0$, and $y_3(0)$ at $t = 1$.
6. As example for showing how to compute several solutions simultaneously, The system of ordinary differential equations are also solved for the following sets of boundary conditions:

$$\begin{cases} y_1^{(1)}(0) = -1.0 \\ y_2^{(1)}(1) = \sin(1) \\ y_3^{(1)}(0) = 1.0 \end{cases} \quad \begin{cases} y_1^{(2)}(0) = -1.0 \\ y_2^{(2)}(1) = \sin(1) \\ y_3^{(2)}(0) = 1.0 \end{cases} \quad (2.16)$$

which are all used to replace the algebraic equations at $t = 0$. It must be noted that the boundary values of both $y_1^{(1)}$ and $y_1^{(2)}$ should be assigned at the same end (in our example, at $t = 0$), and they should be used to replace the algebraic equations at the same end as well (in our example, at $t = 0$). The same is for $y_2^{(1)}, y_2^{(2)}$ and $y_3^{(1)}, y_3^{(2)}$.

The result for all these cases are the same, as boundary conditions are chosen from the same solution.

The program for the example is in the file `Example_Chebcoll_Gauss_Lobatto.cpp`. This file is listed below:

```
#include "stdlib.h"
#include "iostream.h"
#include "math.h"
#include "datatype.h"
#include "nrutil.h"
#include "chebyshev.h"

// X_Coll and T_Coll must be declared as global variables,
// as T_Coll is normally used in the functions computing the
// coefficients and vector of the differential equations.
real *X_Coll,*T_Coll;
```

```

// Function for computing coefficients. In this simple
// example all coefficients are constant. In more general
// cases this function depends on T_Coll.
static real Equ_Coef(int i,int j,int k){
    if(i==1&&j==0)return 1.0;           //a10 at T_Coll[k]
    else if(i==1&&j==1)return 0.0;      //a11 at T_Coll[k]
    else if(i==1&&j==2)return 1.0;      //a12 at T_Coll[k]
    else if(i==1&&j==3)return 0.0;      //a13 at T_Coll[k]
    else if(i==2&&j==0)return 1.0;      //a20 at T_Coll[k]
    else if(i==2&&j==1)return -1.0;     //a21 at T_Coll[k]
    else if(i==2&&j==2)return 0.0;      //a22 at T_Coll[k]
    else if(i==2&&j==3)return 0.0;      //a23 at T_Coll[k]
    else if(i==3&&j==0)return 1.0;      //a30 at T_Coll[k]
    else if(i==3&&j==1)return 0.0;      //a31 at T_Coll[k]
    else if(i==3&&j==2)return 0.0;      //a32 at T_Coll[k]
    else if(i==3&&j==3)return -1.0;     //a33 at T_Coll[k]
    else{
        nrerror("Bad call to Equ_Coef"); //unexpected cases
        return 0.0; // This line is never executed
    }
}

// Function for computing vector. In this simple
// example all components are constant. In more general
// cases they depend on T_Coll.
static real Equ_Vect(int i,int j){
    if(i==1)return 0.0;                 //f1 at T_Coll[k]
    else if(i==2)return 0.0;            //f2 at T_Coll[k]
    else if(i==3)return 0.0;            //f3 at T_Coll[k]
    else{
        nrerror("Bad call to Equ_Vect"); //unexpected cases
        return 0.0; // This line is never executed
    }
}

// The main program
int main(){

    // Declarations
    int i,j,k;
    real a=0.0,b=1.0,**BC_Vect,**Solution;
    int Num_Equ=3,Num_Solution,Num_Coll,N,**BC_Desc;

    // Values used in boundary conditions at t=0

```

```

real y10=-1.0,y20=0.0,y30=1.0;

// Values used in boundary conditions at t=1
real y11,y21,y31;
y11=-cos(1.0);
y21=sin(1.0);
y31=exp(-1.0);

////////////////////////////////////
// Case 1:                                     //
// y1(0),y2(0),y3(0) replacing algebraic equations at t=0 //
////////////////////////////////////
cout<<endl<<"y1(0),y2(0),y3(0) replacing algebraic equations at t=0"
    <<endl;

// Evaluate Num_Solution,Num_Coll,N
Num_Solution=1;
cout<<"input number of collocation points: "<<flush;
cin>>Num_Coll;
N=Num_Coll-1;

// Allocate BC_Desc,BC_Vect,Solution,X_Coll,T_Coll
BC_Desc=matrix(_int_,1,Num_Equ,1,2);
BC_Vect=matrix(_real_,1,Num_Solution,1,Num_Equ);
Solution=tensor(_real_,1,Num_Solution,1,Num_Equ,0,N);
X_Coll=vector(_real_,0,N);
T_Coll=vector(_real_,0,N);

// Evaluate BC_Desc
BC_Desc[1][1]=1; //Boundary value of y1 is given at t=0
BC_Desc[1][2]=1; //Boundary value of y1 is applied at t=0
BC_Desc[2][1]=1; //Boundary value of y2 is given at t=0
BC_Desc[2][2]=1; //Boundary value of y2 is applied at t=0
BC_Desc[3][1]=1; //Boundary value of y3 is given at t=0
BC_Desc[3][2]=1; //Boundary value of y3 is applied at t=0

// Evaluate BC_Vect
BC_Vect[1][1]=y10; BC_Vect[1][2]=y20; BC_Vect[1][3]=y30;

// Compute X_Coll and T_Coll, needed for computing coefficients and
// vectors of the differential equations
ChebCollPnt(a,b,Num_Coll,2,X_Coll,T_Coll);

// Solve the differential equations

```



```

ChebColl_ODES(a,b,Num_Equ,Equ_Coef,Equ_Vect,Num_Solution,BC_Desc,
              BC_Vect,Num_Coll,Solution);

// Output results. The value of y1, y2, y3 at the coordinate T_Coll[i]
// is stored in Solution[1][1][T_Coll[i]], Solution[1][2][T_Coll[i]],
// Solution[1][3][T_Coll[i]] respectively
cout.precision(15);
cout.setf(ios::scientific,ios::floatfield);
cout<<"\n";
for(i=1;i<=Num_Solution;i++)
{
    cout<<"\n"<<"Solution"<<Num_Solution;
    for(j=0;j<=N;j++)
    {
        cout<<"\n"<<T_Coll[j];
        for(k=1;k<=Num_Equ;k++) cout<<"    "<<Solution[i][k][j];
    }
}

cout<<endl;

//Free arrays allocated
free_matrix(BC_Desc,1,Num_Equ,1,2);
free_matrix(BC_Vect,1,Num_Solution,1,Num_Equ);
free_tensor(Solution,1,Num_Solution,1,Num_Equ,0,N);
free_vector(X_Coll,0,N);
free_vector(T_Coll,0,N);

////////////////////////////////////
// Case 2:                                     //
// y1(0),y2(0),y3(0) replacing algebraic equations at t=1 //
////////////////////////////////////
cout<<endl<<"y1(0),y2(0),y3(0) replacing algebraic equations at t=1"
    <<endl;

// Evaluate Num_Solution,Num_Coll,N
Num_Solution=1;
cout<<"input number of collocation points: "<<flush;
cin>>Num_Coll;
N=Num_Coll-1;

// Allocate BC_Desc,BC_Vect,Solution,X_Coll,T_Coll
BC_Desc=matrix(_int_,1,Num_Equ,1,2);
BC_Vect=matrix(_real_,1,Num_Solution,1,Num_Equ);
Solution=tensor(_real_,1,Num_Solution,1,Num_Equ,0,N);

```

```

X_Coll=vector(_real_,0,N);
T_Coll=vector(_real_,0,N);

// Evaluate BC_Desc
BC_Desc[1][1]=1; //Boundary value of y1 is given at t=0
BC_Desc[1][2]=2; //Boundary value of y1 is applied at t=1
BC_Desc[2][1]=1; //Boundary value of y2 is given at t=0
BC_Desc[2][2]=2; //Boundary value of y2 is applied at t=1
BC_Desc[3][1]=1; //Boundary value of y3 is given at t=0
BC_Desc[3][2]=2; //Boundary value of y3 is applied at t=1

// Evaluate BC_Vect
BC_Vect[1][1]=y10; BC_Vect[1][2]=y20; BC_Vect[1][3]=y30;

// Compute X_Coll and T_Coll, needed for computing coefficients and
// vectors of the differential equations
ChebCollPnt(a,b,Num_Coll,2,X_Coll,T_Coll);

// Solve the differential equations
ChebColl_ODES(a,b,Num_Equ,Equ_Coef,Equ_Vect,Num_Solution,BC_Desc,
              BC_Vect,Num_Coll,Solution);

// Output results. The value of y1, y2, y3 at the coordinate T_Coll[i]
// is stored in Solution[1][1][T_Coll[i]], Solution[1][2][T_Coll[i]],
// Solution[1][3][T_Coll[i]] respectively
cout.precision(15);
cout.setf(ios::scientific,ios::floatfield);
cout<<"\n";
for(i=1;i<=Num_Solution;i++)
{
    cout<<"\n"<<"Solution"<<Num_Solution;
    for(j=0;j<=N;j++)
    {
        cout<<"\n"<<T_Coll[j];
        for(k=1;k<=Num_Equ;k++)cout<<"    "<<Solution[i][k][j];
    }
}
cout<<endl;

//Free arrays allocated
free_matrix(BC_Desc,1,Num_Equ,1,2);
free_matrix(BC_Vect,1,Num_Solution,1,Num_Equ);
free_tensor(Solution,1,Num_Solution,1,Num_Equ,0,N);
free_vector(X_Coll,0,N);

```

```

free_vector(T_Coll,0,N);

////////////////////////////////////
// Case 3:                                     //
// y1(1),y2(1),y3(1) replacing algebraic equations at t=0 //
////////////////////////////////////
cout<<endl<<"y1(1),y2(1),y3(1) replacing algebraic equations at t=0"
    <<endl;

// Evaluate Num_Solution,Num_Coll,N
Num_Solution=1;
cout<<"input number of collocation points: "<<flush;
cin>>Num_Coll;
N=Num_Coll-1;

// Allocate BC_Desc,BC_Vect,Solution,X_Coll,T_Coll
BC_Desc=matrix(_int_,1,Num_Equ,1,2);
BC_Vect=matrix(_real_,1,Num_Solution,1,Num_Equ);
Solution=tensor(_real_,1,Num_Solution,1,Num_Equ,0,N);
X_Coll=vector(_real_,0,N);
T_Coll=vector(_real_,0,N);

// Evaluate BC_Desc
BC_Desc[1][1]=2; //Boundary value of y1 is given at t=1
BC_Desc[1][2]=1; //Boundary value of y1 is applied at t=0
BC_Desc[2][1]=2; //Boundary value of y2 is given at t=1
BC_Desc[2][2]=1; //Boundary value of y2 is applied at t=0
BC_Desc[3][1]=2; //Boundary value of y3 is given at t=1
BC_Desc[3][2]=1; //Boundary value of y3 is applied at t=0

// Evaluate BC_Vect
BC_Vect[1][1]=y11; BC_Vect[1][2]=y21; BC_Vect[1][3]=y31;

// Compute X_Coll and T_Coll, needed for computing coefficients and
// vectors of the differential equations
ChebCollPnt(a,b,Num_Coll,2,X_Coll,T_Coll);

// Solve the differential equations
ChebColl_ODES(a,b,Num_Equ,Equ_Coef,Equ_Vect,Num_Solution,BC_Desc,
    BC_Vect,Num_Coll,Solution);

// Output results. The value of y1, y2, y3 at the coordinate T_Coll[i]
// is stored in Solution[1][1][T_Coll[i]], Solution[1][2][T_Coll[i]],
// Solution[1][3][T_Coll[i]] respectively
cout.precision(15);

```

```

cout.setf(ios::scientific,ios::floatfield);
cout<<"\n";
for(i=1;i<=Num_Solution;i++)
{
    cout<<"\n"<<"Solution"<<Num_Solution;
    for(j=0;j<=N;j++)
    {
        cout<<"\n"<<T_Coll[j];
        for(k=1;k<=Num_Equ;k++)cout<<"    "<<Solution[i][k][j];
    }
}

cout<<endl;

//Free arrays allocated
free_matrix(BC_Desc,1,Num_Equ,1,2);
free_matrix(BC_Vect,1,Num_Solution,1,Num_Equ);
free_tensor(Solution,1,Num_Solution,1,Num_Equ,0,N);
free_vector(X_Coll,0,N);
free_vector(T_Coll,0,N);

////////////////////////////////////
// Case 4:                                     //
// y1(1),y2(1),y3(1) replacing algebraic equations at t=1 //
////////////////////////////////////
cout<<endl<<"y1(1),y2(1),y3(1) replacing algebraic equations at t=1"
    <<endl;

// Evaluate Num_Solution,Num_Coll,N
Num_Solution=1;
cout<<"input number of collocation points: "<<flush;
cin>>Num_Coll;
N=Num_Coll-1;

// Allocate BC_Desc,BC_Vect,Solution,X_Coll,T_Coll
BC_Desc=matrix(_int_,1,Num_Equ,1,2);
BC_Vect=matrix(_real_,1,Num_Solution,1,Num_Equ);
Solution=tensor(_real_,1,Num_Solution,1,Num_Equ,0,N);
X_Coll=vector(_real_,0,N);
T_Coll=vector(_real_,0,N);

// Evaluate BC_Desc
BC_Desc[1][1]=2; //Boundary value of y1 is given at t=1
BC_Desc[1][2]=2; //Boundary value of y1 is applied at t=1
BC_Desc[2][1]=2; //Boundary value of y2 is given at t=1

```

```

BC_Desc[2][2]=2; //Boundary value of y2 is applied at t=1
BC_Desc[3][1]=2; //Boundary value of y3 is given at t=1
BC_Desc[3][2]=2; //Boundary value of y3 is applied at t=1

// Evaluate BC_Vect
BC_Vect[1][1]=y11; BC_Vect[1][2]=y21; BC_Vect[1][3]=y31;

// Compute X_Coll and T_Coll, needed for computing coefficients and
// vectors of the differential equations
ChebCollPnt(a,b,Num_Coll,2,X_Coll,T_Coll);

// Solve the differential equations
ChebColl_ODES(a,b,Num_Equ,Equ_Coef,Equ_Vect,Num_Solution,BC_Desc,
              BC_Vect,Num_Coll,Solution);

// Output results. The value of y1, y2, y3 at the coordinate T_Coll[i]
// is stored in Solution[1][1][T_Coll[i]], Solution[1][2][T_Coll[i]],
// Solution[1][3][T_Coll[i]] respectively
cout.precision(15);
cout.setf(ios::scientific,ios::floatfield);
cout<<"\n";
for(i=1;i<=Num_Solution;i++)
{
    cout<<"\n"<<"Solution"<<Num_Solution;
    for(j=0;j<=N;j++)
    {
        cout<<"\n"<<T_Coll[j];
        for(k=1;k<=Num_Equ;k++)cout<<"    "<<Solution[i][k][j];
    }
}

cout<<endl;

//Free arrays allocated
free_matrix(BC_Desc,1,Num_Equ,1,2);
free_matrix(BC_Vect,1,Num_Solution,1,Num_Equ);
free_tensor(Solution,1,Num_Solution,1,Num_Equ,0,N);
free_vector(X_Coll,0,N);
free_vector(T_Coll,0,N);

////////////////////////////////////
// Case 5:                                     //
// y1(0),y2(1) replacing algebraic equations at t=0 and //
// y3(0) replacing algebraic equations at t=1           //
////////////////////////////////////

```

```

cout<<endl<<"y1(0),y2(1) replacing algebraic equations at t=0"
    <<endl<<"y3(0) replacing algebraic equations at t=1"
    <<endl;

// Evaluate Num_Solution,Num_Coll,N
Num_Solution=1;
cout<<"input number of collocation points: "<<flush;
cin>>Num_Coll;
N=Num_Coll-1;

// Allocate BC_Desc,BC_Vect,Solution,X_Coll,T_Coll
BC_Desc=matrix(_int_,1,Num_Equ,1,2);
BC_Vect=matrix(_real_,1,Num_Solution,1,Num_Equ);
Solution=tensor(_real_,1,Num_Solution,1,Num_Equ,0,N);
X_Coll=vector(_real_,0,N);
T_Coll=vector(_real_,0,N);

// Evaluate BC_Desc
BC_Desc[1][1]=1; //Boundary value of y1 is given at t=0
BC_Desc[1][2]=1; //Boundary value of y1 is applied at t=0
BC_Desc[2][1]=2; //Boundary value of y2 is given at t=1
BC_Desc[2][2]=1; //Boundary value of y2 is applied at t=0
BC_Desc[3][1]=1; //Boundary value of y3 is given at t=0
BC_Desc[3][2]=2; //Boundary value of y3 is applied at t=1

// Evaluate BC_Vect
BC_Vect[1][1]=y10; BC_Vect[1][2]=y21; BC_Vect[1][3]=y30;

// Compute X_Coll and T_Coll, needed for computing coefficients and
// vectors of the differential equations
ChebCollPnt(a,b,Num_Coll,2,X_Coll,T_Coll);

// Solve the differential equations
ChebColl_ODES(a,b,Num_Equ,Equ_Coef,Equ_Vect,Num_Solution,BC_Desc,
    BC_Vect,Num_Coll,Solution);

// Output results. The value of y1, y2, y3 at the coordinate T_Coll[i]
// is stored in Solution[1][1][T_Coll[i]], Solution[1][2][T_Coll[i]],
// Solution[1][3][T_Coll[i]] respectively
cout.precision(15);
cout.setf(ios::scientific,ios::floatfield);
cout<<"\n";
for(i=1;i<=Num_Solution;i++)
{
    cout<<"\n"<<"Solution"<<Num_Solution;

```

```

    for(j=0;j<=N;j++)
    {
        cout<<"\n"<<T_Coll[j];
        for(k=1;k<=Num_Equ;k++)cout<<"    "<<Solution[i][k][j];
    }

}
cout<<endl;

//Free arrays allocated
free_matrix(BC_Desc,1,Num_Equ,1,2);
free_matrix(BC_Vect,1,Num_Solution,1,Num_Equ);
free_tensor(Solution,1,Num_Solution,1,Num_Equ,0,N);
free_vector(X_Coll,0,N);
free_vector(T_Coll,0,N);

////////////////////////////////////
// Case 6:                                     //
// Compute two solutions for two sets of boundary conditions //
// y1(0),y2(1),y3(0) replacing algebraic equations at t=0    //
////////////////////////////////////
cout<<endl<<"Compute two solutions for two sets of boundary conditions"
    <<endl<<"y1(0),y2(1),y3(0) replacing algebraic equations at t=0"
    <<endl;

// Evaluate Num_Solution,Num_Coll,N
Num_Solution=2;
cout<<"input number of collocation points: "<<flush;
cin>>Num_Coll;
N=Num_Coll-1;

// Allocate BC_Desc,BC_Vect,Solution,X_Coll,T_Coll
BC_Desc=matrix(_int_,1,Num_Equ,1,2);
BC_Vect=matrix(_real_,1,Num_Solution,1,Num_Equ);
Solution=tensor(_real_,1,Num_Solution,1,Num_Equ,0,N);
X_Coll=vector(_real_,0,N);
T_Coll=vector(_real_,0,N);

// Evaluate BC_Desc
BC_Desc[1][1]=1; //Boundary value of y1 is given at t=0
BC_Desc[1][2]=1; //Boundary value of y1 is applied at t=0
BC_Desc[2][1]=2; //Boundary value of y2 is given at t=1
BC_Desc[2][2]=1; //Boundary value of y2 is applied at t=0
BC_Desc[3][1]=1; //Boundary value of y3 is given at t=0
BC_Desc[3][2]=1; //Boundary value of y3 is applied at t=0

```

```

// Evaluate BC_Vect
BC_Vect[1][1]=y10; BC_Vect[1][2]=y21; BC_Vect[1][3]=y30;
BC_Vect[2][1]=y10; BC_Vect[2][2]=y21; BC_Vect[2][3]=y30;

// Compute X_Coll and T_Coll, needed for computing coefficients and
// vectors of the differential equations
ChebCollPnt(a,b,Num_Coll,2,X_Coll,T_Coll);

// Solve the differential equations
ChebColl_ODES(a,b,Num_Equ,Equ_Coef,Equ_Vect,Num_Solution,BC_Desc,
              BC_Vect,Num_Coll,Solution);

// Output results. The value of y1, y2, y3 at the coordinate T_Coll[i]
// is stored in Solution[1][1][T_Coll[i]], Solution[1][2][T_Coll[i]],
// Solution[1][3][T_Coll[i]] respectively
cout.precision(15);
cout.setf(ios::scientific,ios::floatfield);
cout<<"\n";
for(i=1;i<=Num_Solution;i++)
{
    cout<<"\n"<<"Solution"<<Num_Solution;
    for(j=0;j<=N;j++)
    {
        cout<<"\n"<<T_Coll[j];
        for(k=1;k<=Num_Equ;k++)cout<<"    "<<Solution[i][k][j];
    }
}

cout<<endl;

//Free arrays allocated
free_matrix(BC_Desc,1,Num_Equ,1,2);
free_matrix(BC_Vect,1,Num_Solution,1,Num_Equ);
free_tensor(Solution,1,Num_Solution,1,Num_Equ,0,N);
free_vector(X_Coll,0,N);
free_vector(T_Coll,0,N);
return 0;
}

```


2.4.4 Solution of ordinary differential equations using Chebyshev-Gauss-Radau integration points

We consider the ordinary differential equation system

$$a_{m0} \frac{dy_m}{dt} = \sum_{n=1}^M a_{mn}(t) y_n + f_m(t), \quad t \in [a, b], \quad m = 1, \dots, M \quad (2.17)$$

with boundary conditions

$$y_m(t)_{t=b} = y_m(b) \text{ or no condition for } y_m(t), \quad m = 1, \dots, M \quad (2.18)$$

This system of differential equations is solved by the function

```
int ChebColl_ODES(real a, real b, int Num_Equ,
    real (*Equ_Coef)(int, int, int), real (*Equ_Vect)(int, int),
    int Num_Solution, int *BC_Desc, real **BC_Vect,
    int Num_Coll, real ***Solution)
```

It also gives the solution of the differential equations for several sets of boundary conditions. The variables in it are defined as follows. The inputs are

- **a** and **b**: Defined as before.
- **Num_Equ**: The same as before.
- **Num_Coll**: Number of collocation points as before.
- **Equ_Coef(int i, int j, int k)**: The same as before.
- **Equ_Vect(int i, int j)**: The same as before.
- **Num_Solution**: The same as before.
- **BC_Desc[i]**: Description of the boundary conditions. **BC_Desc[i]=1** means the unknown function y_m has a boundary condition at $t = b$ and this condition is imposed. Otherwise no boundary condition should be imposed to y_m .
- **BC_Vect[i][j]**: Boundary value of the i -th solution of j -th function at the boundary $t = b$.

The output is

- **Solution[i][j][k]**: The same as before.

The returned value has the same meaning as the case using Chebyshev-Gauss-Lobatto integration points.

In this case, for inverting the matrix of the linear algebraic equations of the problem the function **ChebColl_ODES** also calls the function

```
int ChebCollInv(int Num_Equ, int Num_Coll, real **Coef, real **InvCoef)
```

which returns 1 if the matrix is successfully inverted, and 0 when failed. This can be done when a_{m0} are all not zero (even so it may fail sometimes). When some of the coefficients a_{m0} are zero, (2.10) is a combination of differential and algebraic equations, and the blocks on the diagonal of (2.5) are no more all regular. In this case the above function is not applicable. The function

```
int ChebColl_ODES1(real a,real b,int Num_Equ,
    real (*Equ_Coef)(int,int,int),real (*Equ_Vect)(int,int),
    int Num_Solution,int *BC_Desc,real **BC_Vect,
    int Num_Coll,real ***Solution)
```

solve the problem in such situation. In this function the matrix is inverted using Gauss-Jordan elimination:

```
for(i=1;i<=DIM;i++)for(j=1;j<=DIM;j++)Coef1[i][j]=Coef[i][j];
flag=gaussj(Coef1,DIM,Inv_Coef);
```

which returns the value of `flag` having the same meaning as the function `ChebColl_ODES`. The array `Coef1` is destroyed after. The function `ChebColl_ODES1` is applicable to any cases except when the matrix is really singular.

EXAMPLE

Compute the spheroidal free oscillation periods for a self gravitating uniform elastic Earth model. This problem is solved in the book of Lapwood and Usami (1981). It is a simplified case of the spheroidal free oscillation problem of the real Earth models in which the Earth is divided into several co-central layers between which the physical parameters or their derivatives are not continuous. This example may help to understand the program for the spheroidal oscillation problem in the next chapter.

Before writing out the equations of the problem we introduce the following physical parameters which will be used:

- G : The universal constant of gravitation as defined in `parameters.h`.
- r : Distance to the Earth center.
- R : Earth's radius. $R = 6370km$ is adopted in the example.
- ρ : Earth's density. $\rho = 5.52g/cm^3$ is adopted in the example.
- V_p, V_s : Velocities of P and S waves. $V_p = 11.55km/s$ and $V_s = 6.667km/s$ are adopted in our example.
- λ, μ : Lamé's constants. They are computed from the relations

$$V_p = \sqrt{\frac{\lambda + 2\mu}{\rho}}, \quad V_s = \sqrt{\frac{\mu}{\rho}} \quad (2.19)$$

- g : Gravity acceleration. For our case of uniform spherical Earth model it is given by the equation

$$g = \frac{4}{3}\pi G\rho r = \Gamma r, \text{ with } \Gamma = \frac{4}{3}\pi G\rho \quad (2.20)$$

- σ : Angular frequency of oscillation. It is related to the period T with the relation

$$\sigma = \frac{2\pi}{T} \quad (2.21)$$

The differential equations to be solved are

$$\left\{ \begin{array}{l} \frac{dy_1}{dr} = -\frac{2\lambda}{(\lambda+2\mu)r}y_1 + \frac{1}{\lambda+2\mu}y_2 + \frac{n(n+1)\lambda}{(\lambda+2\mu)r}y_3 \\ \frac{dy_2}{dr} = \left\{ -\rho\sigma^2 - \frac{4\rho g}{r} + \frac{4\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_1 - \frac{4\mu}{(\lambda+2\mu)r}y_2 + \\ \left\{ \frac{n(n+1)\rho g}{r} - \frac{2n(n+1)\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_3 + \frac{n(n+1)}{r}y_4 - \rho y_6 \\ \frac{dy_3}{dr} = -\frac{1}{r}y_1 + \frac{1}{r}y_3 + \frac{1}{\mu}y_4 \\ \frac{dy_4}{dr} = \left\{ \frac{\rho g}{r} - \frac{2\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_1 - \frac{\lambda}{(\lambda+2\mu)r}y_2 + \\ \left\{ -\rho\sigma^2 + \frac{2\mu}{(\lambda+2\mu)r^2}[\lambda(2n^2+2n-1) + 2\mu(n^2+n-1)] \right\} y_3 - \\ \frac{3}{r}y_4 - \frac{\rho}{r}y_5 \\ \frac{dy_5}{dr} = 4\pi G\rho y_1 + y_6 \\ \frac{dy_6}{dr} = -\frac{4\pi G n(n+1)\rho}{r}y_3 + \frac{n(n+1)}{r^2}y_5 - \frac{2}{r}y_6 \end{array} \right. \quad (2.22)$$

where n is an integral number which is the degree of spherical harmonics. The boundary conditions are

$$y_2 = 0, \quad y_4 = 0, \quad y_6 = -(n+1)y_5/R. \quad (2.23)$$

at Earth's surface $r = R$.

The case when $n = 0$ should be treated particularly. In this case $y_3 \equiv y_4 \equiv 0$.

Our problem is to find the values of periods T which render the differential equations (2.22) with the boundary conditions (2.23) having non-zero solutions.

The programs are in the file `Example_Chebcoll_Gauss_Radau.cpp` composed of the functions

```
static real Equ_Coef(int i,int j,int k);
static real Equ_Vect(int i,int j);
static int chebcoll_Gauss_Radau(real period,real &f);
static int search_one_mode(int n,real period_start,period_end,
                           real step,real &period,real toll)
void search_many_modes(int n_low,int n_high,int number,
                       real **period);
int main();
```

They are understandable by comments alone. All the programs are listed below:

```
#include "stdlib.h"
#include "stdio.h"
#include "iostream.h"
#include "math.h"
#include "datatype.h"
#include "parameters.h"
#include "nrutil.h"
#include "nr.h"
#include "chebyshev.h"

// In this problem the variable is r rather than t.
// So R_Coll is used to replace T_Coll

// X_Coll and R_Coll must be declared as global variables,
// as R_Coll is normally used in the functions computing
// coefficients and vector of the differential equations.
// R, Rho, Lambda, Mu, Gamma, Omega should be global variables
// for the same reason. They are evaluated in the function main().
real *X_Coll,*R_Coll,R,Rho,Lambda,Mu,Gamma,Sigma;

// Degree (for n) should be a global variable for the same reason.
// Num_Coll will be used in more than one functions so it should
// be a global variable as well
int Degree,Num_Coll;

////////////////////////////////////
// Function for computing coefficients in the differential equations //
// for a given value of Sigma.                                     //
////////////////////////////////////
static real Equ_Coef(int i,int j,int k){

// r, g, n are defined for simplicity
real r=R_Coll[k],g=Gamma*r;
int n=Degree;

// When n=0, y3=y4=0 for any value of r. This is realized by setting
// dy3/dr=0, dy4/dr=0 with boundary conditions y3=0, y4=0 at r=R.
if(n==0&&(i==3||i==4)){
    if(j==0)return 1.0;
    else return 0.0;
}
else if(i==1&&j==0)return 1.0;
else if(i==1&&j==1)return -2.0*Lambda/((Lambda+2.0*Mu)*r);
```

```

else if(i==1&&j==2)return 1.0/(Lambda+2.0*Mu);
else if(i==1&&j==3)return n*(n+1.0)*Lambda/((Lambda+2.0*Mu)*r);
else if(i==2&&j==0)return 1.0;
else if(i==2&&j==1)return -Rho*SQR(Sigma)-4.0*Rho*g/r
    +4.0*Mu*(3.0*Lambda+2.0*Mu)/((Lambda+2.0*Mu)*SQR(r));
else if(i==2&&j==2)return -4.0*Mu/(Lambda+2.0*Mu)/SQR(r);
else if(i==2&&j==3)return n*(n+1.0)*Rho*g/r
    -2.0*n*(n+1.0)*Mu*(3.0*Lambda+2.0*Mu)/((Lambda+2.0*Mu)*SQR(r));
else if(i==2&&j==4)return n*(n+1.0)/r;
else if(i==2&&j==6)return -Rho;
else if(i==3&&j==0)return 1.0;
else if(i==3&&j==1)return -1.0/r;
else if(i==3&&j==3)return 1.0/r;
else if(i==3&&j==4)return 1.0/Mu;
else if(i==4&&j==0)return 1.0;
else if(i==4&&j==1)return Rho*g/r
    -2.0*Mu*(3.0*Lambda+2.0*Mu)/((Lambda+2.0*Mu)*SQR(r));
else if(i==4&&j==2)return -Lambda/((Lambda+2.0*Mu)*r);
else if(i==4&&j==3)return -Rho*SQR(Sigma)
    +(2.0*Mu/((Lambda+2.0*Mu)*SQR(r)))
    *((2.0*SQR(n)+2.0*n-1.0)*Lambda+2.0*(SQR(n)+n-1.0)*Mu);
else if(i==4&&j==4)return -3.0/r;
else if(i==4&&j==5)return -Rho/r;
else if(i==5&&j==0)return 1.0;
else if(i==5&&j==1)return 4.0*PI*G*Rho;
else if(i==5&&j==6)return 1.0;
else if(i==6&&j==0)return 1.0;
else if(i==6&&j==3)return -4.0*PI*G*Rho*n*(n+1.0)/r;
else if(i==6&&j==5)return n*(n+1.0)/SQR(r);
else if(i==6&&j==6)return -2.0/r;
else return 0.0; //all coefficients not mentioned above are zero
}

```

```

////////////////////////////////////
// Function for computing vector. In this simple //
// example all of them are zero                //
////////////////////////////////////
static real Equ_Vect(int i,int j){
return 0.0;
}

```

```

////////////////////////////////////
// Function for solving the differential equations for a given //
// period. First it solves the equation system with three sets of //
// boundary conditions at r=R:                                //

```

```

// (1) y1=1, y3=0, y5=0 //
// (2) y1=0, y3=1, y5=0 //
// (3) y1=0, y3=0, y5=1 //
// without boundary conditions for y2,y4,y6. //
// //
// The case n=0 is a special one. In this case y3=y4=y6=0. The //
// equations are solved with two sets of boundary conditions: //
// (1) y1=1,y3=0,y4=0,y5=0,y6=0 //
// (2) y1=0,y3=0,y4=0,y5=1,y6=0 //
// without boundary condition for y2. //
// //
// As the equations are homogeneous any linear combination of //
// the solutions obtained are also solution of the differential //
// equation system. //
// //
// After the solutions being obtained, A solution which satisfies //
// y4=0 and y6=-(n+1)y5/R at r=R //
// is computed by linearly combining the solutions obtained. //
// //
// Finally the value of y2 at r=R for this solution is set //
// as the value of the function. //
// //
// Take care of the manner that the value of y2 is returned by //
// the parameter f and the returned value of the function, i.e. //
// flag //
////////////////////////////////////

static int chebcoll_Gauss_Radau(real period,real &f){
int i,j,n0=0,n1=1,n2=2,Num_Equ=6,Num_Solution,*BC_Desc,flag;
real **BC_Vect,***Solution,y[4][7],factor[3];

// Evaluate Num_Solution
if(Degree==0)Num_Solution=2;
else Num_Solution=3;

// Compute Sigma from period
Sigma=2*PI/period;

// Allocate arrays
BC_Vect=matrix(_real_,n1,Num_Solution,n1,Num_Equ);
BC_Desc=vector(_int_,n1,Num_Equ);
Solution=tensor(_real_,n1,Num_Solution,n1,Num_Equ,n0,Num_Coll-n1);

// Evaluate boundary conditions
if(Degree==0){

```

```

    BC_Desc[2]=0; //No boundary condition for y2
    BC_Desc[1]=BC_Desc[3]=BC_Desc[4]=BC_Desc[5]=BC_Desc[6]=1;
    BC_Vect[1][1]=1.0; BC_Vect[2][1]=0.0;
    BC_Vect[1][3]=0.0; BC_Vect[2][3]=0.0;
    BC_Vect[1][4]=0.0; BC_Vect[2][4]=0.0;
    BC_Vect[1][5]=0.0; BC_Vect[2][5]=1.0;
    BC_Vect[1][6]=0.0; BC_Vect[2][6]=0.0;
}
else{
    BC_Desc[2]=BC_Desc[4]=BC_Desc[6]=0;
    BC_Desc[1]=BC_Desc[3]=BC_Desc[5]=1;
    BC_Vect[1][1]=1.0; BC_Vect[2][1]=0.0; BC_Vect[3][1]=0.0;
    BC_Vect[1][3]=0.0; BC_Vect[2][3]=1.0; BC_Vect[3][3]=0.0;
    BC_Vect[1][5]=0.0; BC_Vect[2][5]=0.0; BC_Vect[3][5]=1.0;
}

// Solve the differential equations
flag=ChebColl_ODES(0.0,R,Num_Equ,Equ_Coef,Equ_Vect,Num_Solution,
    BC_Desc,BC_Vect,Num_Coll,Solution);

// Compute the solution satisfying  $y_4=0$  and  $y_6=-(n+1)y_5/R$  at  $r=R$ 

//  $y[i][j]$  is the value of the  $i$ -th solution for  $j$ -th variable at  $r=R$ 
for(i=1;i<=Num_Solution;i++)for(j=1;j<=Num_Equ;j++)
    y[i][j]=Solution[i][j][Num_Coll-1];

// Find two independent solutions which have  $y_4=0$  at  $r=R$ . Each of
// them are computed by linearly combining two of the three solutions.
// This step is not needed for  $n=0$ .
if(Degree!=0){
    // First, rearrange the order of the three solutions so that in the last
    // one  $y_4$  has the maximum absolute value.
    for(i=1;i<=2;i++)if(fabs(y[i][4])>fabs(y[3][4]))
        for(j=1;j<=Num_Equ;j++)SWAP(y[3][j],y[i][j]);
    // Then, By linearly combining respectively the first solution and the
    // second solution with the third solution, compute the two
    // solutions of which  $y_4=0$  at  $r=R$ . This is done by multiplying the
    // last one with a factor and then adding it to the first and second one
    for(i=1;i<=2;i++)factor[i]=-y[i][4]/y[3][4];
    for(i=1;i<=2;i++)for(j=1;j<=Num_Equ;j++)y[i][j]+=factor[i]*y[3][j];
}

// Find the solution which has  $y_6=-(n+1)y_5/R$  at  $r=R$  by linearly
// combining the two solutions of which  $y_4=0$  at  $r=R$ 
// First, compute  $y_6+(n+1)y_5/R$  for the two solutions

```

```

for(i=1;i<=2;i++)factor[i]=y[i][6]+((Degree+1.0)/R)*y[i][5];
// Then, rearrange the two solutions so that the last one has larger
// y6+(n+1)y5/R
if(fabs(factor[1])>fabs(factor[2]))for(j=1;j<=Num_Equ;j++){
    SWAP(factor[1],factor[2]);
    SWAP(y[1][j],y[2][j]);
}

// Finally, By linearly combining the first solution with the third
// solution, compute the solutions of which y6+(n+1)y5/R=0 at r=R.
// This is done by multiplying the last one with a factor and then
// adding it to first one
factor[1]/=-factor[2];
for(j=1;j<=Num_Equ;j++)y[1][j]+=factor[1]*y[2][j];

// Free arrays
free_matrix(BC_Vect,n1,Num_Solution,n1,Num_Equ);
free_vector(BC_Desc,n1,Num_Equ);
free_tensor(Solution,n1,Num_Solution,n1,Num_Equ,n0,Num_Coll-n1);

// Print "+" or "-" on screen according as y[1][2] is ">" or "<" 0
if(y[1][2]>0)cout<<"+"<<flush;
if(y[1][2]<0)cout<<"-"<<flush;
// Return y[1][2] as the value of the function
f= y[1][2];
return flag;
}

////////////////////////////////////
// PRINCIPLE FOR SEARCHING THE FREE OSCILLATION PERIODS---- //
// If the function //
//      real chebcoll_Gauss_Radau(real period) //
// returns y2=0 at r=R for the period chosen, then this period is //
// one of the free oscillation period. //
// //
// The function //
//      int search_one_mode(int n,real period_start,real step, //
//      real &period,real toll) //
// try to find one such values of period beginning from period_start //
// to zero. The search is done interval by interval of which the //
// length is step. Here the value of step is negative. //
// //
// The variable of this function are //
// (1) The value of the function -- There are three possible values //
//      0 if no value of y2 is found where y2 changes sign //

```



```

//          1 if a value of period is found where y2 changes sign and //
//          it decreases when approaching the value found           //
//          -1 if a value of period is found where y2 changes sign but //
//          it increases when approaching the value found           //
// (2) n: The degree of the spherical harmonics                      //
// (3) period_start: The period is searched from it to 0             //
// (4) step: The change of sign is checked interval by interval of   //
//       of which the length is step                                 //
// (5) period: The value of period found                             //
// (6) toll: accuracy of the value of period                         //
////////////////////////////////////////////////////////////////
static int search_one_mode(int n,real period_start,real period_end,
                           real step,real &period,real toll){

int i=0;
Degree=n;
i=bisec(chebcoll_Gauss_Radau,period_start,0.0,step,period,toll);
return i;
}

////////////////////////////////////////////////////////////////
// The function which will be called to compute free oscillation    //
// in the main function. It computes "number" periods for every value //
// of "n" between "n_low" and "n_high". As result period[i][j] is the //
// i-th period for n=j. Here i=0,...,number-1.                      //
////////////////////////////////////////////////////////////////
void search_many_modes(int n_low,int n_high,int number,real **period){
int i,j,n,n1=1,n3=3;
real period_start,step,period_temp,toll;

cout.precision(3);
cout.setf(ios::fixed,ios::floatfield);

cout<<endl<<"Computing Spheroidal Free Oscillation Periods"<<endl;

// Accuracy is 0.01 second
toll=0.01;

// Compute the first period for n=0. This case appears only when n_low=0
if(n_low==0){
    n=0;
    // The value of period_temp is the period computed. The next period
    // is searched from period_temp-toll. Its initial value is the
    // largest value of period to be searched
    period_temp=1200;
    do{

```

```

    // Define step to be 1/1000 of the period as the larger the period
    // the larger the difference between successive periods. The search
    // is done from a larger value to low values, hence the step is
    // negative.
    step=-period_temp/1000.0;
    // Begin to search period from period_start=period_temp-toll to
    // low values.
    period_start=period_temp-toll;
    // The lowest value of period to be searched is zero
    period_end=0.0;
    // Find one period and print it to screen.
    i=search_one_mode(n,period_start,period_end,step,period_temp,toll);
    cout<<endl<<"Mode computed: "<<0<<"_S_"<<n<<" - Period="
        <<period_temp<<endl;
    // The value of period found is not acceptable if i=-1.
    if(i==-1)cout<<" - Singular point -"<<"\n";
    }while(i==-1); // Find another period if i=-1.
// If i=0 the value of zero is arrived and stop searching.
if(i==0){
    cout<<endl<<"Mode "<<0<<"_S_"<<0<<endl;
    nrerror("search_many_modes: searching period less then 0");
}
// Save the value of period found.
period[0][0]=period_temp;
}

// Compute the first period for n=1. This case appears only when
// n_low<=1 and n_high>=1
if(n_low<=1&& n_high>=1){
    n=1;
    period_temp=1800;
    do{
        step=-period_temp/1000.0;
        period_start=period_temp-toll;
        period_end=0.0;
        i=search_one_mode(n,period_start,period_end,step,period_temp,toll);
        cout<<endl<<"Mode computed: "<<0<<"_S_"<<n<<" - Period="
            <<period_temp<<endl;
        if(i==-1)cout<<" - Singular point -"<<"\n";
        }while(i==-1);
    if(i==0){
        cout<<endl<<"Mode "<<0<<"_S_"<<0<<endl;
        nrerror("search_many_modes: searching period less then 0");
    }
    period[0][1]=period_temp;
}

```

```

    }

    // Compute the first period for n>=2. In these cases the value of the
    // first period decreases as the value of n increases. The search is done
    // always from n=2 to higher values of n.
    period_temp=2000.0;
    if(n_high>=2)for(n=2;n<=n_high;n++){
        do{
            step=-period_temp/1000.0;
            period_start=period_temp-toll;
            period_end=0.0;
            i=search_one_mode(n,period_start,period_end,step,period_temp,toll);
            cout<<endl<<"Mode computed: "<<0<<"_S_"<<n<<" - Period="
                <<period_temp<<endl;
            if(i==--1)cout<<" - Singular point -"<<"\n";
        }while(i==--1);
        if(i==0){
            cout<<endl<<"Mode "<<0<<"_S_"<<n<<endl;
            nerror("search_many_modes: searching period less then 0");
        }
        if(n>=n_low)period[0][n]=period_temp;
    }

    // Compute another number-1 values of periods for all n_low<n<n_high
    for(j=1;j<=number-1;j++)for(n=n_low;n<=n_high;n++){
        // Begin to search from the preceding period to lower values
        period_temp=period[j-1][n];
        do{
            step=-period[j-1][n]/1000.0;
            period_start=period_temp-toll;
            period_end=0.0;
            i=search_one_mode(n,period_start,period_end,step,period_temp,toll);
            cout<<endl<<"Mode computed: "<<j<<"_T_"<<n<<" - Period="
                <<period_temp<<endl;
            if(i==--1)cout<<" - Singular point -"<<"\n";
        }while(i==--1);
        if(i==0){
            cout<<endl<<"Mode "<<j<<"_S_"<<n<<endl;
            nerror("search_many_modes: searching period less then 0");
        }
        period[j][n]=period_temp;
    }
}

// The main program

```

```

int main(){
int i,j,n0=0,n1=1,n6=6;
int n_low,n_high,number;
real **period,Vp,Vs;

// Input n_low, n_high and number
cout<<endl<<"Inputing data"<<endl;
cout<<"Type the lowest degree: "<<flush;
cin>>n_low;
cout<<endl<<"Type the highest degree: "<<flush;
cin>>n_high;
cout<<endl<<"Type the number of mode to be computed for every degree: "
    <<flush;
cin>>number;

// Input Num_Coll
cout<<endl<<"Type the number of collocation points : "<<flush;
cin>>Num_Coll;

// Allocate arrays
period=matrix(_real_,n0,number-1,n_low,n_high);
X_Coll=vector(_real_,n0,Num_Coll-1);
R_Coll=vector(_real_,n0,Num_Coll-1);

// Evaluate model data
R=6370*1.0e+3;
Rho=5.52*1.0e+3;
Vp=11.55*1.0e+3;
Vs=6.667*1.0e+3;
Mu=SQR(Vs)*Rho;
Lambda=SQR(Vp)*Rho-2.0*Mu;
Gamma=(4.0/3.0)*PI*G*Rho;

// Evaluate X_Coll and R_Coll
ChebCollPnt(0.0,R,Num_Coll,1,X_Coll,R_Coll);

// Compute periods
search_many_modes(n_low,n_high,number,period);

// Output results to screen
printf("\nFree oscillation periods of the modes m_S_n:");
printf("\n  n");
printf("%s%d", "      m=",0);
for(i=1;i<=number-1;i++)printf("%s%d", "      m=",i);
for(j=n_low;j<=n_high;j++){

```

```
    printf("\n%3d",j);
    for(i=0;i<=number-1;i++)printf("%12.2f",period[i][j]);
    }
printf("\n");
free_matrix(period,n0,number-1,n_low,n_high);
return 0;
}
```

Chapter 3

Applications in geophysics

3.1 Introduction

In this introduction we give the file structure of the programs introduced in this chapter. The root directory is still `GeophysicalODEs`. Subdirectories and a part of the files concerned in this chapter are listed as follows where the names ended with a back slash are subdirectories.

```
GeophysicalODEs\  
  Earth_Model.Config  
  Earth_Model_Chebcoll.Config  
  Earth_Models\  
    layer.prem_polynomial  
    layer_description.prem_polynomial  
    layer_radai.prem_polynomial  
    num_coef.prem_polynomial  
    rho.prem_polynomial  
    vp.prem_polynomial  
    vs.prem_polynomial  
    qk.prem_polynomial  
    qmu.prem_polynomial  
    .....  
  Earth_Models_3kms\  
    layer_3kms.prem_Polynomial  
    layer_description_3kms.prem_Polynomial  
    layer_radai_3kms.prem_Polynomial  
    num_data_3kms.prem_Polynomial  
    radii_3kms.prem_Polynomial  
    rho_3kms.prem_Polynomial  
    vp_3kms.prem_Polynomial  
    vs_3kms.prem_Polynomial  
    qk_3kms.prem_Polynomial  
    qmu_3kms.prem_Polynomial
```

```
meanrho_3kms.prem_discrete
.....
model_preparation\
    model_preparation.cpp
    model_preparation.ctl
    model_preparation.out
.....
Earth_Models_Num_Coll\
    num_coll.prem_discrete
.....
Earth's_internal_parameters\
    meanrho\
        nearrho.cpp
        nearrho.ctl
        nearrho.out
        .....
    mass\
        mass.cpp
        mass.ctl
        mass.out
        .....
    gravity\
        gravity.cpp
        gravity.ctl
        gravity.out
        .....
    pressure\
        pressure.cpp
        pressure.ctl
        pressure.out
        .....
    meanmoment\
        meanmoment.cpp
        meanmoment.ctl
        meanmoment.out
        .....
    meanrhominusrho\
        meanrhominusrho.cpp
        meanrhominusrho.ctl
        meanrhominusrho.out
        .....
    flattening\
        flattening.cpp
        flattening.ctl
        flattening.out
```

```

        .....
        momentdifference\
            momentdifference.cpp
            momentdifference.ctl
            momentdifference.out
        .....
Earth's_free_oscillation\
    spheroidal_snrei_odes\
        spheroidal_snrei_odes.cpp
        spheroidal_snrei_odes.ctl
        spheroidal_snrei_odes.out
    totoidal_snrei_odes\
        toroidal_snrei_odes.cpp
        toroidal_snrei_odes.ctl
        toroidal_snrei_odes.out
Earth_tides\
    ln_snrei_odes\
        ln_snrei_odes.cpp
        ln_snrei_odes.ctl
        ln_snrei_odes.out
utilities_geophysics\
    model.h
    model_input.cpp
    model_3kms.cpp
    model_chebcoll.cpp
    model_meanrho_chebcoll.cpp
    model_mass_chebcoll.cpp
    model_gravity_chebcoll.cpp
    model_pressure_chebcoll.cpp
    model_meanmoment_chebcoll.cpp
    model_meanrhominusrho_chebcoll.cpp
    model_flattening_chebcoll.cpp
    model_momentdifference_chebcoll.cpp
    spheroidal_snrei_odes_chebcoll.cpp
    model_lambda_chebcoll.cpp
    model_mu_chebcoll.cpp
    toroidal_snrei_odes_chebcoll.cpp
    tides_static_ln_snrei_odes_chebcoll.cpp
    tides_dynamic_ln_snrei_odes_chebcoll.cpp

```

The file `model.h` contains the prototypes of the functions defined in the files under the subdirectory `utilities_geophysics`. This file is listed below:

```

#ifndef _MODEL_H_
#define _MODEL_H_
// Functions in model_input.cpp

```

```

void model_preparation();
void model_layer();
void model_phys_parameters();
void delete_layer();
void delete_phys_parameters();
void PolyToDisc(int Deg_Poly,real *Poly_Coef,real fac,int Num_Digi,
               real *X_Digi,real *Valu_Digi);
void model_average_oceanless();
// Functions in model_3kms.cpp
void model_3kms_radii(char unit[]);
void model_3kms_parameters();
void model_3kms_output();
void unit_to_si_3kms();
void delete_3kms_radii();
void delete_3kms_parameters();
// Functions in model_chebcoll.cpp
void model_numcoll();
void model_chebcoll();
void unit_to_si_chebcoll();
void delete_chebcoll();
void chebcoll_to_disc(real *param_phys,real *param_data);
void chebcoll_to_poly(real *param_phys,real *param_poly,int *Num_Poly,
                    real accuracy);
void chebcoll_to_3kms(real *param_cheb_phys,real *param_3kms);
real chebcoll_integration(real *param_cheb_phys);
// Functions of application
void model_meanrho_chebcoll();
void delete_meanrho_chebcoll();
void model_mass_chebcoll();um_Poly,real accuracy);
void delete_mass_chebcoll();;
void model_gravity_chebcoll();
void delete_gravity_chebcoll();
void model_pressure_chebcoll();
void delete_pressure_chebcoll();
void model_meanmoment_chebcoll();
void delete_meanmoment_chebcoll();
void model_flattening_chebcoll(real type);
real model_flattening_type();
void delete_flattening_chebcoll();
void model_meanrhominusrho_chebcoll();
void delete_meanrhominusrho_chebcoll();
void model_momentdifference_chebcoll();
void delete_momentdifference_chebcoll();
void model_mu_chebcoll(real Period);
void delete_mu_chebcoll();

```

```

void model_lambda_chebcoll(real Period);
void delete_lambda_chebcoll();
void spheroidal_snrei_odes_chebcoll(int n_low,int n_high,int number,
                                     real **period);
void spheroidal_snrei_odes_chebcoll_eif(int n,real period,real **y);
void toroidal_snrei_odes_chebcoll(int low,int high,int number,
                                   real **period);
void toroidal_snrei_odes_chebcoll_eif(int n,real period,real **y_Coll);
void tides_static_ln_snrei_odes_chebcoll(int n,real &hh,real &ll,
                                          real &kk,int type);
void tides_dynamic_ln_snrei_odes_chebcoll(int n,real &hh,real &ll,
                                          real &kk,real period,int type);

#endif

```

If one or more of these functions are used in a program the file `model.h` should be included.

3.2 Input of Earth model parameters

There exist several Earth models of internal physical parameters. The most recent ones are

1. PEM (Parametrically simple Earth Model) which consists of three models: PEM-A (Average), PEM-C (Continent), PEM-O (Ocean).
2. 1066A and 1066B (Computed from 1066 free oscillation periods).
3. PREM (Preliminary Reference Earth Model).

In all these Earth models the Earth is considered spherically symmetrical and its interior is divided into several co-central spherical layers. The physical parameters are continuous in these layers and may be discontinuous across the boundaries of these layers. The physical parameters are given in two forms:

1. Polynomial: In each layer the physical parameters are given as polynomial of $x = r/R$ where r is the distance to the Earth center and R the Earth's radius.
2. Discrete: In each layer the physical parameters are given at a series of equi-distance points, i.e. at a series of values of r .

The independent physical parameters given are

1. ρ : Density. `Rho` will be used as its synonym.
2. V_p : Velocity of P wave. `Vp` will be used as its synonym.
3. V_s : Velocity of S wave. `Vs` will be used as its synonym.
4. Q_k : Quality factor of the bulk modulus k . `Qk` will be used as its synonym.
5. Q_μ : Quality factor of the shear modulus μ . `Qmu` will be used as its synonym.

The last two parameters are given only in PREM. The PEM and PREM give the physical parameters in both polynomial and discrete forms, the 1066A and 1066B give the physical parameters only in discrete form. More explanation will be given later when using these data.

The Earth model data files are located in the subdirectory `Earth_Models`. The organization of Earth model data will be explained progressively as explaining how they are input.

For convenience of the readers who want to use the results computed but not intend to understand the method in detail for further development, a program is given to create a discrete model in 3 kilometers interval, i.e. the discrete values of the physical parameters of the Earth model are given every 3 kilometers. For all the parameters as function of r computed by our programs, their values are also given in the form of this 3 kilometers model. The 3 kilometers model data files are located in the subdirectory `Earth_Models_3kms`. The organization of data for them will be explained later.

The functions for inputting Earth model data are collected in the file `model_input.cpp`. Before calling these functions, two pointers `FILE *fpctrl`, `*fpout` must be opened. The first one points to the file containing control information and the second the file of output data.

The following global variables are declared in the file `model_input.cpp`:

```
char Model_Dir[50];    //Directory where Earth model data files are located
char Model_Dir_3kms[50]; //Directory where 3kms Earth model data
                        //files are located
char Goal[80];    // Goal of the program, for example, the quantity
                  //to be computed
char Model[80];    // Earth model name
char Model_Ext[30]; // a string identifying the model
char IsOceanless[30]; // average ocean layers with the upmost crust layer
                    // or not, "Oceanless" for yes.
char IsRho[30];    // "RhoIncluded" or "RhoNotIncluded"
char IsVp[30];    // similar to IsRho
char IsVs[30];    // similar to IsRho
char IsQk[30];    // similar to IsRho
char IsQmu[30];    // similar to IsRho
char PolyOrDisc[12]; // "Polynomial" or "Discrete" model
char Int_Method[10]; // method of interpolation for discrete model

int Num_Layer;    // number of continuous layers of the Earth model
char **Description; // name and description of each layer
real *R_Layer;    // radius of layers, contains 'Layers+1' data
int *Num_Data;    // number of Polynomial coefficients or known data
int Num_Total;    //total number of known data
int *Pt_Data;    // pointer to the first data of every layer
real *R_Data;    // discrete value of r of known data
real *Rho_Data;    // polynomial coefficients or discrete value of rho
```

```

real *Vp_Data; // similar to Rho_data
real *Vs_Data; // similar to Rho_data
real *Qk_Data; // similar to Rho_data
real *Qmu_Data; // similar to Rho_data

```

The following are some additional explanation for these variables:

- **Model_Dir**: Full path where the Earth model data files are located.
- **Model_Dir_3kms**: Full path where the 3 kilometers model data files are located.
- **Goal**: Its value could be for example "Computation_of_internal_flattening". It is only a message.
- **Model**: Its value could be for example "Preliminary_Reference_Earth_Model". It is only a message.
- **Model_Ext**: Its value could be "prem_polynomial", "prem_discrete", "1066A_discrete", "1066B_discrete", "pem-a_polynomial", "pem-a_discrete", "pem-c_polynomial", "pem-c_discrete", "pem-o_polynomial", "pem-o_discrete". It is used to identify the files of parameters of different Earth models. For example, the file `Rho.prem_discrete` contains the data of `Rho` of the PREM in discrete form.
- **IsOceanless**: If its value is set to "Oceanless" the ocean layer of the Earth model is averaged with the upmost crust layer by averaging the density and setting the elastic moduli equal to that of the upmost crust layer.
- **IsRho**: Its value is "RhoIncluded" if `Rho` is needed for the computation. The data of `Rho` is read when it is needed. For other values such as "RhoNotIncluded" the program does not read the file containing the data of `Rho`.
- **IsVp**: Its value is "VpIncluded" or "VpNotIncluded"
- **IsVs**: Its value is "VsIncluded" or "VsNotIncluded"
- **IsQk**: Its value is "QkIncluded" or "QkNotIncluded"
- **IsQmu**: Its value is "QmuIncluded" or "QmuNotIncluded"
- **PolyOrDisc**: Its value is "Polynomial" or "Discrete" for polynomial or discrete Earth models.
- **Int_Method**: Its value is "Linear", "Polynomial", "Rational" or "Spline".
- **Num_Layer**: As an example, it equals 13 for "prem_discrete" which is stored in the file `layer.prem_discrete`
- **Description**: `Description[i]` describes the properties of the i -th layer counted from the central sphere. As an example, for "prem_discrete" The value of `Description` is

```

SolidInnerCore
LiquidOuterCore
SolidMantle
SolidMantle
SolidMantle
SolidMantle
SolidMantle
SolidMantle
SolidMantleNonIsotropic
SolidMantleNonIsotropic
SolidCrust
SolidCrust
LiquidOcean

```

which is stored in the file `layer_description.prem.discrete`.

- **R_Layer:** `R_Layer[0]=0`, `R_Layer[i]` equals radius of the *i*-th layer. It is stored in the files like `layer_radius.prem.discrete` as an example.
- **Num_Data:** For polynomial models `Num_Data[i]` equals the number of polynomial coefficients of the *i*-th layer, i.e., the degree of the polynomial plus 1. For discrete model `Num_Data[i]` equals the number of points where the parameters are given in the *i*-th layer. It is stored in the files like `num_disc_data.prem.discrete` or `num_poly_coef.prem.polynomial` as examples.
- **Num_Total:** Total number of known data, i.e. the sum of all `Num_Data[i]`.
- **Pt_Data:** Pointer to the first data of every layer. As an example, for discrete models `Rho_Data[Pt_Data[i]]` is the density at the innermost point of the *i*-th layer.
- **R_Data:** Only needed in discrete models. `R_Data[i]` equals the radius of the *i*-th point where the parameters are given counted from the center. It is stored in the files like `data_radius.prem.discrete` as an example.
- **Rho_Data:** `Rho_Data[i]` equals the value of density at the radius of `R_Data[i]` for discrete models, or the coefficient of x^{i-1} for polynomial model. It is stored in the files like `rho_disc.prem.discrete` or `rho_poly.prem.polynomial` as an example.
- **Vp_Data:** Similar to `Rho_data`
- **Vs_Data:** Similar to `Rho_data`
- **Qk_Data:** Similar to `Rho_data`
- **Qmu_Data:** Similar to `Rho_data`

For illustrating the variables more clearly, we take as an example the density of the PREM in both discrete and polynomial form.

For the PREM in discrete form, we have for `rho`:

i	R_Data[i]	Rho_Data[i]	layer	
				R_Layer[0]= 0
1	0	13.0885	1	Pt_Data[1]= 1,Num_Data[1]=14
2	100	13.0863	1	
3	200	13.0798	1	
4	300	13.0689	1	
5	400	13.0536	1	
6	500	13.0340	1	
7	600	13.0101	1	
8	700	12.9818	1	
9	800	12.9491	1	
10	900	12.9121	1	
11	1000	12.8707	1	
12	1100	12.8250	1	
13	1200	12.7749	1	
14	1221.5	12.7636	1	R_Layer[1]=1221.5
15	1221.5	12.1663	2	Pt_Data[2]=15,Num_Data[2]=24
16	1300	12.1250	2	
17	1400	12.0692	2	
18	1500	12.0099	2	
19	1600	11.9468	2	
20	1700	11.8799	2	
21	1800	11.8090	2	
22	1900	11.7340	2	
23	2000	11.6548	2	
24	2100	11.5712	2	
25	2200	11.4831	2	
26	2300	11.3904	2	
27	2400	11.2930	2	
28	2500	11.1907	2	
29	2600	11.0833	2	
30	2700	10.9709	2	
31	2800	10.8532	2	
32	2900	10.7301	2	
33	3000	10.6015	2	
34	3100	10.4673	2	
35	3200	10.3273	2	
36	3300	10.1813	2	
37	3400	10.0294	2	
38	3480	9.90349	2	R_Layer[2]=3480
39	3480	5.56645	3	Pt_Data[3]=39,Num_Data[3]=4
40	3500	5.55641	3	
41	3600	5.50642	3	
42	3630	5.49145	3	R_Layer[3]=3630
43	3630	5.49145	4	Pt_Data[4]=43,Num_Data[4]=21

44	3700	5.45657	4	
45	3800	5.40681	4	
46	3900	5.35706	4	
47	4000	5.30724	4	
48	4100	5.25729	4	
49	4200	5.20713	4	
50	4300	5.15669	4	
51	4400	5.10590	4	
52	4500	5.05469	4	
53	4600	5.00299	4	
54	4700	4.95073	4	
55	4800	4.89783	4	
56	4900	4.84422	4	
57	5000	4.78983	4	
58	5100	4.73460	4	
59	5200	4.67844	4	
60	5300	4.62129	4	
61	5400	4.56307	4	
62	5500	4.50372	4	
63	5600	4.44317	4	R_Layer[4]=5600
64	5600	4.44316	5	Pt_Data[5]=64,Num_Data[5]=3
65	5650	4.41241	5	
66	5701	4.38071	5	R_Layer[5]=5701
67	5701	3.99214	6	Pt_Data[6]=67,Num_Data[6]=3
68	5736	3.98399	6	
69	5771	3.97584	6	R_Layer[6]=5771
70	5771	3.97584	7	Pt_Data[7]=70,Num_Data[7]=5
71	5821	3.91282	7	
72	5871	3.84980	7	
73	5921	3.78677	7	
74	5971	3.72378	7	R_Layer[7]=5971
75	5971	3.54325	8	Pt_Data[8]=75,Num_Data[8]=5
76	6016	3.51639	8	
77	6061	3.48951	8	
78	6106	3.46264	8	
79	6151	3.43578	8	R_Layer[8]=6151
80	6151	3.35950	9	Pt_Data[9]=80,Num_Data[9]=5
81	6186	3.36330	9	
82	6221	3.36710	9	
83	6256	3.37091	9	
84	6291	3.37471	9	R_Layer[9]=6291
85	6291	3.37471	10	Pt_Data[10]=85,,Num_Data[10]=4
86	6311	3.37688	10	
87	6331	3.37906	10	
88	6346.6	3.38076	10	R_Layer[10]=6346.6

89	6346.6	2.9	11	Pt_Data[11]=89,Num_Data[11]=2
90	6356	2.9	11	R_Layer[11]=6356
91	6356	2.6	12	Pt_Data[12]=91,Num_Data[12]=2
92	6368	2.6	12	R_Layer[12]=6368
93	6368	1.02	13	Pt_Data[13]=93,Num_Data[13]=2
94	6371	1.02	13	R_Layer[13]=6371
				Pt_Data[14]=95

The other parameters V_p , V_s , Q_k , Q_{mu} are similar to Rho . Every variable is stored sequentially in a file. For PREM the correspondences between variables and files are listed below:

variables	files
Num_Layer	layer.prem_discrete
Description	layer_description.prem_discrete
R_Layer	layer_radii.prem_discrete
Num_Data	num_data.prem_discrete
R_Data	radii_data.prem_discrete
Rho_Data	rho.prem_discrete
V_p _Data	vp.prem_discrete
V_s _Data	vs.prem_discrete
Q_k _Data	qk.prem_discrete
Q_{mu} _Data	qmu.prem_discrete

The density of PREM in polynomial form is

1. $13.0885 - 8.8381x^2$
2. $12.5815 - 1.2638x - 3.6426x^2 - 5.5281x^3$
3. $7.9565 - 6.4761x + 5.5283x^2 - 3.0807x^3$
4. $7.9565 - 6.4761x + 5.5283x^2 - 3.0807x^3$
5. $7.9565 - 6.4761x + 5.5283x^2 - 3.0807x^3$
6. $5.3197 - 1.4836x$
7. $11.2494 - 8.0298x$
8. $7.1089 - 3.8045x$
9. $2.6910 + 0.6924x$
10. $2.6910 + 0.6924x$
11. 2.900
12. 2.600

13. 1.020

where $x = r/R$, R being the radius of the Earth. The variables are

i	Rho_Data[i]	layer	
1	13.0885	1	Pt_Data[1]= 1,Num_Data[1]=3
2	0	1	
3	-8.8381	1	
4	12.5815	2	Pt_Data[2]= 4,Num_Data[2]=4
5	-1.2638	2	
6	-3.6426	2	
7	-5.5281	2	
8	7.9565	3	Pt_Data[3]= 8,Num_Data[3]=4
9	-6.4761	3	
10	5.5283	3	
11	-3.0807	3	
12	7.9565	4	Pt_Data[4]=12,Num_Data[4]=4
13	-6.4761	4	
14	5.5283	4	
15	-3.0807	4	
16	7.9565	5	Pt_Data[5]=16,Num_Data[5]=4
17	-6.4761	5	
18	5.5283	5	
19	-3.0807	5	
20	5.3197	6	Pt_Data[6]=20,Num_Data[6]=2
21	-1.4836	6	
22	11.2494	7	Pt_Data[7]=22,Num_Data[7]=2
23	-8.0298	7	
24	7.1089	8	Pt_Data[8]=24,Num_Data[8]=2
25	-3.8045	8	
26	2.6910	9	Pt_Data[9]=26,Num_Data[9]=2
27	0.6924	9	
28	2.6910	10	Pt_Data[10]=28,Num_Data[10]=2
29	0.6924	10	
30	2.900	11	Pt_Data[11]=30,Num_Data[11]=1
31	2.600	12	Pt_Data[12]=31,Num_Data[12]=1
32	1.020	13	Pt_Data[13]=32,Num_Data[13]=1
			Pt_Data[14]=33

The other parameters V_p , V_s , Q_k , Q_{μ} are also similar to Rho . All variables are stored sequentially in a file. For PREM the correspondences between variables and files are listed below:

variables	files
Num_Layer	layer.prem_polynomial
Description	layer_description.prem_polynomial

R_Layer	layer_radaii.prem_polynomial
Num_Data	num_coef.prem_polynomial
Rho_Data	rho.prem_polynomial
Vp_Data	vp.prem_polynomial
Vs_Data	vs.prem_polynomial
Qk_Data	qk.prem_polynomial
Qmu_Data	qmu.prem_polynomial

where Num_Layer, Description, R_Layer are the same as in the discrete model.

Three functions are designed for inputting Earth model data:

```
void model_preparation();
void model_layer();
void model_phys_parameters();
```

Before explaining the function `model_preparation()` it must be noticed that the file `model_input.cpp` has a line

```
#include "Earth_Model.Config"
```

The file `Earth_Model.Config` is located in the directory `GeophysicalODEs`. It is listed below:

```
#ifdef __WINDOWS
#define Earth_Models_Path "d:\\GeophysicalODEs\\Earth_models\\"
#define Earth_Models_3kms_Path "d:\\GeophysicalODEs\\Earth_models_3kms\\"
#endif
#ifdef __UNIX
#define Earth_Models_Path "/dos/GeophysicalODEs/Earth_models/"
#define Earth_Models_3kms_Path "/dos/GeophysicalODEs/Earth_models_3kms/"
#endif
```

In this file `Earth_Models_Path` is defined as the full path of the Earth model data files and `Earth_Models_3kms_Path` the full path of the 3 kilometers model data files. In the function `model_preparation()`, `Earth_Models_Path` is evaluated to `Model_Dir` and `Earth_Models_3kms_Path` to `Model_Dir_3kms`. This mechanism is for making the programs running on WINDOWS like as well as UNIX like operating systems. For Windows, the macro `__WINDOWS` should be defined as compiler option, and for UNIX the macro `__UNIX` should be defined as compiler option. The function `model_preparation()` also read `Goal`, `Model`, `Model_Ext`, `IsRho`, `IsVp`, `IsVs`, `IsQk`, `IsQmu`, `PolyOrDisc` and `Int_Method` sequentially from the file pointer `fpctrl`. Finally this function output the results to the file pointer `fpout`. As the function `fscanf` is used for inputting strings, no space is allowed in the values of these variable. For example, the value of `Model` could be `"Preliminary_Reference_Earth_Model"`, but it couldn't be `"Preliminary Reference Earth Model"`.

The function `model_layer()` read `Num_Layer`, `Description`, `R_Layer`, `Num_Data` and compute `Pt_Data` and `Num_Total`. The data read are also output to the file pointer `fpout`.

The function `model_phys_parameters()` reads the necessary physical parameters of the Earth model. If the Earth model is discrete, it first allocates and reads `R_Data`. Then it allocates and reads the necessary ones of `Rho_Data`, `Vp_Data`, `Vs_Data`, `Qk_Data` and `Qmu_Data` for both the discrete or the polynomial models. The data read are also output to the file pointer `fpout`.

Two functions are designed to free memories occupied by the arrays of Earth model data:

```
void delete_layer();
void delete_phys_parameters();
```

The function `delete_layer()` free the memory occupied by `Description`, `R_Layer`, `Num_Data`, `Pt_Data` and `PolyOrDisc`. This function is called when these variables are no more needed.

The function `delete_phys_parameters()` free the memory occupied by the necessary ones of `Rho_Data`, `Vp_Data`, `Vs_Data`, `Qk_Data` and `Rho_Data`. The function is called when the variables are not needed any more.

Another function in this file is

```
void PolyToDisc(int Deg_Poly, real *Poly_Coef, real fac,
               int Num_Digi, real *X_Digi, real *Valu_Digi);
```

which computes discrete values from a polynomial. This function is suitable for computing the discrete values of the physical parameters at a set of values of r for polynomial Earth models. The inputs are

- `Deg_Poly`: Degree of the polynomial.
- `Poly_Coef`: `Poly_Coef[i]` ($i=0, \dots, \text{Deg_Poly}$) is the coefficient of x^i .
- `fac`: A normalization factor. See the explication of `X_Digi` for further details.
- `Num_Digi`: Number of discrete values to be computed.
- `X_Digi`: `X_Digi[i]` ($i=1, \dots, \text{Num_Digi}$) is the i -th value of the variable where the parameter is to be computed. The variable in the polynomial is normalized by the factor `fac`, i.e. the variable in the polynomial is in fact the value of the vector `X_Digi` divided by `fac`. When applied to the Earth model, `fac=R`, and `X_Digi` are a set of values of r .

The output is

- `Valu_Digi`: `Valu_Digi[i]` ($i=1, \dots, \text{Num_Digi}$) is the value of the parameter at `X_Digi[i]`.

Very often an Earth model is needed to be averaged to an oceanless Earth model. This is done by averaging the ocean (and sediments) layer(s) and the upmost crust layer of the Earth model to a single crust layer. The density of this averaged layer is the average density, and the elastic parameters λ and μ be the same as the upmost crust layer. This is done by the function

```
void model_average_oceanless();
```

It does the following things one after the other if `IsOceanless` is set to "Oceanless":

- Compute the mean density of the layers to be averaged and change the values of `Rho_Data` to the averaged values for these layers.
- Change the array `Description` to adapt to the newly averaged Earth model. The description of the layers above the crust are all changed to "SolidCrust".
- If "VpIncluded" is evaluated to `IsVp` and "VsIncluded" to `IsVs`, then it compute V_p and V_s using the mean density for the upmost crust layer, and change the values of `Vp_Data`, and `Vs_Data` of the layers to be averaged to the values computed.
- If "QkIncluded" is evaluated to `IsQk` and "QmuIncluded" to `IsQmu`, then evaluate the value of Q_k and Q_μ of the upmost crust layer to the layers above.

In this averaged oceanless Earth model, the layer averaged from the ocean (and sediments) layer(s) and the upmost crust layer of the original Earth model is still separated as many layers as in the original Earth model, all of them having the same physical parameters. This is convenient for programing without changing the number of layers.

3.3 Computation of an Earth model in 3 kilometers interval

It is for readers who only want to use the results computed by the programs in this report but not intend to understand the method for further development that the Earth models of 3 kilometers interval are created. For any Earth model selected, all the internal physical parameters in function of r are computed or interpolated at every 3 kilometers. The same will be done for such parameters computed in our programs. Thus the values of all these parameters for any value of r could be obtained with sufficient accuracy simply by linear interpolation using their values of the 3 kilometers interval Earth model.

The functions for computing the 3 kilometers interval Earth models are in the file `model_3kms.cpp`. In this file the following global variables are declared:

```
int *Num_Data_3kms;    // number of data in every layers
int Num_Total_3kms;    //total number of data
int *Pt_Data_3kms;     // pointer to the first data of every layer
real *R_3kms;          // discrete value of r where physical parameters are given
real *Rho_3kms;        // discrete value of rho
real *Vp_3kms;         // similar to Rho_data
real *Vs_3kms;         // similar to Rho_data
real *Qk_3kms;         // similar to Rho_data
real *Qmu_3kms;        // similar to Rho_data
```

Their meanings are similar to the corresponding variables of a discrete Earth model.

The functions for producing the 3 kilometers interval Earth model are

```

void model_3kms_radai(char unit[]);
void model_3kms_parameters();
void unit_to_si_3kms();
void model_3kms_output();
void delete_3kms_radai();
void delete_3kms_parameters();

```

The function `model_3kms_radai(char unit[])` computes `Num_Data_3kms`, `Num_Total_3kms`, `Pt_Data_3km`, and `R_3kms`. The value of the variable `unit` should be "km" or "m" according as the unit of the radius of the Earth model is kilometer or meter.

The function `model_3kms_parameters()` computes the physical parameters `Rho_3kms`, `Vp_3kms`, `Vs_3kms`, `Qk_3kms`, and `Qmu_3kms`.

The function `unit_to_si_3kms()` transforms the the units of the parameters computed to SI units.

The function `model_3kms_output()` writes sequentially the parameters of the 3 kilometers interval Earth model to the corresponding files in the subdirectory `Earth_Models_3kms`. For the Earth model `prem_polynomial` the variables and the corresponding files for the variables are listed below:

variables	files
<code>Num_Layer</code>	<code>layer_3kms.prem_polynomial</code>
<code>Description</code>	<code>layer_description_3kms.prem_polynomial</code>
<code>R_Layer</code>	<code>layer_radai_3kms.prem_polynomial</code>
<code>Num_Data_3kms</code>	<code>num_data_3kms.prem_polynomial</code>
<code>R_3kms</code>	<code>radai_3kms.prem_polynomial</code>
<code>Rho_3kms</code>	<code>rho_3kms.prem_polynomial</code>
<code>Vp_3kms</code>	<code>vp_3kms.prem_polynomial</code>
<code>Vs_3kms</code>	<code>vs_3kms.prem_polynomial</code>
<code>Qk_3kms</code>	<code>qk_3kms.prem_polynomial</code>
<code>Qmu_3kms</code>	<code>qmu_3kms.prem_polynomial</code>

The function `delete_3kms_radai()` delete the arrays allocated in the function `model_3kms_radai((char unit[]))`.

The function `delete_3kms_parameters()` delete the arrays allocated in the function `model_3kms_parameters()`.

The main program for creating the Earth models of 3 kilometers interval is in the file `model_preparation.cpp`. Here we print out this file for explaining how to use the functions presented in the previous section:

```

#include "stdlib.h"
#include "stdio.h"
#include "string.h"
#include "datatype.h"
#include "nrutil.h"
#include "model.h"

```

```
// Global variables declared in model_input.cpp
extern char IsRho[30]; // "RhoIncluded" or "RhoNotIncluded"
extern char IsVp[30]; // similar to IsRho
extern char IsVs[30]; // similar to IsRho
extern char IsQk[30]; // similar to IsRho
extern char IsQmu[30]; // similar to IsRho
extern int Num_Layer; // number of continuous layers of the Earth model

// Global variables declared in model_3kms.cpp
extern int *Num_Data_3kms; // number of data in 3 kms interval
extern int Num_Total_3kms; //total number of data in 3 kms interval
extern int *Pt_Data_3kms; // pointer to the first data of every
                        // layer in 3 kms interval
extern real *R_3kms; // discrete value of r of data in 3 kms interval
extern real *Rho_3kms; // discrete value of rho in 3 kms interval
extern real *Vp_3kms; // similar to Rho_data
extern real *Vs_3kms; // similar to Rho_data
extern real *Qk_3kms; // similar to Rho_data
extern real *Qmu_3kms; // similar to Rho_data

// Declare the file pointers fpctl and fpout
FILE *fpctl,*fpout;

// The main program
int main(){
int i,j,n0=0,n1=1;

// Control information is in the file model_preparation.ctl
fpctl=fopen("model_preparation.ctl","r");

// Results are output to the file model_preparation.out
fpout=fopen("model_preparation.out","w");

// Input control information
model_preparation();

// Input data for layer division
model_layer();

// Input physical parameters of the Earth model
model_phys_parameters();

// Use an averaged Oceanless Earth model?
model_average_oceanless();
```

```

// Compute values of Num_Data_3kms[i], Num_Total_3kms,
// Pt_Data_3kms[i], R_3kms[i]
model_3kms_radaii("km");

// Compute values of Rho_3kms[i], Vp_3kms[i], Vs_3kms[i],
// Qk_3kms[i], Qmu_3kms[i]
model_3kms_parameters();

// Convert the 3 kilometers model to SI unit
unit_to_si_3kms();

// Output the 3 kilometers interval model to the direstory
// Earth_Model_3kms
model_3kms_output();

// Write results to file pointer fpout, i.e. to the file
// model_preparation.out
fprintf(fpout, "\n3kms discrete values of the model:");
fprintf(fpout, "\nLayer Number      r");
if(strcmp(IsRho, "RhoIncluded")==0)fprintf(fpout, "          Rho");
if(strcmp(IsVp, "VpIncluded")==0)fprintf(fpout, "          Vp");
if(strcmp(IsVs, "VsIncluded")==0)fprintf(fpout, "          Vs");
if(strcmp(IsQk, "QkIncluded")==0)fprintf(fpout, "          Qk");
if(strcmp(IsQmu, "QmuIncluded")==0)fprintf(fpout, "          Qmu");

for(i=1; i<=Num_Layer; i++)
for(j=Pt_Data_3kms[i]; j<=Pt_Data_3kms[i+1]-1; j++){
    fprintf(fpout, "\n%3d%6d", i, j);
    fprintf(fpout, "%10.1f", R_3kms[j]);
    if(strcmp(IsRho, "RhoIncluded")==0)fprintf(fpout, "%15.5f", Rho_3kms[j]);
    if(strcmp(IsVp, "VpIncluded")==0)fprintf(fpout, "%15.5f", Vp_3kms[j]);
    if(strcmp(IsVs, "VsIncluded")==0)fprintf(fpout, "%15.5f", Vs_3kms[j]);
    if(strcmp(IsQk, "QkIncluded")==0)fprintf(fpout, "%15.2f", Qk_3kms[j]);
    if(strcmp(IsQmu, "QmuIncluded")==0)fprintf(fpout, "%15.2f", Qmu_3kms[j]);
}

// Delete arrays no more needed
delete_3kms_radaii();
delete_3kms_parameters();
delete_layer();
delete_phys_parameters();

// Close the files associated to the file pointers fpctl and fpout.
fclose(fpctl);
fclose(fpout);

```

```
return 0;
}
```

This program should be easy to be understood.

3.4 Preparation for use in Chebyshev collocation method

As the Earth center is normally the only singular point of the geophysical ordinary differential equations, for the layer No.1 the Chebyshev-Gauss-Radau integration points which don't include the Earth center are adopted, and for the other layers the Chebyshev-Gauss-Lobatto integration points are adopted.

The functions are in the file `model_chebcoll.cpp`. Before calling these functions the file pointer `fpout` must be opened.

The following global variables are declared in this file:

```
char Num_Coll_Dir[80]; //directory where locates the files storing
                        //number of collocation points
int *Num_Coll; //number of collocation points in each layer
int *Pt_Coll; //pointer to the first collocation data of each layer
int Coll_Total; //total number of collocation points
real *X_Coll; //value of x at collocation points for all layers
real *R_Coll; //value of r at collocation point for all layers
real *Rho_Coll; //value of rho at collocation points for all layers
real *Vp_Coll; //similar to *Rho_Coll
real *Vs_Coll; //similar to *Rho_Coll
real *Qk_Coll; //similar to *Rho_Coll
real *Qmu_Coll; //similar to *Rho_Coll
```

In more detail:

- `Num_Coll_Dir`: Full path of the files storing the number of collocation points. it is evaluated through the file `Earth_Model_Chebcoll.Config` which is included in the file `model_chebcoll.cpp`. The file `Earth_Model_Chebcoll.Config` has the lines

```
#ifdef __WINDOWS
#define Earth_Model_Num_Coll_Path \
    "d:\\GeophysicalODEs\\Earth_Models_Num_Coll\\"
#endif
#ifdef __UNIX
#define Earth_Model_Num_Coll_Path \
    "/dos/GeophysicalODEs/Earth_Models_Num_Coll/"
#endif
```

where `Earth_Model_Num_Coll_Path` is defined the full path of the directory of the files containing the files of the number of collocation points. With this mechanism this directory could be re-assigned by merely modifying this file. This is for running the program on different operation systems.

- `Num_Coll[i]`: Number of collocation points in the i -th layer. They are stored in the files in the subdirectory `Earth_Models_Num_Coll` in the directory `GeophysicalODEs`. For the Earth model `prem_discrete`, the file name is `num_coll.prem_discrete`.
- `Pt_Coll`: Similar to `Pt_Data`, `Pt_Coll[1]=1`, `Pt_Coll[i+1] = Pt_Coll[i] + Num_Coll[i]`, $i=1,\dots,\text{Num_Layer}$.
- `Coll_Total`: Sum of all `Num_Coll[i]`
- `X_Coll`: Every layer is projected to $-1 \leq x \leq 1$. `X_Coll` is the array of the values of x at collocation points, stored successively from layer No. 1 to No. `Num_Layer`
- `R_Coll`: values of r at collocation points, also stored successively from layer no. 1 to no. `Num_Layer`
- `Rho_Coll[i]`: Value of ρ at `R_Coll[i]`.
- `Vp_Coll`: Similar to `Rho_Coll`
- `Vs_Coll`: Similar to `Rho_Coll`
- `Qk_Coll`: Similar to `Rho_Coll`
- `Qmu_Coll`: Similar to `Rho_Coll`

From the above definitions of variables we see that in the i -th layer the value of x at the j -th collocation point is `X_Coll[Pt_Coll[i]+j]`, $j=0,\dots,\text{Num_Coll}[i]-1$, remembering that the collocation points x_j are counted for $j = 0, \dots, \text{Num_Coll}[i]-1$. The same is for the other variables.

The functions are explained one after the other in the following text.

The function

```
void model_numcoll();
```

checks if the file of the number of collocation points exists. If the file exists the function displays the numbers of collocation points and asks if they are to be re-assigned. Otherwise the function asks to assign the number of collocation points.

The function

```
void model_chebcoll();
```

first inputs the number of collocation points for each layer, i.e. the vector `Num_Coll[i]`, $i=1,\dots,\text{Num_Layer}$, and computes `Pt_Coll`, then computes the value of x , r at collocation points. After this it allocates the arrays and computes the values for the necessary physical parameters at collocation points. For the polynomial model the function `PolyToDisc` is used. For the discrete model interpolation is used according to the method evaluated to `Int_Method`. It finally outputs the results to the file pointer `fpout`.

The function

```
void unit_to_si_chebcoll();
```

transforms the parameters at collocation points to the unit system SI.

The function

```
void delete_chebcoll();
```

free the memories of the array allocated in the function `model_chebcoll()`.

The function

```
void chebcoll_to_disc(real *param_cheb_phys, real *param_disc)
```

computes the discrete model of a physical parameter from its value at collocation points. The input `*param_cheb_phys` may be any array of physical parameters in the same form as `*Rho_Coll`, and the output `*param_disc` is in the same form as `*Rho_Data` for discrete model.

The function

```
void chebcoll_to_poly(real *param_cheb_phys, real *param_poly_output,
    int *Num_Poly_output, real accuracy)
```

computes the polynomial model of a physical parameter from its value at collocation points. The input `*param_cheb_phys` is the same as the last function. Another input value `accuracy` is the relative accuracy of the output expected. It is used to avoid too high degree polynomial models. The outputs `*param_poly_output` and `*Num_Poly_output` are respectively in the same forms as `*Rho_Data` and `Num_Data` of a polynomial model.

The function

```
void chebcoll_to_3kms(real *param_cheb_phys, real *param_3kms);
```

computes the values of a physical parameter for the 3 kilometers interval Earth model. Before calling this function the function `model_3kms_radii(char unit[])` must be executed.

The function

```
real chebcoll_integration(real *param_cheb_phys)
```

computes the integration of a parameter over the radius of the Earth. The input `param_cheb_phys` are the values of the parameter at collocation points in the same form as `Rho_Coll`. For Example, the invocation

```
y=chebcoll_integration(Rho_Coll)
```

computes $y = \int_0^R \rho(r) dr$.

3.5 Computation of some internal parameters determined by density

In this section some internal physical parameters related to density are computed. Results are listed for the PREM in polynomial form. In all cases the ocean layer is averaged with the upmost crust layer.

3.5.1 Mean density inside any co-central sphere

The mean density D inside the sphere of radius r is

$$D(r) = \frac{3}{r^3} \int_0^r \rho(x) x^2 dx \quad (3.1)$$

where $r \leq R$. It satisfies the ordinary differential equation

$$\frac{dD}{dr} = -\frac{3}{r}D + \frac{3\rho}{r} \quad (3.2)$$

Although D can be computed from the integral formula (3.1) for any value of r , but as its values at the collocation points will be used for other applications, we compute it by solving the ordinary differential equation (3.2). The Earth center $r = 0$ is a singular point of the differential equation. It implies a boundary condition

$$D = \rho \text{ at } r = 0 \quad (3.3)$$

which could be easily obtained by multiplying both sides of (3.2) with r and then setting $r = 0$. This is in agreement with the physical property of D . The boundary conditions across the boundaries of the layers of the Earth model are D continuous.

The computation is done layer by layer from the center to the surface of the Earth. For the layer No.1, i.e. the central sphere, no boundary condition is imposed. For other layers, the values of D at the inner most collocation point is imposed to be the value of D at the outermost collocation points in the adjacent inner layer, thus assures the continuity of D across the boundaries of adjacent layers.

The function `main()` is in the file `meanrho.cpp`. For illustrating how the function depicted in the last section are used, the full list of this file is given below:

```
#include "stdlib.h"
#include "stdio.h"
#include "string.h"
#include "fstream.h"
#include "datatype.h"
#include "nrutil.h"
#include "model.h"

extern int Coll_Total, Num_Total, Num_Layer;
extern int *Pt_Coll, *Pt_Data;
extern real *X_Coll, *R_Coll, *R_Data;
extern char PolyOrDisc[12];
extern char Model_Ext[30];
extern char IsOceanless[30];
extern char Model_Dir_3kms[80];
extern int Num_Total_3kms;
extern real *MeanRho_Coll;
```

```
FILE *fpctl,*fpout;

int main(){
int i,j,n1=1,*Num_Poly,flag;
real *MeanRho_Data,*MeanRho_3kms;

fpctl=fopen("meanrho.ctl","r");
fpout=fopen("meanrho.out","w");
model_preparation();
model_layer();
model_phys_parameters();

// average to an oceanless Earth model?
model_average_oceanless();

// input collocation points
model_numcoll();

// compute Earth's internal physical parameters at collocation points
model_chebcoll();

// transform physical parameters at collocation points to SI unit system
unit_to_si_chebcoll();

// original Earth model parameters no more needed
delete_phys_parameters();

// compute mean density at collocation points. The array storing mean
// density at collocation points is real *MeanRho_Coll declared and
// allocated in the file model_meanrho_chebcoll.cpp which contains the
// function model_meanrho_chebcoll()
flag=model_meanrho_chebcoll();
if(flag==0)nrerror("Computation failed");

// compute mean density for the 3 kilometers interval Earth model
model_3kms_radii("m");
MeanRho_3kms=vector(_real_,n1,Num_Total_3kms);
chebcoll_to_3kms(MeanRho_Coll,MeanRho_3kms);

// write the value of mean density at 3 kilometers interval to a
// file named meanrho_3kms with the extension given by Model_Ext
// in the subdirectory Earth_Models_3kms.
char meanrho3kms[80];
strcpy(meanrho3kms,Model_Dir_3kms);
strcat(meanrho3kms,"\\D_3kms.");
```

```

if(strcmp(IsOceanless,"Oceanless")==0)strcat(meanrho3kms,"oceanless_");
strcat(meanrho3kms,Model_Ext);
ofstream fpmeanrho3kms(meanrho3kms,ios::out);
fpmeanrho3kms.precision(10);
for(i=1;i<=Num_Total_3kms;i++)fpmeanrho3kms<<MeanRho_3kms[i]<<endl;
fpmeanrho3kms.close();

// output mean density at collocation points to the file pointer fpout
fprintf(fpout,"\nMeanRho at collocation points:");
fprintf(fpout,"\nLayer Number      x");
fprintf(fpout,"      r");
fprintf(fpout,"      MeanRho");
fprintf(fpout,"\n      ");
fprintf(fpout,"      (km)");
fprintf(fpout,"      (g/cm**3)");
for(i=1;i<=Num_Layer;i++)
for(j=Pt_Coll[i];j<=Pt_Coll[i+1]-1;j++){
    fprintf(fpout,"\n%3d%6d",i,j-Pt_Coll[i]);
    fprintf(fpout,"%11.5f",X_Coll[j]);
    fprintf(fpout,"%9.2f",R_Coll[j]*1.0e-3);
    fprintf(fpout,"%11.6f",MeanRho_Coll[j]*1.0e-3);
}

// for discrete Earth models, compute mean density for the values of r
// where the physical parameters are given in the Earth model, and
// then write them to the file pointer fpout
if(strcmp(PolyOrDisc,"Discrete")==0){
    MeanRho_Data=vector(_real_,n1,Num_Total);
    chebcoll_to_disc(MeanRho_Coll,MeanRho_Data);
    fprintf(fpout,"\nDiscrete model of mean density:");
    fprintf(fpout,"\nLayer Number      r      MeanRho");
    fprintf(fpout,"\n      (km)      (g/cm**3)");
    for(i=1;i<=Num_Layer;i++)
    for(j=Pt_Data[i];j<=Pt_Data[i+1]-1;j++){
        fprintf(fpout,"\n%3d%6d",i,j);
        fprintf(fpout,"%9.2f",R_Data[j]*1.0e-3);
        fprintf(fpout,"%11.5f",MeanRho_Data[j]*1.0e-3);
    }
    free_vector(MeanRho_Data,n1,Num_Total);
}

// for polynomial Earth models, compute polynomial expression for mean
// density and then write them to the file pointer fpout
if(strcmp(PolyOrDisc,"Polynomial")==0){
    MeanRho_Data=vector(_real_,n1,Coll_Total);

```

```

    Num_Poly=vector(_int_,n1,Num_Layer);
    chebcoll_to_poly(MeanRho_Coll,MeanRho_Data,Num_Poly,1.0e-6);
    fprintf(fpout,"\nPolynomial model of mean density:");
    fprintf(fpout,"\nLayer Degree    Coefficient");
    fprintf(fpout,"\n                (g/cm**3)");
    for(i=1;i<=Num_Layer;i++)
    for(j=Pt_Coll[i];j<=Pt_Coll[i]+Num_Poly[i]-1;j++){
        fprintf(fpout,"\n%3d%6d",i,j-Pt_Coll[i]);
        fprintf(fpout,"%17.5e",MeanRho_Data[j]*1.0e-3);
    }
    free_vector(MeanRho_Data,n1,Coll_Total);
    free_vector(Num_Poly,n1,Num_Layer);
}

// remove arrays allocated
free_vector(MeanRho_3kms,n1,Num_Total_3kms);
delete_meanrho_chebcoll();
delete_chebcoll();
delete_layer();
fclose(fpctl);
fclose(fpout);
cout<<"mean rho successfully computed"<<endl;
return 0;
}

```

The control information is in the file `meanrho.ct1` which is listed below.

```

Computation_of_mean_density_inside_a_co-central_sphere
Preliminary_reference_Earth_model
prem_polynomial
Oceanless
RhoIncluded
nVpIncluded
nVsIncluded
nQkIncluded
nQmuIncluded
Polynomial
Spline

```

The values of mean density at collocation points are computed by the function `model_meanrho_chebcoll()` in the file `model_meanrho_chebcoll.cpp`. In this file the array for mean density at collocation points is declared as a global variable:

```
real *MeanRho_Coll; //mean rho at collocation points
```

It is allocated in the function `model_meanrho_chebcoll()`. When it is no longer needed it can be deleted using the function `delete_meanrho_chebcoll()` which is in the same

file. The same as the values of density, the value of mean density in i -th layer at j -th collocation point is `MeanRho_Coll[Pt_Coll[i]+j]`, $j=0, \dots, \text{Num_Coll}[i]-1$.

The file `model_meanrho_chebcoll.cpp` is understandable with the help of comments in it. No further explanation will be given. It is listed below:

```
#include "stdlib.h"
#include "stdio.h"
#include "math.h"
#include "datatype.h"
#include "nrutil.h"
#include "chebyshev.h"
#include "parameters.h"

extern int Num_Layer;
extern real *R_Layer;
extern int *Num_Coll;
extern int *Pt_Coll;
extern int Coll_Total;
extern real *X_Coll;
extern real *R_Coll;
extern real *Rho_Coll;

real *MeanRho_Coll; //mean density at collocation points

static int layer;

// function for computing the coefficients of the differential equation
// in the layer-th layer at k-th collocation point
static real Equ_Coef(int i,int j,int k){
if(i==1&&j==0)return 1.0;
if(i==1&&j==1)return -3.0/R_Coll[Pt_Coll[layer]+k];
else return 0.0;
}

// function for computing the vector of the differential equation
// in layer-th layer at j-th collocation point
static real Equ_Vect(int i,int j){
if(i==1)return 3.0*Rho_Coll[Pt_Coll[layer]+j]/R_Coll[Pt_Coll[layer]+j];
else return 0.0;
}

// function for computing the mean density at collocation points
// by solving differential equations
int model_meanrho_chebcoll(){
int i,n0=0,n1=1,n2=2,Num_Equ=1,Num_Solution=1,flag=1;
```

```

real **BC_Vect,***Solution;

// BC_Desc1 is used to characterize the boundary condition in 1-st layer
// and BC_Desc in other layers
int *BC_Desc1,**BC_Desc;

// allocate necessary arrays
MeanRho_Coll=vector(_real_,n1,Coll_Total);
BC_Vect=matrix(_real_,n1,Num_Solution,n1,Num_Equ);
BC_Desc1=vector(_int_,n1,Num_Equ);
BC_Desc=matrix(_int_,n1,Num_Equ,n1,n2);

// no boundary condition is imposed for the 1-st layer
BC_Desc1[1]=0;

// for other layers the value at the inner boundary is imposed at the
// same boundary as boundary condition
BC_Desc[1][1]=1; BC_Desc[1][2]=1;

// solution of the differential equation is done layer by layer
for(layer=1;layer<=Num_Layer;layer++){
    // the dimension of the array solution is different from layer to layer
    // as the number collocation points are different for different layers
    Solution=tensor(_real_,n1,Num_Solution,n1,n1,n0,Num_Coll[layer]-n1);

    if(layer==1){
        // in the 1st layer the Chebychev-Gauss-Radau integration points are
        // used in the collocation method
        if(flag==1)flag=ChebColl_ODES1(R_Layer[layer-1],R_Layer[layer],
            Num_Equ,Equ_Coef,Equ_Vect,Num_Solution,BC_Desc1,BC_Vect,
            Num_Coll[layer],Solution);
    }else{
        // in other layers the value of the unknown function at the outmost
        // collocation point is imposed as the boundary condition at the
        // innermost collocation point
        BC_Vect[1][1]=MeanRho_Coll[Pt_Coll[layer]-1];
        // the Chebychev-Gauss-Lobatto integration points are used in the
        // collocation method
        if(flag==1)flag=ChebColl_ODES1(R_Layer[layer-1],R_Layer[layer],
            Num_Equ,Equ_Coef,Equ_Vect,Num_Solution,BC_Desc,BC_Vect,
            Num_Coll[layer],Solution);
    }
    // copy the values of the known function at collocation points computed
    // from the array solution to the array assigned for it
}

```



```

    for(i=0;i<=Num_Coll[layer]-1;i++)
        MeanRho_Coll[Pt_Coll[layer]+i]=Solution[1][1][i];
    // free the array solution
    free_tensor(Solution,n1,Num_Solution,n1,n1,n0,Num_Coll[layer]-n1);
}

// delete arrays allocated in this function
free_matrix(BC_Vect,n1,Num_Solution,n1,Num_Equ);
free_matrix(BC_Desc,n1,Num_Equ,n1,n2);
free_vector(BC_Desc1,n1,Num_Equ);
return flag;
}

// function for deleting the array allocated in the previous function
void delete_meanrho_chebcoll(){
int n1=1;
free_vector(MeanRho_Coll,n1,Coll_Total);
}

```

Here we give the result for the PREM in polynomial form. The file meanrho.out is listed below.

```

=====
Goal: Computation_of_mean_density_inside_a_co-central_sphere
=====
Earth Model: Preliminary_reference_Earth_model
Location of Earth Model Files: d:\GeophysicalODEs\Earth_models
Location of 3kms Earth Model Files: d:\GeophysicalODEs\Earth_models_3kms
Extension of Earth model data files: prem_polynomial
Ocean layer averaged with the upmost crust layer
Parameters required: Rho
Type Of The Earth Model: Polynomial
=====

Original Model Data
=====

Number of layers of the model: 13
Description of every layer
  1   SolidCore
  2   LiquidCore
  3   SolidMantle
  4   SolidMantle
  5   SolidMantle
  6   SolidMantle
  7   SolidMantle

```

```

8      SolidMantle
9      SolidMantleNonIsotropic
10     SolidMantleNonIsotropic
11     SolidCrust
12     SolidCrust
13     LiquidOcean
Radius of layers(km):
0:      0.0
1:    1221.5
2:    3480.0
3:    3630.0
4:    5600.0
5:    5701.0
6:    5771.0
7:    5971.0
8:    6151.0
9:    6291.0
10:   6346.6
11:   6356.0
12:   6368.0
13:   6371.0
Number of polynomial coefficients in each layer:
Layer    1  2  3  4  5  6  7  8  9 10 11 12 13
Number    3  4  4  4  4  2  2  2  2  2  1  1  1
Polynomial coefficients of Rho(g/cm**3):
1:      13.08850      0.00000     -8.83810
2:      12.58150     -1.26380     -3.64260     -5.52810
3:       7.95650     -6.47610      5.52830     -3.08070
4:       7.95650     -6.47610      5.52830     -3.08070
5:       7.95650     -6.47610      5.52830     -3.08070
6:       5.31970     -1.48360
7:      11.24940     -8.02980
8:       7.10890     -3.80450
9:       2.69100      0.69240
10:      2.69100      0.69240
11:       2.90000
12:       2.60000
13:       1.02000
Number of collocation points
Layer    1  2  3  4  5  6  7  8  9 10 11 12 13
Number   20 20 10 20  8  8  8  8  8  8  8  8  8
Data at collocation points
Layer Number      x      r      Rho
              (km)  (g/cm**3)
1           0   -0.99676    1.98  13.08850

```

1	1	-0.97094	17.75	13.08843
1	2	-0.91998	48.87	13.08798
1	3	-0.84519	94.55	13.08655
1	4	-0.74851	153.60	13.08336
1	5	-0.63245	224.48	13.07753
1	6	-0.50000	305.38	13.06819
1	7	-0.35460	394.18	13.05467
1	8	-0.20003	488.58	13.03652
1	9	-0.04027	586.16	13.01369
1	10	0.12054	684.37	12.98652
1	11	0.27822	780.67	12.95580
1	12	0.42869	872.57	12.92271
1	13	0.56806	957.70	12.88879
1	14	0.69272	1033.83	12.85577
1	15	0.79944	1099.01	12.82551
1	16	0.88546	1151.54	12.79976
1	17	0.94854	1190.07	12.78012
1	18	0.98705	1213.59	12.76781
1	19	1.00000	1221.50	12.76361
2	0	-1.00000	1221.50	12.16633
2	1	-0.98636	1236.90	12.15839
2	2	-0.94582	1282.69	12.13429
2	3	-0.87947	1357.60	12.09330
2	4	-0.78914	1459.61	12.03429
2	5	-0.67728	1585.93	11.95591
2	6	-0.54695	1733.11	11.85687
2	7	-0.40170	1897.14	11.73621
2	8	-0.24549	2073.54	11.59374
2	9	-0.08258	2257.50	11.43039
2	10	0.08258	2444.00	11.24857
2	11	0.24549	2627.96	11.05244
2	12	0.40170	2804.36	10.84796
2	13	0.54695	2968.39	10.64278
2	14	0.67728	3115.57	10.44587
2	15	0.78914	3241.89	10.26688
2	16	0.87947	3343.90	10.11541
2	17	0.94582	3418.81	10.00015
2	18	0.98636	3464.60	9.92800
2	19	1.00000	3480.00	9.90344
3	0	-1.00000	3480.00	5.56646
3	1	-0.93969	3484.52	5.56419
3	2	-0.76604	3497.55	5.55766
3	3	-0.50000	3517.50	5.54767
3	4	-0.17365	3541.98	5.53543
3	5	0.17365	3568.02	5.52241

3	6	0.50000	3592.50	5.51018
3	7	0.76604	3612.45	5.50023
3	8	0.93969	3625.48	5.49373
3	9	1.00000	3630.00	5.49148
4	0	-1.00000	3630.00	5.49148
4	1	-0.98636	3643.43	5.48478
4	2	-0.94582	3683.37	5.46488
4	3	-0.87947	3748.72	5.43235
4	4	-0.78914	3837.70	5.38809
4	5	-0.67728	3947.88	5.33325
4	6	-0.54695	4076.26	5.26919
4	7	-0.40170	4219.33	5.19743
4	8	-0.24549	4373.20	5.11958
4	9	-0.08258	4533.66	5.03738
4	10	0.08258	4696.34	4.95268
4	11	0.24549	4856.80	4.86750
4	12	0.40170	5010.67	4.78402
4	13	0.54695	5153.74	4.70457
4	14	0.67728	5282.12	4.63162
4	15	0.78914	5392.30	4.56763
4	16	0.87947	5481.28	4.51496
4	17	0.94582	5546.63	4.47568
4	18	0.98636	5586.57	4.45141
4	19	1.00000	5600.00	4.44320
5	0	-1.00000	5600.00	4.44320
5	1	-0.90097	5605.00	4.44014
5	2	-0.62349	5619.01	4.43155
5	3	-0.22252	5639.26	4.41908
5	4	0.22252	5661.74	4.40518
5	5	0.62349	5681.99	4.39260
5	6	0.90097	5696.00	4.38387
5	7	1.00000	5701.00	4.38074
6	0	-1.00000	5701.00	3.99212
6	1	-0.90097	5704.47	3.99131
6	2	-0.62349	5714.18	3.98905
6	3	-0.22252	5728.21	3.98578
6	4	0.22252	5743.79	3.98216
6	5	0.62349	5757.82	3.97889
6	6	0.90097	5767.53	3.97663
6	7	1.00000	5771.00	3.97582
7	0	-1.00000	5771.00	3.97582
7	1	-0.90097	5780.90	3.96334
7	2	-0.62349	5808.65	3.92837
7	3	-0.22252	5848.75	3.87783
7	4	0.22252	5893.25	3.82174

7	5	0.62349	5933.35	3.77120
7	6	0.90097	5961.10	3.73623
7	7	1.00000	5971.00	3.72375
8	0	-1.00000	5971.00	3.54326
8	1	-0.90097	5979.91	3.53794
8	2	-0.62349	6004.89	3.52303
8	3	-0.22252	6040.97	3.50148
8	4	0.22252	6081.03	3.47756
8	5	0.62349	6117.11	3.45601
8	6	0.90097	6142.09	3.44110
8	7	1.00000	6151.00	3.43577
9	0	-1.00000	6151.00	3.35949
9	1	-0.90097	6157.93	3.36024
9	2	-0.62349	6177.36	3.36235
9	3	-0.22252	6205.42	3.36541
9	4	0.22252	6236.58	3.36879
9	5	0.62349	6264.64	3.37184
9	6	0.90097	6284.07	3.37395
9	7	1.00000	6291.00	3.37471
10	0	-1.00000	6291.00	3.37471
10	1	-0.90097	6293.75	3.37500
10	2	-0.62349	6301.47	3.37584
10	3	-0.22252	6312.61	3.37705
10	4	0.22252	6324.99	3.37840
10	5	0.62349	6336.13	3.37961
10	6	0.90097	6343.85	3.38045
10	7	1.00000	6346.60	3.38075
11	0	-1.00000	6346.60	2.90000
11	1	-0.90097	6347.07	2.90000
11	2	-0.62349	6348.37	2.90000
11	3	-0.22252	6350.25	2.90000
11	4	0.22252	6352.35	2.90000
11	5	0.62349	6354.23	2.90000
11	6	0.90097	6355.53	2.90000
11	7	1.00000	6356.00	2.90000
12	0	-1.00000	6356.00	2.28340
12	1	-0.90097	6356.59	2.28340
12	2	-0.62349	6358.26	2.28340
12	3	-0.22252	6360.66	2.28340
12	4	0.22252	6363.34	2.28340
12	5	0.62349	6365.74	2.28340
12	6	0.90097	6367.41	2.28340
12	7	1.00000	6368.00	2.28340
13	0	-1.00000	6368.00	2.28340
13	1	-0.90097	6368.15	2.28340

13	2	-0.62349	6368.56	2.28340
13	3	-0.22252	6369.17	2.28340
13	4	0.22252	6369.83	2.28340
13	5	0.62349	6370.44	2.28340
13	6	0.90097	6370.85	2.28340
13	7	1.00000	6371.00	2.28340

MeanRho at collocation points:

Layer	Number	x	r (km)	MeanRho (g/cm**3)
1	0	-0.99676	1.98	13.088499
1	1	-0.97094	17.75	13.088459
1	2	-0.91998	48.87	13.088188
1	3	-0.84519	94.55	13.087332
1	4	-0.74851	153.60	13.085418
1	5	-0.63245	224.48	13.081916
1	6	-0.50000	305.38	13.076317
1	7	-0.35460	394.18	13.068201
1	8	-0.20003	488.58	13.057313
1	9	-0.04027	586.16	13.043613
1	10	0.12054	684.37	13.027311
1	11	0.27822	780.67	13.008878
1	12	0.42869	872.57	12.989028
1	13	0.56806	957.70	12.968674
1	14	0.69272	1033.83	12.948865
1	15	0.79944	1099.01	12.930703
1	16	0.88546	1151.54	12.915257
1	17	0.94854	1190.07	12.903471
1	18	0.98705	1213.59	12.896085
1	19	1.00000	1221.50	12.893569
2	0	-1.00000	1221.50	12.893569
2	1	-0.98636	1236.90	12.866593
2	2	-0.94582	1282.69	12.792177
2	3	-0.87947	1357.60	12.685918
2	4	-0.78914	1459.61	12.564308
2	5	-0.67728	1585.93	12.438715
2	6	-0.54695	1733.11	12.314112
2	7	-0.40170	1897.14	12.190925
2	8	-0.24549	2073.54	12.067560
2	9	-0.08258	2257.50	11.942295
2	10	0.08258	2444.00	11.814340
2	11	0.24549	2627.96	11.684277
2	12	0.40170	2804.36	11.554134
2	13	0.54695	2968.39	11.427223
2	14	0.67728	3115.57	11.307842
2	15	0.78914	3241.89	11.200855

2	16	0.87947	3343.90	11.111215
2	17	0.94582	3418.81	11.043459
2	18	0.98636	3464.60	11.001228
2	19	1.00000	3480.00	10.986880
3	0	-1.00000	3480.00	10.986880
3	1	-0.93969	3484.52	10.965795
3	2	-0.76604	3497.55	10.905642
3	3	-0.50000	3517.50	10.815062
3	4	-0.17365	3541.98	10.706489
3	5	0.17365	3568.02	10.593923
3	6	0.50000	3592.50	10.490844
3	7	0.76604	3612.45	10.408685
3	8	0.93969	3625.48	10.355942
3	9	1.00000	3630.00	10.337786
4	0	-1.00000	3630.00	10.337786
4	1	-0.98636	3643.43	10.284338
4	2	-0.94582	3683.37	10.129590
4	3	-0.87947	3748.72	9.889028
4	4	-0.78914	3837.70	9.584656
4	5	-0.67728	3947.88	9.240753
4	6	-0.54695	4076.26	8.880105
4	7	-0.40170	4219.33	8.521527
4	8	-0.24549	4373.20	8.178856
4	9	-0.08258	4533.66	7.861129
4	10	0.08258	4696.34	7.573445
4	11	0.24549	4856.80	7.318058
4	12	0.40170	5010.67	7.095412
4	13	0.54695	5153.74	6.904971
4	14	0.67728	5282.12	6.745794
4	15	0.78914	5392.30	6.616899
4	16	0.87947	5481.28	6.517447
4	17	0.94582	5546.63	6.446813
4	18	0.98636	5586.57	6.404584
4	19	1.00000	5600.00	6.390531
5	0	-1.00000	5600.00	6.390531
5	1	-0.90097	5605.00	6.385319
5	2	-0.62349	5619.01	6.370771
5	3	-0.22252	5639.26	6.349889
5	4	0.22252	5661.74	6.326904
5	5	0.62349	5681.99	6.306365
5	6	0.90097	5696.00	6.292244
5	7	1.00000	5701.00	6.287222
6	0	-1.00000	5701.00	6.287222
6	1	-0.90097	5704.47	6.283040
6	2	-0.62349	5714.18	6.271369

6	3	-0.22252	5728.21	6.254623
6	4	0.22252	5743.79	6.236200
6	5	0.62349	5757.82	6.219747
6	6	0.90097	5767.53	6.208440
6	7	1.00000	5771.00	6.204420
7	0	-1.00000	5771.00	6.204420
7	1	-0.90097	5780.90	6.192955
7	2	-0.62349	5808.65	6.160905
7	3	-0.22252	5848.75	6.114785
7	4	0.22252	5893.25	6.063856
7	5	0.62349	5933.35	6.018197
7	6	0.90097	5961.10	5.986721
7	7	1.00000	5971.00	5.975511
8	0	-1.00000	5971.00	5.975511
8	1	-0.90097	5979.91	5.964640
8	2	-0.62349	6004.89	5.934396
8	3	-0.22252	6040.97	5.891247
8	4	0.22252	6081.03	5.844100
8	5	0.62349	6117.11	5.802273
8	6	0.90097	6142.09	5.773680
8	7	1.00000	6151.00	5.763543
9	0	-1.00000	6151.00	5.763543
9	1	-0.90097	6157.93	5.755435
9	2	-0.62349	6177.36	5.732922
9	3	-0.22252	6205.42	5.700921
9	4	0.22252	6236.58	5.666121
9	5	0.62349	6264.64	5.635401
9	6	0.90097	6284.07	5.614487
9	7	1.00000	6291.00	5.607089
10	0	-1.00000	6291.00	5.607089
10	1	-0.90097	6293.75	5.604161
10	2	-0.62349	6301.47	5.595986
10	3	-0.22252	6312.61	5.584249
10	4	0.22252	6324.99	5.571326
10	5	0.62349	6336.13	5.559776
10	6	0.90097	6343.85	5.551834
10	7	1.00000	6346.60	5.549010
11	0	-1.00000	6346.60	5.549010
11	1	-0.90097	6347.07	5.548427
11	2	-0.62349	6348.37	5.546795
11	3	-0.22252	6350.25	5.544439
11	4	0.22252	6352.35	5.541828
11	5	0.62349	6354.23	5.539478
11	6	0.90097	6355.53	5.537854
11	7	1.00000	6356.00	5.537274

12	0	-1.00000	6356.00	5.537274
12	1	-0.90097	6356.59	5.536362
12	2	-0.62349	6358.26	5.533807
12	3	-0.22252	6360.66	5.530120
12	4	0.22252	6363.34	5.526035
12	5	0.62349	6365.74	5.522360
12	6	0.90097	6367.41	5.519820
12	7	1.00000	6368.00	5.518914
13	0	-1.00000	6368.00	5.518914
13	1	-0.90097	6368.15	5.518687
13	2	-0.62349	6368.56	5.518053
13	3	-0.22252	6369.17	5.517137
13	4	0.22252	6369.83	5.516120
13	5	0.62349	6370.44	5.515205
13	6	0.90097	6370.85	5.514571
13	7	1.00000	6371.00	5.514345

Polynomial model of mean density:

Layer Degree Coefficient
(g/cm**3)

1	0	1.30885e+001
1	1	2.31807e-013
1	2	-5.30286e+000
2	0	4.99595e+001
2	1	-8.59384e+002
2	2	9.32929e+003
2	3	-6.17527e+004
2	4	2.72058e+005
2	5	-8.27798e+005
2	6	1.75359e+006
2	7	-2.54609e+006
2	8	2.42001e+006
2	9	-1.35785e+006
2	10	3.41217e+005
3	0	1.00810e+002
3	1	-3.79326e+002
3	2	5.40238e+002
3	3	-2.68829e+002
4	0	2.84447e+002
4	1	-2.05126e+003
4	2	6.90395e+003
4	3	-1.34226e+004
4	4	1.60552e+004
4	5	-1.17249e+004
4	6	4.81654e+003
4	7	-8.55780e+002

5	0	1.84352e+001
5	1	-2.07622e+001
5	2	8.03117e+000
6	0	2.45962e+001
6	1	-3.32287e+001
6	2	1.42686e+001
7	0	1.51479e+001
7	1	-1.23686e+001
7	2	2.75471e+000
8	0	3.65866e+001
8	1	-7.25758e+001
8	2	5.86332e+001
8	3	-1.71201e+001
9	0	5.24111e+001
9	1	-1.14053e+002
9	2	9.38704e+001
9	3	-2.67039e+001
10	0	2.64093e+001
10	1	-3.53523e+001
10	2	1.44672e+001
11	0	1.34725e+001
11	1	-7.95401e+000
12	0	3.47302e+001
12	1	-4.87390e+001
12	2	1.95232e+001
13	0	1.52163e+001
13	1	-9.70196e+000

3.5.2 Mass inside any co-central sphere

The mass inside any co-central sphere of radius r inside the Earth is computed from the mean density with the formula

$$m = \frac{4}{3}\pi r^3 D \quad (3.4)$$

At collocation points, the values of m are computed from the values of D according to this formula. The functions are in the file `model_mass_chebcoll.cpp`. A global variable

```
real *Mass_Coll; //mass at collocation points
```

for the values of m at collocation points is declared in this file. The function `model_mass_chebcoll()` allocate this array and compute it. And the function `delete_mass_chebcoll` delete it.

The function `model_meanrho_chebcoll()` must be executed before calling the function `model_mass_chebcoll()`.

The main function is in the file `mass.cpp`. The control information is in the file `mass.ct1`. The result is written to the file `mass.out`.

For the PREM in polynomial form the result is listed below.

Layer	Degree	Coefficient (10**24kg)	Layer	Degree	Coefficient (10**24kg)
1	0	1.21160e-017	5	2	7.68032e+000
1	1	-2.34751e-015	6	0	3.35319e+000
1	2	1.07928e-013	6	1	-6.97569e+000
1	3	1.41776e+001	6	2	9.70200e+000
1	4	1.24043e-011	7	0	5.03319e+000
1	5	-5.74409e+000	7	1	-2.04168e+001
2	0	4.53187e-003	7	2	3.32366e+001
2	1	3.89289e-012	7	3	-1.18605e+001
2	2	-2.98604e-011	8	0	4.08995e-001
2	3	1.36284e+001	8	1	3.82498e-001
2	4	-1.02672e+000	8	2	5.19258e+000
2	5	-2.36741e+000	9	0	6.75401e+000
2	6	-2.99404e+000	9	1	-1.25271e+001
3	0	1.50241e+000	9	2	1.17569e+001
3	1	-3.79574e+000	10	0	6.98312e+000
3	2	8.41416e+000	10	1	-1.29923e+001
4	0	1.09779e+000	10	2	1.19931e+001
4	1	-1.90647e+000	11	0	-3.38792e+000
4	2	6.70313e+000	11	1	9.36571e+000
4	3	-3.89203e+000	12	0	-1.42608e+000
4	4	7.81155e+000	12	1	7.39925e+000
4	5	-3.65883e+000	13	0	-1.44353e+000
5	0	8.28761e-001	13	1	7.41670e+000
5	1	-2.34552e+000			

3.5.3 Gravity inside the Earth

The gravity inside the Earth at a distance r to the Earth center is computed from the mean density with the formula

$$g = G \frac{m}{r^2} = \frac{4}{3} \pi G r D \quad (3.5)$$

At collocation points, the values of g are computed from the values of D according to this formula. The functions are in the file `model_gravity_chebcoll.cpp`. A global variable

```
real *Gravity_Coll; //gravity at collocation points
```

for the values of g at collocation points is declared in this file. The function `model_gravity_chebcoll()` allocate this array and compute it. And the function `delete_gravity_chebcoll` delete it.

The function `model_meanrho_chebcoll()` must be executed before calling the function `model_gravity_chebcoll()`.

The main function is in the file `gravity.cpp`. The control information is in the file `gravity.ct1`. The result is written to the file `gravity.out`.

For the PREM in polynomial form the result is listed below.

Layer	Degree	Coefficient (m/s**2)	Layer	Degree	Coefficient (m/s**2)
1	0	7.38992e-016	5	0	9.97230e+000
1	1	2.32976e+001	5	1	-9.18606e-001
1	2	1.54543e-012	5	2	1.07912e+000
1	3	-9.43914e+000	6	0	1.85348e+001
2	0	3.30067e+000	6	1	-1.79801e+001
2	1	-3.14127e+001	6	2	9.45239e+000
2	2	4.22555e+002	7	0	3.83587e+000
2	3	-2.00773e+003	7	1	1.44735e+001
2	4	6.05357e+003	7	2	-8.46109e+000
2	5	-1.18609e+004	8	0	1.49728e+001
2	6	1.45805e+004	8	1	-8.33226e+000
2	7	-1.02460e+004	8	2	3.19337e+000
2	8	3.14272e+003	9	0	2.59231e+001
3	0	4.63618e+001	9	1	-3.06089e+001
3	1	-1.52990e+002	9	2	1.45191e+001
3	2	2.18605e+002	10	0	2.51233e+001
3	3	-1.06373e+002	10	1	-2.89846e+001
4	0	8.07400e+001	10	2	1.36945e+001
4	1	-4.41043e+002	11	0	1.40708e+001
4	2	1.18964e+003	11	1	-4.24758e+000
4	3	-1.78158e+003	12	0	1.73022e+001
4	4	1.55354e+003	12	1	-7.48663e+000
4	5	-7.40439e+002	13	0	1.72614e+001
4	6	1.49272e+002	13	1	-7.44586e+000

3.5.4 Pressure inside the Earth

The Pressure inside the Earth at a distance r to the Earth center is given by the integral formula

$$p(r) = - \int_R^r \rho(x)g(x)dx \quad (3.6)$$

It satisfies the differential equation

$$\frac{dp}{dr} = -\rho g \quad (3.7)$$

In our program the values of p at collocation points are computed by solving the differential equation. As p is known to be zero at the Earth surface and continuous inside the Earth, the solution is done layer by layer from the surface to the center. As boundary condition, the value of p at the outmost collocation point is imposed to be zero in the outmost layer, and in the other layers the value of p at the outmost collocation point is imposed to be equal to its value at the innermost collocation point of its adjacent outer layer.

The functions are in the file `model_pressue_chebcoll.cpp`. A global variable

```
real *Pressure_Coll; //pressure at collocation points
```

for the values of p at collocation points is declared in this file. The function `model_pressure_chebcoll()` allocate this array and compute it. And the function `delete_pressure_chebcoll` delete it.

The function `model_meanrho_chebcoll()` and `model_gravity_chebcoll()` must be executed before calling the function `model_pressure_chebcoll()`.

The main function is in the file `pressure.cpp`. The control information is in the file `pressure.ct1`. The result is written to the file `pressure.out`.

For the PREM in polynomial form the result is listed below.

Layer	Degree	Coefficient (10**11Pa)	Layer	Degree	Coefficient (10**11Pa)
1	0	3.63855e+000	6	1	-3.61146e+000
1	1	5.91672e-005	6	2	5.94787e-001
1	2	-9.71590e+000	7	0	4.32315e+000
1	3	3.00452e-002	7	1	-6.13586e+000
1	4	5.11301e+000	7	2	1.30790e+000
2	0	3.72389e+000	7	3	5.00676e-001
2	1	-1.57825e+000	8	0	3.88100e+000
2	2	5.12555e-001	8	1	-6.11465e+000
2	3	-3.38173e+001	8	2	2.65048e+000
2	4	8.25161e+001	8	3	-4.18887e-001
2	5	-1.11789e+002	9	0	3.07542e+000
2	6	9.05386e+001	9	1	-5.04035e+000
2	7	-3.17412e+001	9	2	2.96786e+000
3	0	5.12698e+000	9	3	-1.00500e+000
3	1	-1.16278e+001	10	0	3.02823e+000
3	2	1.16089e+001	10	1	-4.89656e+000
3	3	-5.41033e+000	10	2	2.82182e+000
4	0	6.86084e+000	10	3	-9.55558e-001
4	1	-2.59400e+001	11	0	2.20640e+000
4	2	5.89922e+001	11	1	-2.59970e+000
4	3	-8.53401e+001	11	2	3.92389e-001
4	4	7.20891e+001	12	0	1.97248e+000
4	5	-3.33255e+001	12	1	-2.51704e+000
4	6	6.62165e+000	12	2	5.44561e-001
5	0	3.63301e+000	13	0	1.96952e+000
5	1	-4.79219e+000	13	1	-2.51112e+000
5	2	1.11591e+000	13	2	5.41595e-001
6	0	2.99373e+000			

3.5.5 Mean moment of inertia of the mass inside any internal stratum of the Earth: $I = (A + B + C)/3$

The mean moment of inertia $I = \frac{A + B + C}{3}$ of the mass inside any equi-density surface in the interior of the Earth is given by the integral formula

$$I(x) = \frac{8\pi}{3} \int_0^r \rho(x) x^4 dx \quad (3.8)$$

It satisfies the differential equation

$$\frac{dI}{dr} = \frac{8\pi}{3} \rho r^4 \quad (3.9)$$

It is seen that if ρ is constant $I = \frac{8\pi}{15} \rho r^5$. Hence the magnitude of I at different collocation points in the 1-st layer would be too different. This would cause large round error. To avoid this defect we introduce

$$I' = \frac{8\pi}{15} r^5 I \quad (3.10)$$

which satisfies the differential equation

$$\frac{dI'}{dr} = -\frac{5}{r} I' + \frac{5\rho}{r} \quad (3.11)$$

As boundary conditions, $I' = 0$ at the Earth center $r = 0$, and it is continuous across the boundaries of adjacent layers of the Earth model.

In our program I' is computed by solving differential equation from the 1-st to outmost layer, and then I is computed from I' .

The functions are in the file `model_meanmoment_chebcoll.cpp`. A global variable

```
real *MeanMoment_Coll; //mean moment of inertia at collocation points
```

for the values of I at collocation points is declared in this file. The function `model_meanmoment_chebcoll()` allocates this array and computes it. And the function `delete_meanmoment_chebcoll` deletes it.

The main function is in the file `meanmoment.cpp`. The control information is in the file `meanmoment.ct1`. The result is written to the file `meanmoment.out`.

For the PREM in polynomial form the result is listed below.

Layer	Degree	Coefficient (10**37kgm**2)	Layer	Degree	Coefficient (10**37kgm**2)
1	0	-1.17102e-019	5	1	-5.30024e+001
1	1	1.35525e-019	5	2	4.34146e+001
1	2	3.13053e-015	6	0	2.27388e+001
1	3	-1.91798e-013	6	1	-6.13381e+001
1	4	4.17189e-012	6	2	4.68482e+001
1	5	2.30185e+001	7	0	1.95291e+001
1	6	1.92562e-010	7	1	-6.75399e+001

1	7	-1.11024e+001	7	2	7.22744e+001
2	0	1.52862e-003	7	3	-1.61926e+001
2	1	-3.10658e-002	8	0	3.48813e+000
2	2	3.29258e-001	8	1	-2.81304e+000
2	3	-1.95463e+000	8	2	-9.75822e+000
2	4	7.11160e+000	8	3	1.71314e+001
2	5	5.87873e+000	9	0	-2.32958e+001
2	6	2.09292e+001	9	1	9.69603e+001
2	7	-2.25121e+001	9	2	-1.29636e+002
3	0	-9.77434e-001	9	3	6.40205e+001
3	1	9.69143e+000	10	0	3.92115e+001
3	2	-2.44598e+001	10	1	-9.20642e+001
3	3	2.38938e+001	10	2	6.09016e+001
4	0	-5.79269e-001	11	0	3.30636e+001
4	1	9.45680e+000	11	1	-7.55607e+001
4	2	-3.80627e+001	11	2	5.05301e+001
4	3	8.28053e+001	12	0	2.79294e+001
4	4	-1.02754e+002	12	1	-5.98970e+001
4	5	8.17961e+001	12	2	3.99878e+001
4	6	-2.41903e+001	13	0	-1.20398e+001
5	0	1.80291e+001	13	1	2.00600e+001

3.5.6 Difference of mean density and density

In computing the internal flattening by solving Clairaut's equation the value of $D - \rho$ is needed. Although it can be computed directly from D and ρ . But this would not have the maximum accuracy. From the differential equation satisfied by D it can be easily obtained that $D - \rho$ satisfies the differential equation

$$\frac{d(D - \rho)}{dr} = -\frac{3}{r}(D - \rho) - \frac{d\rho}{dr} \quad (3.12)$$

In our program $D - \rho$ is computed by solving this differential equation.

The functions are in the file `model_pressue_chebcoll.cpp`. A global variable

```
real *DiffRho_Coll; //pressure at collocation points
```

for the values of $D - \rho$ at collocation points is declared in this file. The function `model_meanrhominusrho_chebcoll()` allocates this array and computes it. And the function `delete_meanrhominusrho_chebcoll` deletes it.

It is worth while noticing that the values of $\frac{d\rho}{dr}$ at collocation points are computed according to (2.3) and (1.54) in the program.

The main function is in the file `meanrhominusrho.cpp`. The control information is in the file `meanrhominusrho.ct1`. The result is written to the file `meanrhominusrho.out`.

The result will be used to compute internal flattening of the Earth.

3.5.7 Earth's internal flattening

The differential equation satisfied by the flattening of the Earth's internal equi-density surfaces is derived out under the assumption that the Earth is in hydrostatic equilibrium state. It is

$$\frac{d^2 f}{dr^2} + \frac{6}{r} \frac{\rho}{D} \frac{df}{dr} - \frac{6}{r^2} \left(1 - \frac{\rho}{D}\right) f = 0 \quad (3.13)$$

It accepts one boundary condition

$$R \left(\frac{df}{dr} \right)_{r=R} + 2f_{r=R} = \frac{5}{2}m \text{ or } f_{r=R} = f_R \quad (3.14)$$

where ω is the angular rate of Earth's rotation, M is the Earth's mass, $m = \frac{\omega^2 R^3}{GM}$, and f_R is the Earth's flattening. The first one is used when computing the hydrostatic equilibrium flattening. But in certain occasions the second one is used.

Let

$$f' = r \frac{df}{dr} \quad (3.15)$$

Then the second order differential equation (3.13) becomes a system of two differential equations:

$$\begin{cases} \frac{df}{dr} = \frac{1}{r} f' \\ \frac{df'}{dr} = \frac{6}{r} \frac{D - \rho}{D} f + \frac{1}{r} \left(1 - 6 \frac{\rho}{D}\right) f' \end{cases} \quad (3.16)$$

And the boundary condition (3.14) transforms to

$$f'_R + 2f_R = \frac{5}{2}m \text{ or } f_{r=R} = f_R \quad (3.17)$$

The flattening f and $r \frac{df}{dr}$ are computed by the function

```
void model_flattening_chebcoll(real type);
```

for several situations:

1. Use Earth's angular velocity: Impose Hydrostatic State, **type=1**.
2. Use the Observed value of Earth surface flattening 1/298.257, **type=2**.

The variable **type** is generated by the function

```
real model_flattening_type();
```

which asks to input the choice.

The differential equations are solved layer by layer from the center to the surface of the Earth. In 1-st layer an arbitrary value of f is imposed at the outer boundary as boundary condition. Both f and $r \frac{df}{dr}$ are assumed to be continuous across all boundaries of adjacent

layers, and the computed values of f and $r \frac{df}{dr}$ are forced to satisfy the boundary condition at Earth's surface by multiplying a factor. This is easy to understand as the differential equations are linear and homogeneous, any solution of it multiplied by a constant is still a solution of it.

The functions are in the file `model_flattening_chebcoll.cpp`. In this file the following global variables are declared:

```
real *Flatten_Coll; //flattening f at collocation points
real *rdfdr_Coll; //r(df/dr) at collocation points
```

They are allocated in the function `model_flattening_chebcoll(real type)` and can be removed with the function `delete_flattening_chebcoll()`.

The main function is in the file `flattening.cpp`. The control information is in the file `flattening.ct1`. The result is written to the file `flattening.out`.

For the PREM in polynomial form the result of hydrostatic flattening is listed below.

Polynomial model of the flattening f :

Layer	Number	coefficients	Layer	Number	coefficients
1	0	2.41105e-003	4	6	5.08115e-001
1	1	1.49635e-007	4	7	-8.67005e-002
1	2	2.75222e-004	5	0	9.70442e-004
1	3	3.20077e-005	5	1	3.03135e-003
2	0	1.69828e-002	5	2	-6.79137e-004
2	1	-4.26760e-001	6	0	1.79547e-003
2	2	5.65725e+000	6	1	1.18704e-003
2	3	-4.49004e+001	6	2	3.51575e-004
2	4	2.37153e+002	7	0	1.80981e-003
2	5	-8.74403e+002	7	1	1.15673e-003
2	6	2.29421e+003	7	2	3.67562e-004
2	7	-4.27969e+003	8	0	2.22865e-003
2	8	5.55883e+003	8	1	2.60643e-004
2	9	-4.78561e+003	8	2	8.46834e-004
2	10	2.45678e+003	9	0	2.12710e-003
2	11	-5.69630e+002	9	1	4.70379e-004
3	0	1.23113e-002	9	2	7.38537e-004
3	1	-4.82156e-002	10	0	1.85314e-003
3	2	7.76115e-002	10	1	1.02674e-003
3	3	-4.03957e-002	10	2	4.56073e-004
4	0	3.68055e-002	11	0	1.39823e-003
4	1	-2.65616e-001	11	1	1.93773e-003
4	2	8.79524e-001	12	0	1.38913e-003
4	3	-1.63537e+000	12	1	1.94684e-003
4	4	1.86028e+000	13	0	1.38100e-003
4	5	-1.29371e+000	13	1	1.95498e-003

Polynomial model of $r(df/dr)$:

Layer	Number	coefficients	Layer	Number	coefficients
-------	--------	--------------	-------	--------	--------------

1	0	-4.58661e-012	4	7	-5.88995e-002
1	1	1.71326e-009	5	0	-9.59726e-003
1	2	5.58095e-004	5	1	3.57284e-002
1	3	2.28056e-006	5	2	-3.84880e-002
1	4	3.04075e-004	5	3	1.40535e-002
1	5	1.02117e-004	6	0	-9.89780e-003
2	0	-1.87064e-001	6	1	2.31742e-002
2	1	6.60912e+000	6	2	-1.15074e-002
2	2	-1.10109e+002	7	0	1.79161e-004
2	3	1.14159e+003	7	1	5.37756e-003
2	4	-8.19593e+003	7	2	-9.05919e-003
2	5	4.29696e+004	7	3	5.42871e-003
2	6	-1.69256e+005	8	0	-2.22649e-002
2	7	5.07947e+005	8	1	6.32442e-002
2	8	-1.16550e+006	8	2	-5.69106e-002
2	9	2.03269e+006	8	3	1.78699e-002
2	10	-2.65043e+006	9	0	-4.04720e-002
2	11	2.50397e+006	9	1	1.06390e-001
2	12	-1.61955e+006	9	2	-8.81245e-002
2	13	6.41711e+005	9	3	2.41443e-002
2	14	-1.17491e+005	10	0	-1.69294e-002
3	0	-3.92994e-002	10	1	3.51653e-002
3	1	1.63109e-001	10	2	-1.62981e-002
3	2	-2.23535e-001	11	0	-2.09807e-002
3	3	1.05079e-001	11	1	4.05553e-002
4	0	-5.28155e-002	11	2	-1.76263e-002
4	1	2.56018e-001	12	0	-2.59967e-002
4	2	-4.41497e-001	12	1	4.70836e-002
4	3	2.33882e-001	12	2	-1.91303e-002
4	4	2.61652e-001	13	0	-6.87543e-003
4	5	-4.71671e-001	13	1	8.83203e-003
4	6	2.75024e-001			

3.5.8 Difference of moments of inertia of the mass inside any internal stratum of the Earth: $C - A$

$C - A$ is given by the integral formula

$$C - A = \frac{8\pi}{15} \int_0^r \rho(x) \frac{d}{dx} [x^5 f(x)] dx = \frac{8\pi}{15} \int_0^r \rho(x) \left[5x^4 f(x) + x^5 \frac{df(x)}{dx} \right] dx \quad (3.18)$$

It satisfies the differential equation

$$\frac{d(C - A)}{dr} = \frac{8\pi}{15} \rho \left[5r^4 f + r^5 \frac{df}{dr} \right] \quad (3.19)$$

The same as the case of I , for reducing the round off error, we introduce

$$C - A = \frac{8\pi}{15} r^5 (C' - A') \quad (3.20)$$

It can be obtained from (3.19) that $C' - A'$ satisfies the differential equation

$$\frac{d(C' - A')}{dr} = -\frac{5}{r}(C' - A') + \frac{\rho[5f + r(df/dr)]}{r}, \quad (3.21)$$

As boundary condition, $C' - A'$ vanishes at the Earth center and is continuous across the boundaries of adjacent layers of the Earth model.

In our program $C' - A'$ is computed by solving its differential equation. $C - A$ is computed from $C' - A'$. The functions are in the file `model_momentdifference_chebcoll.cpp`. In this file the following global variable is declared:

```
real *DiffMoment_Coll; // value of C-A at collocation points
```

It is allocated and computed in the function `model_momentdifference_chebcoll(real type)` and can be removed with the function `delete_momentdifference_chebcoll()`.

The main function is in the file `momentdifference.cpp`. The control information is in the file `momentdiffernece.ctl`. The result is output in the file `momentdifference.out`.

For the PREM in polynomial form the result for the hydrostatic flattening is listed below.

Layer	Degree	Coefficient (10**35kgm**2)	Layer	Degree	Coefficient (10**35kgm**2)
1	0	7.11518e-012	5	1	8.70116e+000
1	1	-4.67103e-009	5	2	-1.77905e+001
1	2	5.00532e-007	5	3	1.31938e+001
1	3	-2.02723e-005	6	0	9.84461e+000
1	4	3.97947e-004	6	1	-2.61158e+001
1	5	5.54573e+000	6	2	1.89484e+001
1	6	2.25949e-002	7	0	4.59420e+000
1	7	-2.08848e+000	7	1	-1.45246e+001
2	0	6.11789e-004	7	2	1.25510e+001
2	1	-1.25187e-002	8	0	-1.64274e+000
2	2	1.29705e-001	8	1	1.02771e+001
2	3	-7.64747e-001	8	2	-1.86078e+001
2	4	2.68871e+000	8	3	1.25865e+001
2	5	-4.09037e-001	9	0	-1.25528e+001
2	6	7.21653e+000	9	1	5.02269e+001
2	7	-5.97436e+000	9	2	-6.60323e+001
3	0	-5.99077e-001	9	3	3.09716e+001
3	1	4.74399e+000	10	0	1.76853e+001
3	2	-1.10791e+001	10	1	-4.12159e+001
3	3	9.48415e+000	10	2	2.61439e+001
4	0	-3.12925e-001	11	0	1.56894e+001

4	1	3.64199e+000	11	1	-3.56638e+001
4	2	-1.32471e+001	11	2	2.25819e+001
4	3	2.73094e+001	12	0	1.37184e+001
4	4	-3.49340e+001	12	1	-2.97152e+001
4	5	2.84739e+001	12	2	1.85995e+001
4	6	-8.17410e+000	13	0	-4.87248e+000
5	0	-1.34651e+000	13	1	7.47520e+000

3.6 Earth Tides: Computation of Love numbers for the SNREI Earth

By Love numbers, we mean the Love numbers describing the response of the Earth to tidal forces as well as and the loading Love numbers describing the response of the Earth to surface loads. In this problem the ocean layer of the Earth model adopted is normally averaged with the upmost crust layer, conserving the mass of the Earth and let the elastic moduli of the averaged layer be the same as the upmost crust layer of the original Earth model. This is done by invoking the function

```
void model_average_oceanless();
```

at once the Earth model data are input.

3.6.1 Principle for computing Love numbers of static response

For computing both the Love numbers and the loading Love numbers, The following system of ordinary differential equations must be solved:

$$\begin{cases}
\frac{dy_1}{dr} = -\frac{2\lambda}{(\lambda+2\mu)r}y_1 + \frac{1}{\lambda+2\mu}y_2 + \frac{n(n+1)\lambda}{(\lambda+2\mu)r}y_3 \\
\frac{dy_2}{dr} = \left\{ -\frac{4\rho g}{r} + \frac{4\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_1 - \frac{4\mu}{(\lambda+2\mu)r}y_2 + \\
\left\{ \frac{n(n+1)\rho g}{r} - \frac{2n(n+1)\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_3 + \frac{n(n+1)}{r}y_4 - \rho y_6 \\
\frac{dy_3}{dr} = -\frac{1}{r}y_1 + \frac{1}{r}y_3 + \frac{1}{\mu}y_4 \\
\frac{dy_4}{dr} = \left\{ \frac{\rho g}{r} - \frac{2\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_1 - \frac{\lambda}{(\lambda+2\mu)r}y_2 + \\
\left\{ \frac{2\mu}{(\lambda+2\mu)r^2} [\lambda(2n^2+2n-1) + 2\mu(n^2+n-1)] \right\} y_3 - \\
\frac{3}{r}y_4 - \frac{\rho}{r}y_5 \\
\frac{dy_5}{dr} = 4\pi G\rho y_1 + y_6 \\
\frac{dy_6}{dr} = -\frac{4\pi G n(n+1)\rho}{r}y_3 + \frac{n(n+1)}{r^2}y_5 - \frac{2}{r}y_6
\end{cases} \quad (3.22)$$

For both the Love numbers and the loading Love numbers, the regularity boundary conditions at the Earth center and the boundary conditions across the internal boundaries of discontinuity are the same. Here the regularity boundary conditions at the Earth center will be treated implicitly with the help of Chebyshev-Gauss-Radau integration points. The boundary conditions across the internal boundaries of discontinuity are: $y_1 \sim y_6$ are all continuous across solid-solid boundaries, and across solid-liquid boundaries no condition exists for y_3 and $y_1 \sim y_6$ except y_3 are continuous.

For the Love numbers the boundary conditions at the Earth surface are

$$(y_2)_{r=R} = 0, (y_4)_{r=R} = 0, (y_6)_{r=R} = -\frac{n+1}{R}(y_5)_{r=R} + (2n+1)g_R \quad (3.23)$$

where g_R is the value of gravity at the Earth surface. The Love numbers are defined by

$$h_n = \frac{y_1(R)}{R}, l_n = \frac{y_3(R)}{R}, k_n = \frac{y_5(R)}{Rg_R} - 1, n \geq 2 \quad (3.24)$$

where $y_1(R)$, $y_3(R)$ and $y_5(R)$ are the values of y_1 , y_3 and y_5 at the Earth surface.

For the loading Love numbers the boundary conditions at the Earth surface are

$$(y_2)_{r=R} = -\frac{(2n+1)g_R^2}{4\pi G}, (y_4)_{r=R} = 0, \text{ for } n \geq 0 \quad (3.25)$$

and

$$(y_5)_{r=R} = Rg_R, \text{ for } n = 1 \quad (3.26)$$

or

$$(y_6)_{r=R} = -\frac{n+1}{R}(y_5)_{r=R} + (2n+1)g_R, \text{ for } n \neq 1 \quad (3.27)$$

The Love numbers are defined by

$$h'_n = \frac{y_1(R)}{R}, l'_n = \frac{y_3(R)}{R}, k'_n = \frac{y_5(R)}{Rg_R} - 1, n \geq 0 \quad (3.28)$$

In the liquid core $\mu = 0$. (3.22) reduces to 4 differential equations and 2 algebraic equations. In Earth models the Williamson-Adams condition

$$\frac{1}{\rho} \frac{d\rho}{dr} + \frac{\rho g}{\lambda} = 0 \quad (3.29)$$

is normally satisfied in the liquid core. This makes the differential-algebraic equations independent. So further development is needed.

Set $\mu = 0$ in (3.22). The 3-rd equation reduces to $y_4 = 0$, and the others become

$$\frac{dy_1}{dr} = -\frac{2}{r}y_1 + \frac{1}{\lambda}y_2 + \frac{n(n+1)}{r}y_3 \quad (3.30)$$

$$\frac{dy_2}{dr} = -\frac{4\rho g}{r}y_1 + \frac{n(n+1)\rho g}{r}y_3 - \rho y_6 \quad (3.31)$$

$$\frac{\rho g}{r}y_1 - \frac{1}{r}y_2 - \frac{\rho}{r}y_5 = 0 \quad (3.32)$$

$$\frac{dy_5}{dr} = 4\pi G\rho y_1 + y_6 \quad (3.33)$$

$$\frac{dy_6}{dr} = -\frac{4\pi G n(n+1)\rho}{r}y_3 + \frac{n(n+1)}{r^2}y_5 - \frac{2}{r}y_6 \quad (3.34)$$

Multiply (3.30) with ρg and then subtract (3.31) from it, one obtains

$$\rho g \frac{dy_1}{dr} - \frac{dy_2}{dr} = \frac{2\rho g}{r}y_1 + \frac{\rho g}{\lambda}y_2 + \rho y_6 \quad (3.35)$$

Multiply (3.32) with r , and then differentiate with respect to r , one obtains

$$\rho g \frac{dy_1}{dr} - \frac{dy_2}{dr} = -\frac{d\rho g}{dr}y_1 + \frac{d\rho}{dr}y_5 + \rho \frac{dy_5}{dr} \quad (3.36)$$

From (3.35) and (3.36) one can obtain

$$\frac{2\rho g}{r}y_1 + \frac{\rho g}{\lambda}y_2 + \rho y_6 = -\frac{d\rho g}{dr}y_1 + \frac{d\rho}{dr}y_5 + \rho \frac{dy_5}{dr} \quad (3.37)$$

From the formula

$$g(r) = 4\pi G \frac{1}{r^2} \int_0^r \rho(x)x^2 dx \quad (3.38)$$

one can obtain

$$\frac{dg}{dr} = 4\pi G\rho - \frac{2g}{r} \quad (3.39)$$

Eliminate y_6 and $\frac{dg}{dr}$ from (3.33), (3.37) and (3.39), one obtains

$$\frac{\rho g}{\lambda}y_2 = -\frac{d\rho}{dr}gy_1 + \frac{d\rho}{dr}y_5 \quad (3.40)$$

Substitute (3.32) into it, one obtains

$$\left(\frac{1}{\rho} \frac{d\rho}{dr} + \frac{\rho g}{\lambda} \right) (gy_1 - y_5) = 0 \quad (3.41)$$

This equation is automatically satisfied if the Williamson-Adams condition is satisfied.

The equation (3.41) means that the equations (3.30) to (3.33) are not independent. So from now on the equation (3.31) is dropped. Introduce $y_7 = \frac{dy_5}{dr}$ to replace y_6 , (3.33) and (3.34) become

$$\frac{dy_5}{dr} = y_7 \quad (3.42)$$

$$\begin{aligned} \frac{dy_7}{dr} = & 4\pi G\rho \frac{dy_1}{dr} + 4\pi g \frac{d\rho}{dr} y_1 - \frac{4\pi g\rho n(n+1)}{r} y_3 + \\ & \frac{n(n+1)}{r^2} y_5 - \frac{2}{r} y_7 + \frac{8\pi G\rho}{r} y_1 \end{aligned} \quad (3.43)$$

Multiply (3.30) with $4\pi G\rho$ and then add to (3.43), one obtains

$$\frac{dy_7}{dr} = 4\pi g \frac{d\rho}{dr} y_1 + \frac{n(n+1)}{r^2} y_5 - \frac{2}{r} y_7 + \frac{4\pi G\rho}{\lambda} y_2 \quad (3.44)$$

Eliminate y_2 between (3.32) and (3.44), one obtains

$$\begin{aligned} \frac{dy_7}{dr} = & 4\pi g \frac{d\rho}{dr} y_1 + \frac{n(n+1)}{r^2} y_5 - \frac{2}{r} y_7 + \frac{4\pi G\rho^2 g}{\lambda} y_1 - \frac{4\pi G\rho^2}{\lambda} y_5 \\ = & 4\pi G\rho \left(\frac{1}{\rho} \frac{d\rho}{dr} + \frac{\rho g}{\lambda} \right) + \left[\frac{n(n+1)}{r^2} - \frac{4\pi G\rho^2}{\lambda} \right] y_5 - \frac{2}{r} y_7 \\ = & \left[\frac{n(n+1)}{r^2} - \frac{4\pi G\rho^2}{\lambda} \right] y_5 - \frac{2}{r} y_7 \end{aligned} \quad (3.45)$$

The differential-algebraic equations to be solved in the liquid core are (3.42), (3.45) and (3.32). Here we write them together

$$\begin{cases} \frac{dy_5}{dr} = y_7 \\ \frac{dy_7}{dr} = \left[\frac{n(n+1)}{r^2} - \frac{4\pi G\rho^2}{\lambda} \right] y_5 - \frac{2}{r} y_7 \\ \rho g y_1 - y_2 - \rho y_5 = 0 \end{cases} \quad (3.46)$$

In practice y_5 and y_7 are solved from the first 2 differential equations. From the last algebraic equation only $\rho g y_1 - y_2$ can be obtained rather than y_1 and y_2 separately. At the inner core surface and the Core-Mantle-Boundary, y_6 and y_7 are related as $y_7 = y_6 + 4\pi G\rho y_1$ according to (3.33). This is the nature of the problem.

In our program the computations are done in the following way. As 3 regularity conditions are imposed at the Earth center, 3 sets of independent boundary conditions are imposed at the inner core surface to obtain 3 sets independent solutions of the differential equations. At the inner core surface the boundary conditions are

$$y_4 = 0, \quad \rho g y_1 - y_2 - \rho y_5 = 0, \quad y_5 \text{ and } y_6 \text{ continuous} \quad (3.47)$$

From the 3 sets of independent solutions in the inner core only 1 set of solutions satisfying the boundary conditions at the inner core surface is obtained. And the differential equations in the liquid core are solved for this set of boundary conditions to obtain only 1 set

of solutions. At the Core-Mantle-Boundary, 3 independent sets of boundary conditions satisfying (3.47) are again constructed from this solution:

$$\begin{cases} y_1 = \frac{y_5}{g}, y_2 = 0, y_3 = y_4 = 0, y_5 \text{ and } y_6 \text{ continuous} \\ y_1 = 0, y_2 = -\rho y_5, y_3 = y_4 = 0, y_5 \text{ and } y_6 \text{ continuous} \\ y_1 = 0, y_2 = 0, y_3 = 1, y_4 = y_5 = y_6 = 0 \end{cases} \quad (3.48)$$

This 3 sets of boundary conditions are used to compute 3 sets of independent solutions in the mantle where $y_1 \sim y_6$ are all continuous. At the surface of the Earth, the 3 boundary conditions (3.25) to (3.27) are finally used to obtain one solution satisfying them. The Love numbers are then computed from this solution. The loading Love numbers are different from the Love numbers only in the last step.

The case when $n = 0$ should be treated particularly. In this case $y_3 = y_4 = 0$ and $y_6 = 0$ for any values of r . And y_1, y_2, y_5 are all continuous across any boundaries of discontinuity. So 2 sets of independent solutions are computed from the center to the surface of the Earth. At the Earth's surface, 2 boundary conditions are used to obtain 1 solution satisfying them. This solution is used to compute the Love numbers or loading Love numbers.

3.6.2 Principle for computing Love numbers of dynamic response

The difference of the dynamic response from the static response is the consideration of the inertial term in the equations of motion. The differential equations to be solved are

$$\begin{cases} \frac{dy_1}{dr} = -\frac{2\lambda}{(\lambda+2\mu)r}y_1 + \frac{1}{\lambda+2\mu}y_2 + \frac{n(n+1)\lambda}{(\lambda+2\mu)r}y_3 \\ \frac{dy_2}{dr} = \left\{ -\rho\sigma^2 - \frac{4\rho g}{r} + \frac{4\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_1 - \frac{4\mu}{(\lambda+2\mu)r}y_2 + \\ \left\{ \frac{n(n+1)\rho g}{r} - \frac{2n(n+1)\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_3 + \frac{n(n+1)}{r}y_4 - \rho y_6 \\ \frac{dy_3}{dr} = -\frac{1}{r}y_1 + \frac{1}{r}y_3 + \frac{1}{\mu}y_4 \\ \frac{dy_4}{dr} = \left\{ \frac{\rho g}{r} - \frac{2\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_1 - \frac{\lambda}{(\lambda+2\mu)r}y_2 + \\ \left\{ -\rho\sigma^2 + \frac{2\mu}{(\lambda+2\mu)r^2}[\lambda(2n^2+2n-1) + 2\mu(n^2+n-1)] \right\} y_3 - \\ \frac{3}{r}y_4 - \frac{\rho}{r}y_5 \\ \frac{dy_5}{dr} = 4\pi G\rho y_1 + y_6 \\ \frac{dy_6}{dr} = -\frac{4\pi G n(n+1)\rho}{r}y_3 + \frac{n(n+1)}{r^2}y_5 - \frac{2}{r}y_6 \end{cases} \quad (3.49)$$

where σ is the angular frequency of the tidal wave. The boundary conditions and definitions of the Love numbers and loading Love numbers are the same as in the case of static response.

In the liquid core $\mu = 0$. This makes $y_4 = 0$, and the differential equations (3.49) reduce to the following differential-algebraic equations

$$\left\{ \begin{array}{l} \frac{dy_1}{dr} = -\frac{2}{r}y_1 + \frac{1}{\lambda}y_2 + \frac{n(n+1)}{r}y_3 \\ \frac{dy_2}{dr} = \left[-\rho\sigma^2 - \frac{4\rho g}{r} \right] y_1 + \frac{n(n+1)\rho g}{r}y_3 - \rho y_6 \\ \frac{\rho g}{r}y_1 - \frac{1}{r}y_2 - \rho\sigma^2 y_3 - \frac{\rho}{r}y_5 = 0 \\ \frac{dy_5}{dr} = 4\pi G\rho y_1 + y_6 \\ \frac{dy_6}{dr} = -\frac{4\pi G n(n+1)\rho}{r}y_3 + \frac{n(n+1)}{r^2}y_5 - \frac{2}{r}y_6 \end{array} \right. \quad (3.50)$$

The satisfaction of the Williamson-Adams condition does not cause the problem of independence between these equations.

At solid-liquid boundaries, the boundary conditions are

$$y_4 = 0, \quad \frac{\rho g}{r}y_1 - \frac{1}{r}y_2 - \rho\sigma^2 y_3 - \frac{\rho}{r}y_5 = 0, \quad y_1, y_2, y_4, y_5, y_6 \text{ continuous} \quad (3.51)$$

The problem is solved as follows. 3 sets of independent solutions are computed in the inner core as for the case of static response. From these solutions and the first condition in (3.51) 2 independent sets of boundary conditions are obtained at the inner core surface. As boundary condition for solving the problem in the outer core, y_3 for both sets of boundary conditions are computed from the second formula in (3.51), and y_1, y_2, y_4, y_5, y_6 are continuous. With these boundary conditions the differential-algebraic equations (3.50) are solved. At the Core-Mantle-Boundary, as y_1, y_2, y_4, y_5, y_6 are continuous but not y_3 , 3 sets of independent boundary conditions are obtained by setting $y_3 = 0$ in both these sets of solutions and adding another one: $y_1 = 0, y_2 = 0, y_3 = 1, y_4 = 0, y_5 = 0, y_6 = 0$. These 3 sets of boundary conditions are then used to compute 3 sets of solutions in the mantle. The next things to be done is the same as for the static response.

3.6.3 Programming

For the ease of programming, the original differential equations are modified such that in the liquid core they automatically become the required differential-algebraic equations. Take the case of dynamic response as an example. The differential equations are modified

to

$$\left\{ \begin{array}{l} r \frac{dy_1}{dr} = -\frac{2\lambda}{\lambda+2\mu}y_1 + \frac{r}{\lambda+2\mu}y_2 + \frac{n(n+1)\lambda}{\lambda+2\mu}y_3 \\ r^2 \frac{dy_2}{dr} = \left\{ -\rho r^2 \sigma^2 - 4\rho gr + \frac{4\mu(3\lambda+2\mu)}{\lambda+2\mu} \right\} y_1 - \frac{4\mu r}{\lambda+2\mu}y_2 + \\ \left\{ n(n+1)\rho gr - \frac{2n(n+1)\mu(3\lambda+2\mu)}{\lambda+2\mu} \right\} y_3 + n(n+1)ry_4 - \rho r^2 y_6 \\ \mu r \frac{dy_3}{dr} = -\mu y_1 + \mu y_3 + ry_4 \\ r^2 \frac{dy_4}{dr} = \left\{ \rho gr - \frac{2\mu(3\lambda+2\mu)}{\lambda+2\mu} \right\} y_1 - \frac{\lambda r}{\lambda+2\mu}y_2 + \\ \left\{ -\rho r^2 \sigma^2 + \frac{2\mu}{\lambda+2\mu}[\lambda(2n^2+2n-1) + 2\mu(n^2+n-1)] \right\} y_3 - \\ 3ry_4 - \rho ry_5 \\ \frac{dy_5}{dr} = 4\pi G \rho y_1 + y_6 \\ r^2 \frac{dy_6}{dr} = -4\pi G n(n+1)\rho ry_3 + n(n+1)y_5 - 2ry_6 \end{array} \right. \quad (3.52)$$

When $n = 0$ the 3-rd and 4-th equations are replaced by $\frac{dy_3}{dr} = 0$ and $\frac{dy_4}{dr} = 0$. In this case the boundary conditions for y_3 and y_4 are: $y_3 = 0$ and $y_4 = 0$ at inner core surface, and continuous elsewhere.

The subroutines for computing the Love numbers and loading Love numbers are the functions

```
void tides_static_ln_snrei_odes_chebcoll(int n,real &hh,real &ll,
                                         real &kk,int type);
void tides_dynamic_ln_snrei_odes_chebcoll(int n,real &hh,real &ll,
                                         real &kk,real period,int type);
```

The Love numbers are computed when `type=1`, and the loading Love numbers are computed when `type=2`. The parameter `period` in the second function is the period of the tidal wave in hours. Before invoking these functions the following functions must be executed first:

```
void model_meanrho_chebcoll();
void model_gravity_chebcoll();
void model_mu_chebcoll(real Period);
void model_lambda_chebcoll(real Period);
```

The first 2 functions are explained in the last section. The 3-rd function compute μ . It is in the files `model_mu_chebcoll.cpp`. In this file the global variable

```
real *Mu_Coll; //values of Mu at collocation points
```

is declared. It is allocated in the function `void model_mu_chebcoll(real Period)` and can be removed by the function `delete_mu_chebcoll()`. The last function computes λ . It is similar to the 3-rd one.

When computing μ and λ the dependency of V_p and V_s on the period of seismic wave as for the case of PREM is taken into account. Let's denote the period of the seismic wave by T . $V_s(T)$ and $V_p(T)$ are first computed using the formula

$$\begin{cases} V_s(T) = V_s(1) \left(1 - \frac{\ln T}{\pi} \frac{1}{Q_\mu} \right) \\ V_p(T) = V_p(1) \left[1 - \frac{\ln T}{\pi} \left(\frac{1-E}{Q_k} + \frac{E}{Q_\mu} \right) \right] \end{cases} \quad (3.53)$$

where $E = \frac{3}{4}[V_s(1)/V_p(1)]^2$. Then these values are used to compute μ and λ using the formulas

$$V_p = \sqrt{\frac{\lambda + 2\mu}{\rho}}, \quad V_s = \sqrt{\frac{\mu}{\rho}} \quad (3.54)$$

The main program is in the file `ln_snrei_odes.cpp`. The control information is in the file `ln_snrei_odes.ct1`. The result is written to the file `ln_snrei_odes.out`.

This program first asks if static or dynamic response is considered. If dynamic response is considered, it asks the period of the tidal wave. Then it asks what period will be used for computing λ and μ . As result, Love numbers are computed for $n = 2, \dots, 5$, and loading Love numbers are computed for $n = 0, \dots, 12000$, not for every values of n in the interval for the last case.

Some result for the case of static response are listed below.

Wave period for computing elastic parameters: 200.000000 seconds

Love Numbers:

n	h	l	k
2	0.60660265	0.08480453	0.29993951
3	0.28990582	0.01508319	0.09273446
4	0.17648465	0.01044187	0.04180095
5	0.13024647	0.00864378	0.02458287

Loading Love Numbers:

n	h'	l'	k'	n*l'	n*k'
0	-0.13216882	0.00000000	0.00000000	0.00000000	0.00000000
1	-0.28613898	0.10243511	0.00000000	0.10243511	0.00000000
2	-0.99402670	0.02198035	-0.30666314	0.04396070	-0.61332628
3	-1.05449824	0.06892343	-0.19717136	0.20677028	-0.59151409
4	-1.05838012	0.05768450	-0.13468371	0.23073801	-0.53873482
5	-1.09258669	0.04514075	-0.10566360	0.22570373	-0.52831801
6	-1.15147245	0.03755970	-0.09126951	0.22535823	-0.54761704
7	-1.22141937	0.03319522	-0.08299802	0.23236654	-0.58098616
8	-1.29440002	0.03051069	-0.07748491	0.24408549	-0.61987928
9	-1.36667525	0.02870862	-0.07337319	0.25837756	-0.66035870
10	-1.43673766	0.02741202	-0.07007455	0.27412021	-0.70074548
20	-1.99052452	0.02284128	-0.05196714	0.45682552	-1.03934285
30	-2.32777750	0.02047183	-0.04150216	0.61415498	-1.24506476
40	-2.53289713	0.01801455	-0.03392148	0.72058208	-1.35685915

50	-2.66750554	0.01581594	-0.02832411	0.79079706	-1.41620558
60	-2.76433479	0.01396532	-0.02411625	0.83791904	-1.44697520
70	-2.83990879	0.01242442	-0.02088257	0.86970945	-1.46178006
80	-2.90304100	0.01113569	-0.01834508	0.89085541	-1.46760651
90	-2.95877041	0.01004947	-0.01631743	0.90445255	-1.46856877
100	-3.01012532	0.00912686	-0.01467185	0.91268609	-1.46718531
200	-3.48230757	0.00454146	-0.00743646	0.90829103	-1.48729255
300	-3.95799727	0.00311044	-0.00536569	0.93313303	-1.60970722
400	-4.40657583	0.00254834	-0.00444411	1.01933556	-1.77764284
500	-4.80245210	0.00228208	-0.00390191	1.14104086	-1.95095453
600	-5.13531452	0.00212103	-0.00351561	1.27261761	-2.10936838
700	-5.40497885	0.00199625	-0.00320636	1.39737631	-2.24444944
800	-5.61716079	0.00188349	-0.00294311	1.50679293	-2.35448530
900	-5.78029539	0.00177548	-0.00271250	1.59793227	-2.44125429
1000	-5.90341403	0.00167112	-0.00250804	1.67112005	-2.50803722
2000	-6.22075605	0.00094182	-0.00134376	1.88363776	-2.68752420
3000	-6.23051154	0.00063025	-0.00089801	1.89074982	-2.69404033
4000	-6.23119148	0.00047273	-0.00067371	1.89090935	-2.69483519
5000	-6.23150534	0.00037818	-0.00053905	1.89092372	-2.69524299
6000	-6.23171303	0.00031516	-0.00044925	1.89093182	-2.69550641
7000	-6.23186151	0.00027013	-0.00038510	1.89093738	-2.69570562
8000	-6.23197328	0.00023637	-0.00033699	1.89094157	-2.69588596
9000	-6.23206062	0.00021010	-0.00029956	1.89094493	-2.69607571
10000	-6.23213067	0.00018909	-0.00026963	1.89094756	-2.69629602
11000	-6.23218793	0.00017190	-0.00024514	1.89094945	-2.69656581
12000	-6.23223550	0.00015758	-0.00022474	1.89095065	-2.69690331
Inf	-6.23275563			1.89097465	-2.69679519

3.7 Free Oscillation of the SNREI Earth

The free oscillation of the SNREI Earth is decomposed to toroidal and spheroidal parts. We consider them one after another.

3.7.1 Toroidal free oscillations

This is a proper value problem. The differential equations

$$\begin{cases} \frac{dy_1}{dr} = \frac{1}{r}y_1 + \frac{1}{\mu}y_2 \\ \frac{dy_2}{dr} \left[-\rho\sigma^2 + \frac{(n-1)(n+1)\mu}{r^2} \right] y_1 - \frac{3}{r}y_2 \end{cases} \quad (3.55)$$

are to be solved in the mantle. The boundary conditions are

$$\begin{cases} y_1 \text{ and } y_2 \text{ continuous everywhere} \\ (y_2)_{\text{CMB}} = 0 \end{cases} \quad (3.56)$$

and

$$(y_2)_{\text{Earth surface or ocean floor}} = 0 \quad (3.57)$$

depending on if the ocean layer of the Earth model is averaged with the upmost crust layer or not. The objective of the problem is to find values of σ which makes the differential equations (3.55) having non-zero solutions satisfying the boundary conditions (3.56) and (3.57). Such values of σ are called proper values, and the corresponding non-zero solutions of y_1 and y_2 are called proper functions.

In (3.55) σ is the angular frequency of the oscillation. It is related to the period T by

$$\sigma = 2\pi/T \quad (3.58)$$

The toroidal free oscillations exist only for $n \geq 2$. The proper value-proper function pairs are also called modes. The toroidal free oscillation modes are classified as 0-T_n, 1-T_n, ... in descending order of the period T , 0-T_n being called basic tone, and 1-T_n ... overtones.

The functions for solving this problem are in the file `toroidal_snrei_odes_chebcoll`. They are

```
static int toroidal_odes_chebcoll(real period,real &f);
static int toroidal_odes_one_mode(int n,real period_start,
    real period_end,real step,real &period,real toll);
void toroidal_snrei_odes_chebcoll(int n_low,int n_high,int number,
    real **period);
void toroidal_snrei_odes_chebcoll_eif(int n,real period,real **y_Coll);
```

The 1-st function solves the differential equations (3.55) with the boundary conditions in (3.56) and the value of period transformed to it and returns the value of y_2 at the outmost solid surface of the Earth by the parameter `f`, i.e. the ocean floor for Earth models with an ocean layer or the surface of the Earth for Earth models without an ocean layer including the case that the ocean layer is averaged with the upmost crust layer. The most sophisticated part of the program is to find the values of the period which makes the returned value vanishes, i.e. satisfies the boundary condition (3.57).

The 2-nd function finds one such value starting from `period.start` interval by interval of length `step` towards `period.end` with the accuracy `toll`. The value found is transformed to the reference variable `period`. The returned value of the function may be 0, 1, -1 or -2 of which the meanings are explained as follows:

- 0: No value searching for is found from `period.start` to `period.end`.
- 1: The possible value searching for is found, and the returned value of the 1-st function diminishes as the value of period transformed to it approaches the value found.
- -1: The possible value searching for is found, and the returned value of the 1-st function increases as the value of period transformed to it approaches the value found.

- -2: Failed solving the differential equations for the value of `period` returned.

The value fund is considered to be the solution of the problem only if 1 is returned.

The 3-rd function uses the second one to compute `number` proper values for every n between `n_low` and `n_high`. The period of `i_T_n` is given in `period[i][n]`, $i = 0, \dots, \text{number}-1$, $n = \text{n_low}, \dots, \text{n_high}$. The values of periods computed are accurate to 10^{-8} second which assures 10 significant digits when computing proper functions. Experiences told us that the period of `0_T_n` decreases as n increase. So the search is always done from `0_T_2` to `0_T_3`, `0_T_4`, $\dots$. The starting period for searching `0_T_2` is 3000 seconds, and that for `0_T_{n+1}` is the period of `0_T_n`. The next step is to search `i_T_n`, `i_T_{n+1}`, $\dots$ for $n > 0$, the starting period for searching `i_T_n` being the period of `i-1_T_n`.

The 4-th function computes the proper functions for the proper period of `period` and of degree n . The values of y_1 and y_2 at the i -th collocation points are returned to `y_Coll[1][i]` and `y_Coll[2][i]` respectively. The proper functions are normalized such that

$$n(n+1) \int_0^R \rho r^2 y_1^2 dr = 1 \quad (3.59)$$

which is often made in the perturbation formulation of free oscillation.

The main function is in the file `toroidal_snrei_odes.cpp`. It first asks to compute proper period or proper function, and then asks to input `n_low`, `n_high` and `number`. If the proper periods are computed, the periods are output to two files, one is the standard output file assigned to the file pointer `fpout`, and the other is named according to the Earth model adopted for being kept, as this computation is quite time consuming. If proper functions are computed, their values at collocation points are given in separate files for every mode. The files are named according to the mode such as `0_T_2.prem.polynomial`. The results are also given in the 3 kilometers interval formats. For example, the result of the mode `0_T_2` for the polynomial model of PREM is given in the files `0_T_2.3kms.prem.polynomial`. The first line gives the value of its period with the unit of second, and the 2 columns starting by the second line are respectively the value of y_1 and y_2 .

The result of some proper periods are

Ocean layer averaged with the upmost crust layer

Free oscillation periods of the modes `m_T_n`:

n	$m=0$	$m=1$	$m=2$	$m=3$	$m=4$	$m=5$
2	2639.6231	757.8025	448.4596	309.2867	231.1484	186.2012
3	1707.7908	695.1664	435.9343	304.9830	229.3865	185.3183
4	1307.7180	631.0090	420.3889	299.4643	227.0922	184.1573
5	1078.9436	571.5455	402.5870	292.8926	224.3083	182.7313
6	928.5907	519.5857	383.3033	285.4427	221.0835	181.0558
7	820.9001	475.5979	363.2998	277.2912	217.4702	179.1478
8	739.1413	438.8173	343.3116	268.6080	213.5215	177.0258
9	674.4499	408.0150	324.0039	259.5523	209.2897	174.7087
10	621.6755	381.9432	305.8966	250.2733	204.8235	172.2158
11	577.6021	359.5485	289.3079	240.9153	200.1677	169.5656
12	540.1050	340.0261	274.3525	231.6236	195.3614	166.7763
13	507.7169	322.7898	260.9849	222.5453	190.4392	163.8647

14	479.3885	307.4157	249.0608	213.8219	185.4320	160.8463
15	454.3474	293.5922	238.3939	205.5738	180.3706	157.7353
16	432.0118	281.0839	228.7967	197.8848	175.2897	154.5445
17	411.9344	269.7069	220.1029	190.7935	170.2328	151.2860
18	393.7650	259.3138	212.1745	184.2937	165.2530	147.9721
19	377.2255	249.7837	204.9008	178.3444	160.4090	144.6164
20	362.0917	241.0156	198.1932	172.8850	155.7560	141.2366
21	348.1806	232.9240	191.9804	167.8497	151.3358	137.8559
22	335.3413	225.4357	186.2042	163.1772	147.1713	134.5034
23	323.4478	218.4875	180.8157	158.8154	143.2657	131.2131
24	312.3943	212.0245	175.7741	154.7225	139.6076	128.0199
25	302.0907	205.9985	171.0439	150.8653	136.1774	124.9543
26	292.4601	200.3671	166.5948	147.2178	132.9525	122.0387
27	283.4363	195.0930	162.4004	143.7589	129.9114	119.2847
28	274.9617	190.1430	158.4377	140.4721	127.0352	116.6936
29	266.9861	185.4875	154.6865	137.3431	124.3082	114.2583
30	259.4658	181.1001	151.1291	134.3603	121.7170	111.9665

Proper functions are not listed as they ask to more space.

3.7.2 Spheroidal free oscillations

The same as the toroidal free oscillation problem, this is a proper value problem. The differential equations to be solved are

$$\left\{ \begin{array}{l} \frac{dy_1}{dr} = -\frac{2\lambda}{(\lambda+2\mu)r}y_1 + \frac{1}{\lambda+2\mu}y_2 + \frac{n(n+1)\lambda}{(\lambda+2\mu)r}y_3 \\ \frac{dy_2}{dr} = \left\{ -\rho\sigma^2 - \frac{4\rho g}{r} + \frac{4\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_1 - \frac{4\mu}{(\lambda+2\mu)r}y_2 + \\ \left\{ \frac{n(n+1)\rho g}{r} - \frac{2n(n+1)\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_3 + \frac{n(n+1)}{r}y_4 - \rho y_6 \\ \frac{dy_3}{dr} = -\frac{1}{r}y_1 + \frac{1}{r}y_3 + \frac{1}{\mu}y_4 \\ \frac{dy_4}{dr} = \left\{ \frac{\rho g}{r} - \frac{2\mu(3\lambda+2\mu)}{(\lambda+2\mu)r^2} \right\} y_1 - \frac{\lambda}{(\lambda+2\mu)r}y_2 + \\ \left\{ -\rho\sigma^2 + \frac{2\mu}{(\lambda+2\mu)r^2}[\lambda(2n^2+2n-1) + 2\mu(n^2+n-1)] \right\} y_3 - \\ \frac{3}{r}y_4 - \frac{\rho}{r}y_5 \\ \frac{dy_5}{dr} = 4\pi G\rho y_1 + y_6 \\ \frac{dy_6}{dr} = -\frac{4\pi G n(n+1)\rho}{r}y_3 + \frac{n(n+1)}{r^2}y_5 - \frac{2}{r}y_6 \end{array} \right. \quad (3.60)$$

where σ is the angular frequency of free oscillation to be computed as proper value. These equations are the same as in the problem for computing dynamic love numbers.

The boundary conditions at the Earth center and the boundary of discontinuity inside the Earth are the same as the case of the problem for computing dynamic love numbers

and are treated in the same way. The boundary conditions at the Earth surface are

$$(y_2)_{r=R} = 0, (y_4)_{r=R} = 0, (y_6)_{r=R} = -\frac{n+1}{R}(y_5)_{r=R} \quad (3.61)$$

When $n = 0$, $y_3 = y_4 \equiv 0$, the number of equations reduces to 4, and further more the solution for y_6 is $y_6 = 0$.

The computations are done as well by 4 functions which are in the file `spheroidal_snrei_odes_chebcoll.cpp`. They are

```
static int spheroidal_odes_chebcoll(real period,real &f);
static int spheroidal_odes_one_mode(int n,real period_start,
                                   real period_end,real step,real &period,real toll);
void spheroidal_snrei_odes_chebcoll(int n_low,int n_high,
                                   int number,real **period);
void spheroidal_snrei_odes_chebcoll_eif(int n,real period,real **y_Coll);
```

Their properties are similar to the corresponding ones as the of toroidal free oscillation problem.

The main function is in the file `spheroidal_snrei_odes.cpp`. It runs in a similar way as the one for toroidal free oscillation.

In this case the proper functions are normalized such that

$$\int_0^R \rho r^2 [y_1^2 + n(n+1)y_3^2] dr = 1 \quad (3.62)$$

which is often done in perturbation theory of free oscillations.

The result of some free oscillation periods are

Ocean layer averaged with the upmost crust layer

Free oscillation periods of the modes `m_S_n`:

n	m= 0	m= 1	m= 2	m= 3	m= 4	m= 5
0	1227.9863	612.9832	398.3004	305.6491	243.4856	204.6539
1	2475.8386	1059.5528	708.0227	583.3222	504.9865	449.7336
2	3233.3718	1471.4985	1065.9551	904.1868	580.6599	478.3276
3	2134.2815	1064.5080	805.3274	705.4135	488.0454	461.0821
4	1545.5688	852.9190	725.4762	545.3094	438.8574	420.2187
5	1190.0511	729.8475	660.6847	447.3996	414.8819	369.8487
6	963.3878	656.9917	595.5520	392.4113	380.5750	332.0653
7	812.0470	604.0450	536.7185	372.4669	331.7444	303.7907
8	707.7117	555.7785	488.4144	354.8824	294.3164	283.5861
9	633.8778	509.2297	449.0234	339.0457	269.6435	264.6442
10	579.5056	465.4472	416.4319	324.4324	258.8055	240.5055
11	537.2244	425.9618	388.9907	310.7144	249.4103	224.2442
12	502.6939	391.3632	365.4787	297.7287	240.8654	212.7865
13	473.5333	361.4817	344.9488	285.4138	232.8988	202.9052
14	448.3861	336.0206	326.4836	273.7564	225.3583	194.5241
15	426.4004	315.3730	308.5948	262.7580	218.1573	187.4973

16	406.9852	299.4842	290.4210	252.4169	211.2504	181.5068
17	389.6992	286.1493	273.4246	242.7216	204.6178	176.1967
18	374.1969	274.2719	258.1065	233.6498	198.2539	171.3238
19	360.2008	263.4829	225.1709	159.3068	129.8650	122.7299
20	347.4842	253.5913	231.9864	217.2489	153.6779	118.8051
21	335.8607	244.4690	220.8071	209.8450	193.8839	148.5352
22	325.1761	236.0197	210.6616	202.9199	175.5145	143.8743
23	315.3026	228.1670	196.4344	170.5055	150.5655	138.0321
24	306.1338	220.8483	192.9656	190.3466	165.7557	153.7457
25	297.5812	214.0111	185.2896	184.5461	161.2551	143.5058
26	289.5707	207.6106	179.3813	177.9291	156.9921	140.2154
27	282.0401	201.6079	174.3106	171.3292	152.9542	126.0040
28	274.9373	195.9691	169.5526	165.1874	149.1282	123.4050
29	268.2180	190.6643	165.0670	159.4705	145.5014	120.9168
30	261.8446	185.6666	160.8304	154.1376	116.2903	115.3705

The proper functions are not listed for the same reason as for the case of toroidal free oscillations.

Appendix A

Derivation of the formula (1.56)

The truncated Chebyshev series (1.48) is in fact a polynomial of degree N which equals $u(x_i)$ at the collocation points x_i for $i = 0, 1, \dots, N$. So it should also be obtained by polynomial interpolation:

$$I_N u = \sum_{j=0}^N I_j(x) u(x_j) \quad (\text{A.1})$$

where the Lagrangian interpolant

$$I_j(x) = \prod_{k=0, k \neq j}^N \frac{x - x_k}{x_j - x_k} \quad (\text{A.2})$$

is a N -th degree polynomial and satisfies

$$I_j(x_i) = \delta_{ij}, \quad i, j = 0, 1, \dots, N \quad (\text{A.3})$$

Now let's reduce $I_j(x)$ to a closed form. As the Gauss-Lobatto integration points are discussed here, it can be verified by using (1.38) that $x_0, x_1, \dots, x_N$ are solutions of the equation

$$(1 - x^2) \frac{dT_N(x)}{dx} = 0 \quad (\text{A.4})$$

As $(1 - x^2) \frac{dT_N(x)}{dx}$ is a polynomial of degree $N+1$, it can be written in the form of

$$(1 - x^2) \frac{dT_N(x)}{dx} = C_{N+1} \prod_{k=0}^N (x - x_k) \quad (\text{A.5})$$

where C_{N+1} is the coefficient of x^{N+1} . Hence $I_j(x)$ can be written in the form of

$$I_j(x) = C \frac{(1 - x^2) \frac{dT_N(x)}{dx}}{x - x_j} \quad (\text{A.6})$$

where C is a constant coefficient to be determined from (A.3). It is seen that (A.6) satisfies

$$I_j(x_i) = 0, \quad i \neq j \quad (\text{A.7})$$

So it remains to determine C using

$$I_j(x_j) = 1 \quad (\text{A.8})$$

By using L'Hopital's rule, it can be obtained that

$$I_j(x_j) = \begin{cases} -2C \lim_{x \rightarrow 1} \frac{dT_N(x)}{dx}, & j = 0 \\ C \lim_{x \rightarrow x_j} \frac{d}{dx} \left[(1-x^2) \frac{dT_N(x)}{dx} \right], & j = 1, \dots, N-1 \\ 2C \lim_{x \rightarrow -1} \frac{dT_N(x)}{dx}, & j = N \end{cases} \quad (\text{A.9})$$

Substituting (1.21) into it and applying L'Hopital's rule again, we can obtain

$$I_j(x_j) = -N^2 \bar{c}_j C T_N(x_j) = (-1)^{j+1} N^2 \bar{c}_j C \quad (\text{A.10})$$

where use is made of the formula $T_N(x_j) = (-1)^j$ which is easily obtained by substituting (1.38) into (1.21). From this equation we obtain

$$C = \frac{(-1)^{j+1}}{N^2 \bar{c}_j} \quad (\text{A.11})$$

Hence the final closed form of the interpolant is

$$I_j(x) = \frac{(-1)^{j+1} (1-x^2) \frac{dT_N(x)}{dx}}{N^2 \bar{c}_j (x-x_j)} \quad (\text{A.12})$$

By using further (1.21) and L'Hopital's rule, it can be finally shown that

$$(D_N)_{lj} = \left[\frac{dI_j(x)}{dx} \right]_{x=x_i} = \begin{cases} \frac{\bar{c}_i}{\bar{c}_j} \frac{(-1)^{i+j}}{x_i - x_j}, & i \neq j, \\ -\frac{x_i}{2(1-x_i^2)}, & i = j = 1, \dots, N-1 \\ \frac{2N^2+1}{6}, & i = j = 0 \\ -\frac{6}{2N^2+1}, & i = j = N \end{cases} \quad (\text{A.13})$$

which is the formula to be proven.

Bibliography

- [1] Canuto C, Hussaini M Y, Quarteroni A, Zang T A. Spectral Methods in Fluid Dynamics. New York: Springer-Verlag, 1987
- [2] Crossley D J. The Free Oscillation Equations at the Center of the Earth. Geophys. J. R. Astr. Soc., 1975, 41: 153~163
- [3] Dziewonski A M, Anderson D L. Preliminary Reference Earth Model. Phys. Earth Planet. Inter., 1981, 25: 297~356
- [4] Dziewonski A M, Hales A L, Lapwood E R. Parametrically Simple Earth Model Consistent with Geophysical Data. Phys. Earth Planet. Inter., 1975, 10: 12~48
- [5] Gilbert F, Dziewonski A M. An Application of Normal Mode Theory to the Retrieval of Structural Parameters and Source Mechanisms from Seismic Spectra. Philos. Trans. R. Soc. London, Ser. A, 1975, 278: 187~269
- [6] Guo J. Introduction to Geophysics (in Chinese). Beijing: Press of Surveying and Mapping, 2000
- [7] Guo J, Huang J, Zheng F. Computation of the Earth's Internal Flattening and Some Other Physical Parameters (in Chinese). Journal of Wuhan Technical University of Surveying and Mapping, 1996, 21(2): 109~114
- [8] Guo J, Zhang F. Removing the Singularity of Earth's Free Oscillation Ordinary Differential Equations at the Earth Center by Chebyshev Collocation Method (in Chinese). Journal of Wuhan Technical University of Surveying and Mapping, 1996, 21(4): 320~322
- [9] Lapwood E R, Usami T. Free Oscillation of the Earth. London: Cambridge University Press, 1981
- [10] Peyret R. Introduction to Spectral Methods with Application to Fluid Mechanics. Nice: Von Karman Institute for Fluid Dynamics, Université de Nice, 1986
- [11] Press W H, Teukolsky S A, Vetterling W T, Flannery B P. Numerical Recipes in C: The Art of Scientific Computing. Second Edition. London: Cambridge University Press, 1992
- [12] Smylie D E, Szeto M K, Sato K. Elastic Boundary Conditions in Long-Period Core Oscillation. Geophys. J. Int., 1990, 100: 308~319

ISBN 7-81030-798-3



9 787810 307987 >

ISBN 7-81030-798-3/P · 71
定价：60.00元